



Algorithms Design and Analysis



الفصل الثاني ٢٠٢٢/٢٠٢٣

د. حسن المعيدي



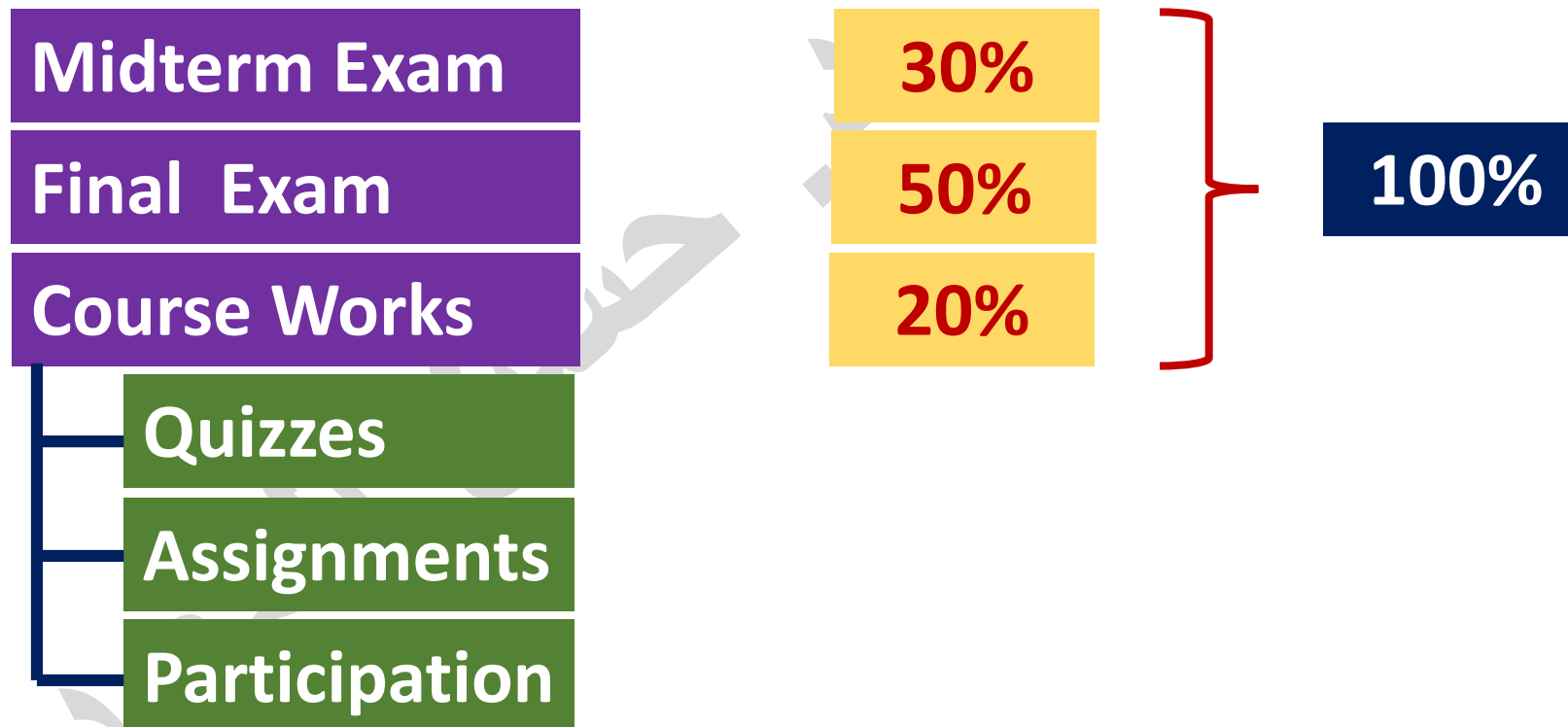
0

Course Information

Course Information

- **Instructors** : **Dr. Hasan Muaidi**
- **Office hours** :
- **Website** : <https://s3.ebalqa.courses/nhusonmoodle/login/index.php>
- Lecture material, course announcements and assignments will be posted.
 - It is the responsibility of the student to login regularly to Moodle and check for all course updates and requirements.

Course Evaluation





Assignment Submission Policy

- Assignments are to be done individually.
- You should not give your solution to other students.
- Students who copy homework will receive a **zero** and those students letting another copy will also receive a **zero**.
- Assignments are due on the day that they are due.
- Do not leave submission for the last minute.
- You should add a cover sheet to your assignment.
- No assignment will be graded until the assignment's cover sheet has been submitted.
- You should submit your assignment as a one pdf file.
- Only files with pdf extensions are allowed; else you will assign a **zero**.
- The compressed files are not allowed.
- All submissions will be electronic via the university e-learning system.

Attendance Policy

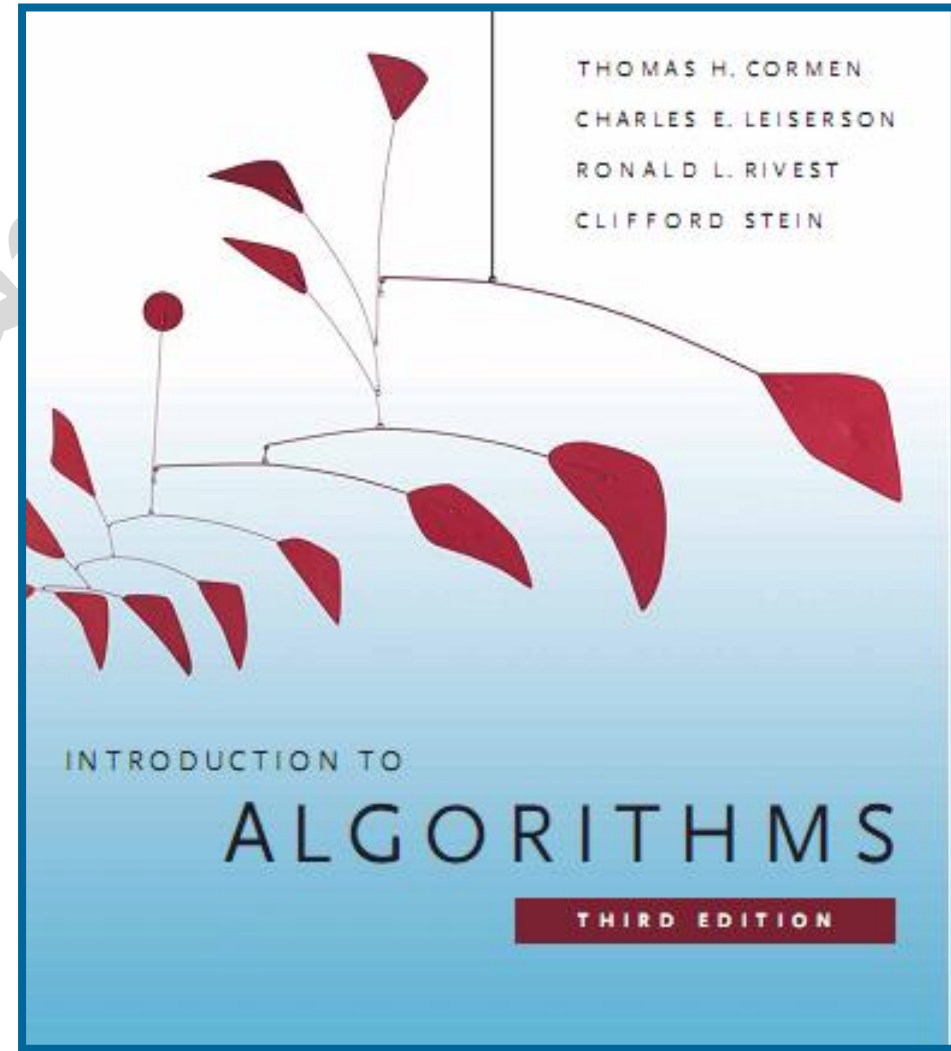
- Attendance is required for this class.
- Students attendance is recorded and monitored through the [MS-Teams](#) from the beginning to the end of each class lecture.
- Absence of 10% of the total class lectures will result in a first warning.
- Absence of 15% or more of the total class lectures will result in forfeiting the course and the student will not be permitted to attend the final examination.
- An excused absence is considered only when the student notifies the instructor on the day, or prior to the day, of the absence.

Textbook

Introduction To Algorithms

Thomas H. Cormen
Charles E. Leiserson
Ronald L. Rivest
Clifford Stein

Massachusetts Institute of
Technology
3rd Edition
2009





Syllabus

1- Introduction

2- The Efficiency of Algorithms

3- Recursive Algorithms

4- Divide-And-Conquer Algorithms

5- Graph Algorithms



Important Dates

Deadline to Dropout the Course

24-05-2023

Midterm Exam

15-04-2023

30-04-2023

Final Exam

10-06-2023

22-06-2023

Last Day For Teaching

08-06-2023

Summer Semester

16-07-2023



1

Introduction

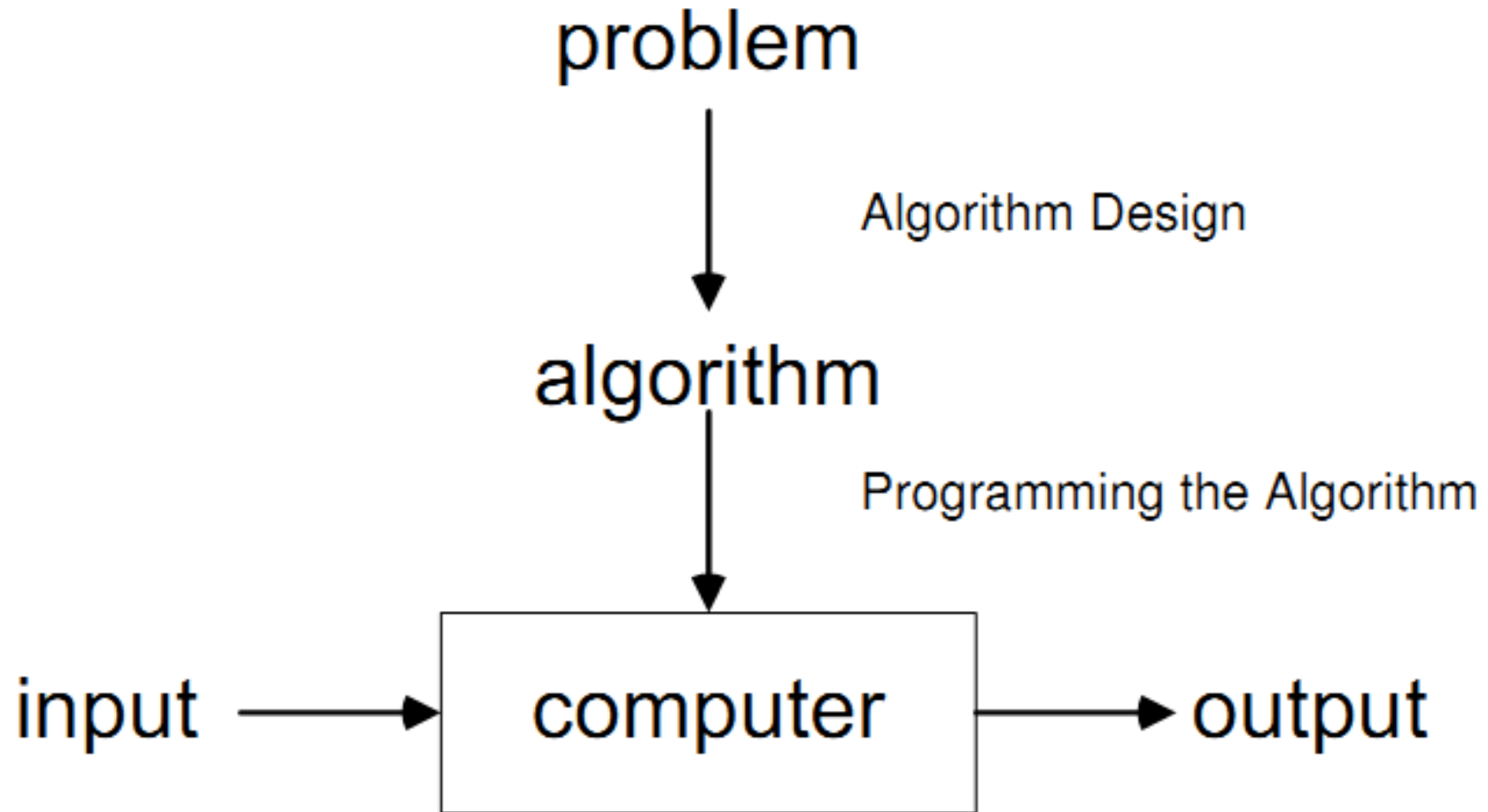
Chapter Learning Outcomes

CLO

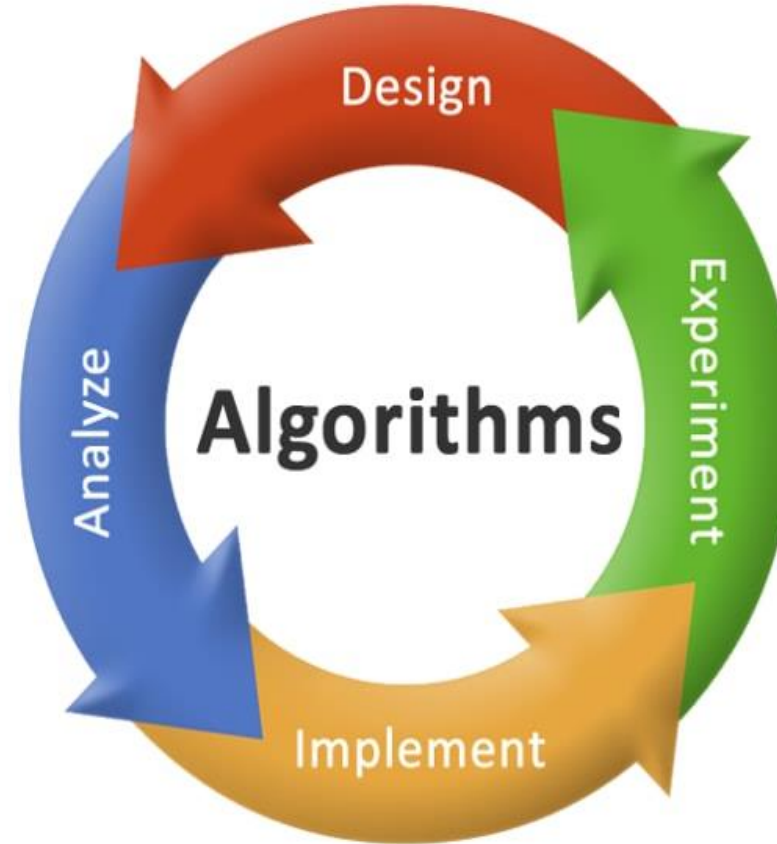
1

- Define the meaning of algorithms.
- Understand why we study algorithms?
- Understand and know some algorithm proprieties.
- Understand and know how to represent algorithms.
- Write algorithms to solve some practical problems.

Introduction



Phases

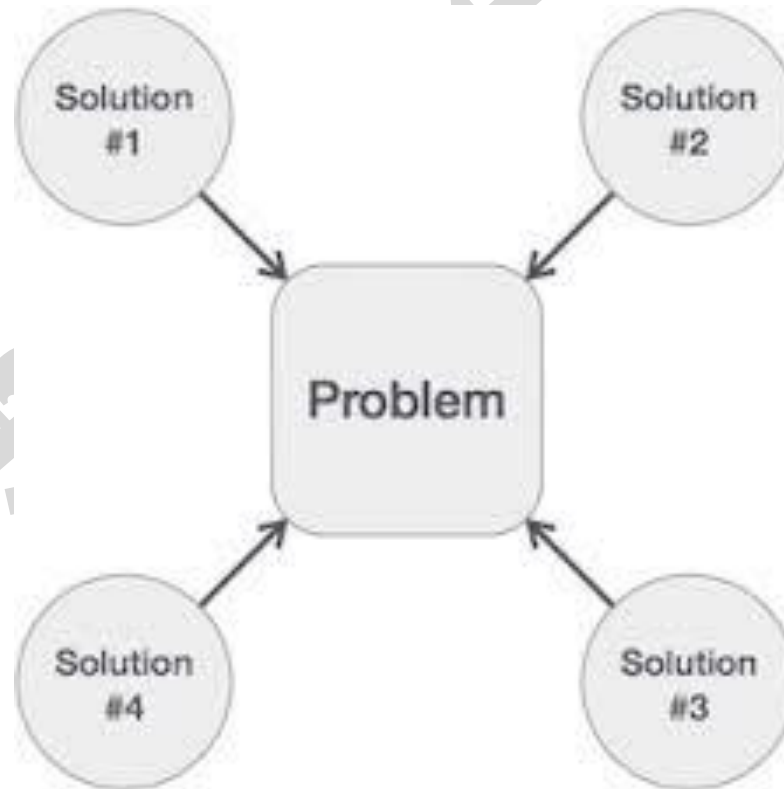


Why We Study Algorithms?

The programming process is a complicated one. You must first understand the program specifications, of course, and then you need to organize your thoughts and create the program. This is a difficult task when the program is not easy. You must break the main tasks that must be accomplished into smaller ones in order to be able to eventually write fully developed code. **Writing algorithm will save you time later during the construction & testing phase of a program's development.**

Why We Study Algorithms?

If two or more algorithms solve the same problem, which one should we choose...?



Why We Study Algorithms?

Algorithm efficiency is an important attribute in comparing one algorithm to another for the same task.

Example: Traveling Salesman Problem (TSP)

Suggest a solution (an algorithm) for the following problem:

1. Starting from city x , the salesman must travel to all cities before returning to home city.
2. The distance between every pair of cities is given.
3. The salesman must visit every city exactly once.
4. Objective: **Minimize the total distance to be traveled.**



Example: Traveling Salesman Problem (TSP)

5 Cities

From city

To city

	A	B	C	D	E
A	α	2	5	7	1
B	6	α	3	8	2
C	8	7	α	4	7
D	12	4	6	α	5
E	1	3	2	8	α

Example: Traveling Salesman Problem (TSP)

Brute Force Solution

1. Consider city (A) as the starting and ending point. Since route is cyclic, we can consider any point as starting point.
2. Generate all $(n-1)!$ permutations of cities.
3. Calculate cost of every permutation and keep track of minimum cost permutation.
4. Return the permutation with minimum cost.

Example: Traveling Salesman Problem (TSP)

Brute Force Solution

- List all possible solutions for a problem and pickup the best one.

Find the number of permutations to arrange **5** cities.

- $P(5,5) = 5! = 120$.

Example: Traveling Salesman Problem (TSP)

1-	$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$	15
2-	$A \rightarrow B \rightarrow C \rightarrow E \rightarrow D \rightarrow A$	32
3-	$A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow A$	24
4-	$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow A$	25
.		
.		
120-	$E \rightarrow D \rightarrow C \rightarrow B \rightarrow A \rightarrow E$	28

	A	B	C	D	E
A	α	2	5	7	1
B	6	α	3	8	2
C	8	7	α	4	7
D	12	4	6	α	5
E	1	3	2	8	α

Example: Traveling Salesman Problem (TSP)

Assume the brute-force method takes 3 second to find the minimum cost for 5 cities for TSP. Approximate the time if the number of cities is 15.

- $5! = 120 \rightarrow 3 \text{ second}$
- $15! = 1,307,674,368,000 \rightarrow ? \text{ second}$
- $15! = 1,307,674,368,000 \rightarrow 32,691,859,200 \text{ second}$

Example: Traveling Salesman Problem (TSP)

- $15! = 1,307,674,368,000 \rightarrow 32,691,859,200$ Seconds



Algorithm

The word algorithm comes from the name of Persian author, "Abu Ja'far Mohammed Ibn Mousa Al Khawarizmi (825 A.D.)"



Informal Definition

- An algorithm is a list that looks like
 - **STEP 1:** Do something
 - **STEP 2:** Do something
 - **STEP 3:** Do something
 - . . .
 - . . .
 - . . .
 - **STEP n:** Stop, you are finished

Formal Definition

An algorithm is an order sequence of instructions that is guaranteed to solve a specific problem.

حسن المعيني



Algorithm Proprieties

1. Correctness
2. Produce Result
3. Finiteness
4. Well-Ordered
5. Unambiguous Instructions (Primitive)
6. Effectiveness and Generality



Algorithm Proprieties

1- Correctness : An algorithm should produce the correct output for a given set of input values.

د. حسن المعيني

Algorithm Proprieties

2- Produce Result : An algorithm should produce an output after a finite number of steps.

```
1. Sum = 0
2. For i = 1 to 20 Do
3.     Sum = Sum + i
4. End For
```

```
1. Sum = 0
2. For i = 1 to 20 Do
3.     Sum = Sum + i
4. End For
5. Print Sum
```

Algorithm Proprieties

3- Finiteness : An algorithm must contain a finite number of steps.

```
1.  $n = 1$   
2. While  $n \leq 100$  Do  
3.     Print  $n$   
4. End While
```

```
1.  $n = 1$   
2. While  $n \leq 100$  Do  
3.     Print  $n$   
4.      $n = n + 1$   
5. End While
```

Algorithm Proprieties

4- Well-Ordered : We must know the correct order in which to execute the instructions.

1. Read a
2. Read b
3. If $b=0$ Then *Return Back*
4. Print a/b

1. Read a
2. Read b
3. If $b=0$ Then *Return Back to Step 2*
4. Print a/b

Algorithm Proprieties

5- Unambiguous Instructions (Primitive) : Each operation in an algorithm must be sufficiently clear so that it does not need to be simplified.

Which of the following are primitive operations for a computer?

- Add x and y to get the sum z
- See whether x is greater than, equal to, or less than y
- Sort a list of names into alphabetical order
- Factor an arbitrary integer into all of its prime factors
- Make a cherry pie

Algorithm Proprieties

6- Effectiveness and Generality : The algorithm should be applicable for all possible input values.

```
1. Read  $a$ 
2. Read  $b$ 
3.  $c = a + b$ 
4. Print  $c$ 
5.  $d = a/b$ 
6. Print  $d$ 
```

```
1. Read  $a$ 
2. Read  $b$ 
3.  $C = a + b$ 
4. Print  $c$ 
5. If  $b=0$  then
6. {
7.   print  $b$  should not equal zero
8. }
9. Else
10. {
11.    $d = a/b$ 
12.   Print  $d$ 
13. }
```

Operations in an Algorithm

- Sequential operations.
- Conditional operation.
- Iterative operations.

Representing Algorithms

How to express algorithms **clear**, **precise**, and **unambiguous** ?

We can use the **pseudocode** to express algorithms

Pseudocode

- **Pseudocode** is an informal language that helps programmers develop algorithms.
- **Pseudocode** focuses on the logic of the algorithm or program. It avoids language-specific elements, such as variable declarations, import and include statements, ...
- No **pseudocode** syntax exists.

Example

Write an algorithm using pseudocode to find the largest element in an unsorted list of numbers.

ALGORITHM MAX_ELEMENT(A, N)

1. $max = A[1]$
2. **For** $i = 2$ **to** N **Do**
3. **If** $A[i] > max$ **Then**
4. $max \leftarrow A[i]$
5. **End If**
6. **End For**
7. **Print** max

Example

Write an algorithm using pseudocode to get two numbers from the user (dividend and divisor), testing to make sure that the divisor number is not zero, and displaying their quotient

ALGORITHM QUOTIENT()

1. **Read** *dividend, divisor*
2. **If** *divisor = 0* **Then**
3. **Print** “divisor must be non-zero”
4. **Exit Algorithm**
5. **Else**
6. $quotient = dividend/divisor$
7. **Print** *quotient*
8. **End If**

Example

Write an algorithm using pseudocode to print the factorial of a number n entered by user.

ALGORITHM FACTORIAL()

1. Read n
2. $Fact = 1$
3. For $i = 1$ to n Do
4. $Fact = Fact * i$
5. End For
6. Print $Fact$

Exercise

Write an algorithm using pseudocode for the following problems:

1

Print the product of numbers from 1 to n where n is provided by user.

2

Print the sum of numbers from 1 to n where n is provided by user.

3

Find the smallest element in a 1D array of n elements.

4

Find the largest element in a 1D array of n elements.

Exercise

5

Print the factorial of a number n entered by user.

6

Read a number in feet, converts it to meters, and displays the result. One foot is 0.305 meter.

7

Prompt the user to enter two points (x_1, y_1) and (x_2, y_2) and displays the distance between them.

Exercise

8

Prompt the user to enter two integers and checks whether the first number is divisible by the second.

9

Prompt the user to enter an integer and determines whether it is divisible by 5 and 6, whether it is divisible by 5 or 6, and whether it is divisible by 5 or 6, but not both.

10

Read an unspecified number of integers, determines how many positive and negative values have been read, and computes the total and average of the input values (not counting zeros).

Exercise

11

Display all the numbers from 1 to 500 that are divisible by 3 and 4.

12

Calculate the product of the following series.

$$\frac{1}{5} \times \frac{1}{9} \times \frac{1}{13} \times \cdots \times \frac{1}{97}$$

Exercise

13

Calculate the product of the following series.

$$\frac{1}{3} \times \frac{1}{8} \times \frac{1}{13} \times \cdots \times \frac{1}{118}$$

14

Read a number n and displays the next ten even numbers following it.

15

Read a number n and displays the next ten odd numbers following it.

Exercise

16

Display all the numbers from 2 to 20 and determine if it is a prime number or not.

Number	Type
2	Prime
3	Prime
4	Not Prime
5	Prime
...	
20	Not Prime

17

Count the number of occurrences of a given number in a given array of n integers.

Exercise

18

Sort 10 integer values (reading from keyboard) in ascending order.

19

Sort 10 integer values (reading from keyboard) in descending order.

20

Search an unsorted array (A) for a given value (x).

Check - Chapter Learning Outcomes

CLO

1

- Define the meaning of algorithms.
- Understand why we study algorithms?
- Understand and know some algorithm proprieties.
- Understand and know how to represent algorithms.
- Write algorithms to solve some practical problems.

End of Chapter

1



2

The Efficiency of Algorithms

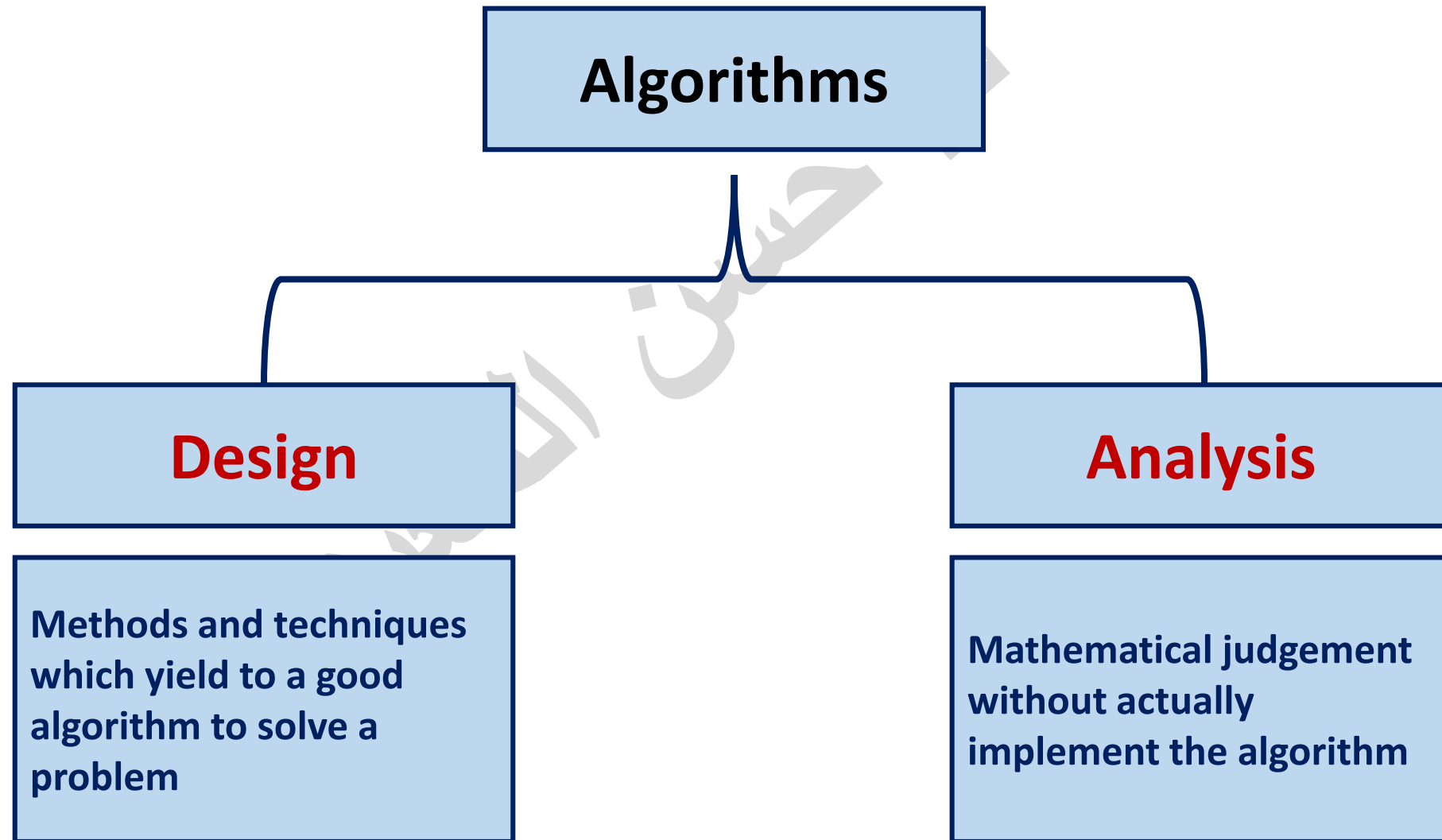
Chapter Learning Outcomes

CLO

2

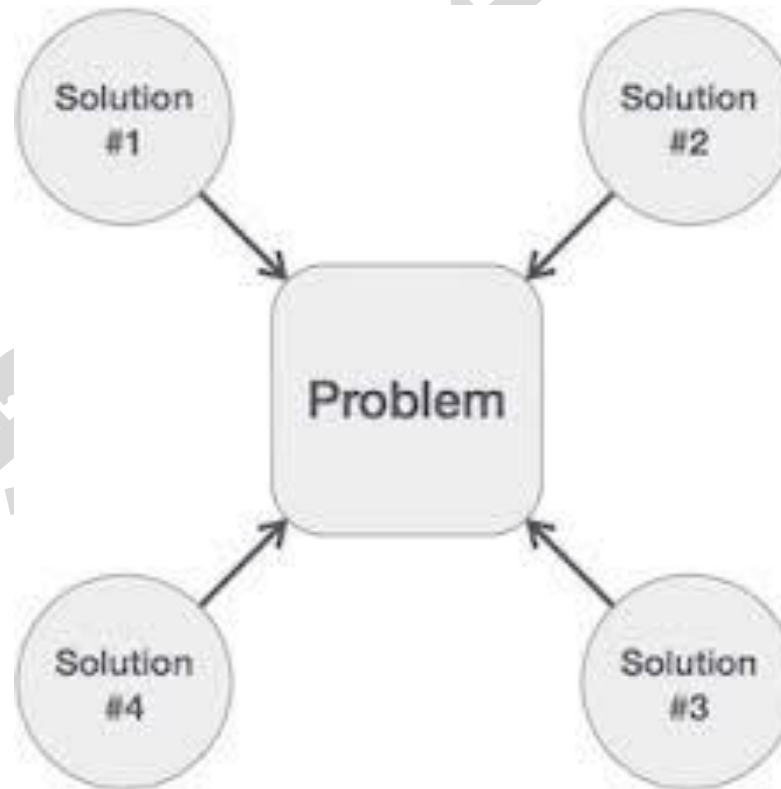
- Distinguish between design and analysis phases.
- Understand the meaning of order of magnitude.
- Analyze the complexity of Linear search algorithm.
- Analyze the complexity of Insertion-sort algorithm.
- Analyze the complexity of Selection-sort algorithm.
- Characterize algorithms with respect to asymptotic growth.
- Apply asymptotic notation to describe the behavior of algorithms in best, average and worst cases.

Study of Algorithms



The Efficiency of Algorithms

If two or more algorithms solve the same problem, which one should we choose...?



The Efficiency of Algorithms

Time Efficiency

The number of steps required by the algorithm to solve the problem as a function of input size $T(n)$.



TIME EFFICIENCY

Space Efficiency

The amount of memory the algorithm uses as a function of input size $S(n)$.

STORAGE SPACE
EFFICIENCY
GUIDE



Order of Magnitude

Complexity Analysis

- Is the systematic study of the cost of a computation, measured either in time units or in operations performed, or in the amount of storage space required.
- The goal is to have a meaningful measure that permits comparison of algorithms and/or implementations independent of operating platform.

Complexity Analysis

The analysis of an algorithm focuses on time and space complexity.

- The space complexity refers to the amount of memory required by an algorithm to run to completion.
- The time complexity is the amount of time required by an algorithm to run to completion.

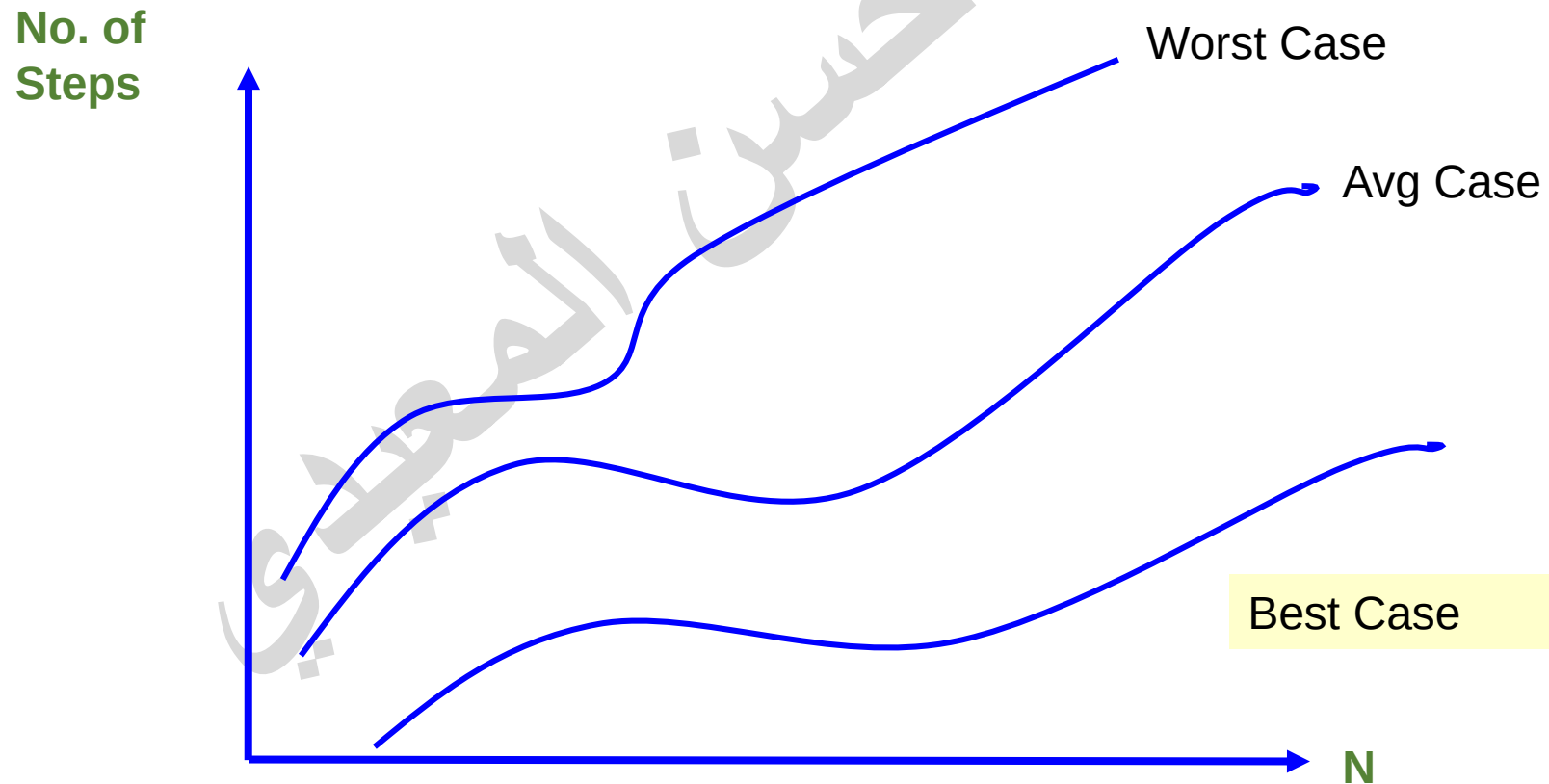
Complexity Analysis

Complexity analysis involves two phases:

- Algorithm analysis: analysis of the algorithm or data structure to produce a function $T(n)$ measuring the complexity.
- Order of magnitude analysis: analysis of the function $T(n)$ to determine the general complexity category to which it belongs.

Time Complexity

Different times can arise for the same algorithm.



Time Complexity

- **Worst-Case Time Complexity**

Is the function defined by the maximum amount of time needed by an algorithm for an input of size “n”.

- **Average-Case Time Complexity**

Is the execution of an algorithm having typical input of size “n”.

- **Best-Case Time Complexity**

Is the function defined by the minimum amount of time needed by an algorithm for an input size “n”.

Example : Sequential Search

EXAMPLE

Given a list of 10 elements, find a specific element in the list. List does NOT have to be sorted. Analyze the time of your work in (best, average, worst cases); assume each comparison needs 8ms.

4	1	7	0	5	2	9	3	6	8
---	---	---	---	---	---	---	---	---	---

Sequential Search – Best Case

SOLUTION

4	1	7	0	5	2	9	3	6	8
---	---	---	---	---	---	---	---	---	---

When ?

No. of comparisons :

Total time :

Sequential Search – Average Case

SOLUTION

4	1	7	0	5	2	9	3	6	8
---	---	---	---	---	---	---	---	---	---

When ?

No. of comparisons :

Total time :

Sequential Search – Worst Case

SOLUTION

4	1	7	0	5	2	9	3	6	8
---	---	---	---	---	---	---	---	---	---

When ?

No. of comparisons :

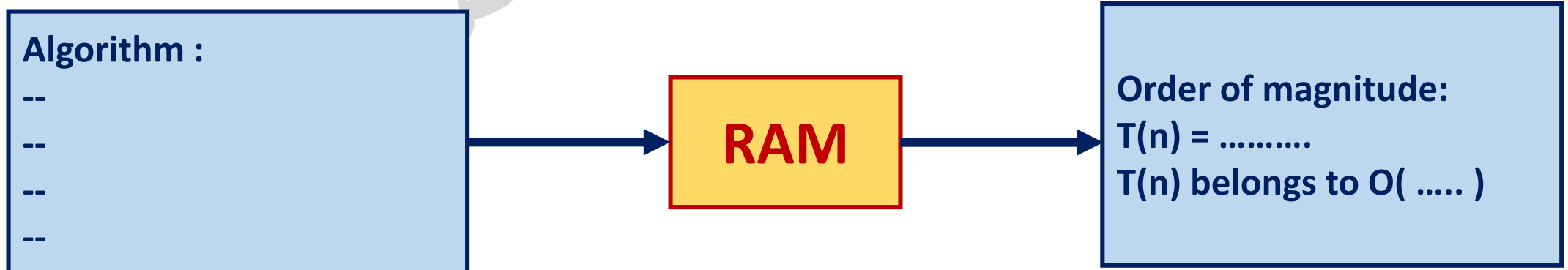
Total time :

Order of Magnitude

- Not interested in knowing the **exact** number of steps the algorithm performs.
- Interested in knowing how the number of steps **grows** with increased input size (n).
- **Order of magnitude** - $O(\dots)$ – measures how the number of steps grows with input size (n).

Random-Access Machine (RAM)

- To analyze the running time complexity, we need a computational model of a computer.
- This model is called Random-Access Machine (RAM).



Random-Access Machine (RAM)

- Memory consists of an infinite array of cells.
- Each cell can store at most one data item (bit, byte, a record, ..).
- Any memory cell can be accessed in unit time.
- Instructions are executed sequentially.
- All basic instructions take unit time:
 - Load/Store Operations (= , ...)
 - Arithmetic Operations (+, -, *, /, ^,)
 - Logic Operations (>, <, ==, !=,)
- Running time of an algorithm is the number of RAM instructions it executes

Random-Access Machine (RAM)

RAM model is not realistic, e.g.

- Memory is finite (even though we often imagine it to be infinite when we program).
- Not all memory accesses take the same time (cache, main memory, disk).
- Not all arithmetic operations take the same time (e.g. multiplications are expensive)

Rules For Finding Magnitude

- Rule 1: consecutive (sequential) statements
 - Add each statement
- Rule 2: if-then-else
 - test + if , where if > else
otherwise, test + else
- Rule 3: loops
 - test * iterations
- Rule 4: Nested Loops
 - iterations * iterations

Examples

Algorithm Analysis – Example [1]

EXAMPLE

Analyze the following algorithm-code.

[1]		
Get x, y		
Compute S = x+y		
Compute M = x*y		
Display S		
Display M		

Algorithm Analysis – Example [2]

EXAMPLE

Analyze the following algorithm-code.

[2]		Time
Assign $S = 0$ Get n For $i = 1$ to n Do Compute $S = S + i$ End For Compute $S = S / n$ Display S		

Algorithm Analysis – Example [3]

EXAMPLE

Analyze the following algorithm-code.

[3]

```
Get n
Assign S = 0
For j = 1 To n Do
    For k = 1 To n Do
        Compute S = S + j + k
    End For
End For
Display S
```


Algorithm Analysis – Example [4]

EXAMPLE

Analyze the following algorithm-code.

[4]

```
Get n
Assign S = 0
For j = 1 To n Do
    For k = 1 To j Do
        Compute S = S + j + k
    End For
End For
Display S
```

Algorithm Analysis – Example [5]

EXAMPLE

Analyze the following algorithm-code.

[5]

```
Get n
Assign S = 0
For j = 1 To n Do
    For k = j To n Do
        Compute S = S + j + k
    End For
End For
Display S
```

Algorithm Analysis – Example [6]

EXAMPLE

Analyze the following algorithm-code.

[6]

Suppose that the function $\text{Bar}(n)$ requires $\Theta(n^2)$.

Assign $x = 0$

For $i = 1$ To n Do

 Compute $x = x + \text{Bar}(i)$

End For

Display x

Algorithm Analysis – Example [7]

EXAMPLE

Analyze the following algorithm-code.

<pre>For i = 2 To 20 Do For j = 5 To 30 Do Statement End For For k = 4 To 27 Do Statement End For Statement End For Statement</pre>		

Algorithm Analysis – Example [8]

EXAMPLE

Analyze the following algorithm-code.

```
For i = 1 To n+8 Do
{
  For j = 1 To i Do
  {
    sum = sum + j ;
  }
  For k = 3 To 5n Do
  {
    c[k] = c[k] + sum;
  }
}
```

Algorithm Analysis – Example [9]

EXAMPLE

Analyze the following algorithm-code.

```
For i = 3 To 60 Do
{
  m = 1
  For j = i To 50 Do
  {
    m = m * j
  }
  Print m
}
```

Algorithm Analysis – Example [10]

EXAMPLE

Analyze the following algorithm-code.

[7]		
Assign $S = 0$ Get n While ($n > 1$) Do Compute $S = S + n$ Compute $n = n - 1$ End While Display S		

Algorithm Analysis – Example [11]

EXAMPLE

Analyze the following algorithm-code.

[7]		
Assign $S = 0$ Get n While ($n > 1$) Do Compute $S = S + n$ Compute $n = n / 2$ End While Display S		

Algorithm Analysis – Example [12]

EXAMPLE

Analyze the following algorithm-code.

[8]		
Assign $S = 0$ Get n Do Compute $S = S + n$ Compute $n = n - 1$ While ($n > 1$) Display S		

Algorithm Analysis – Example [13]

EXAMPLE

Analyze the following algorithm-code.

[9]

```
Assign S = 0
Get n
Do
    Compute S = S + n
    Compute n = n / 3
While (n > 1)
Display S
```

Algorithm Analysis – Example [14]

EXAMPLE

Analyze the following algorithm-code.

[10]

```
Get n
Assign S = 0
For j = 1 To 2n Do
    Increment S
End For
Display S
```

Algorithm Analysis – Example [15]

EXAMPLE

Analyze the following algorithm-code.

[11]

```
Get n
Assign S = 0
For j = 1 To 200 Do
    Increment S
End For
Display S
```

Algorithm Analysis – Example [16]

EXAMPLE

Analyze the following algorithm-code.

[12]

```
Get n
Assign S = 0
For j = 1 To 3n Do
    For k = j To n Do
        Increment S
    End For
End For
Display S
```

Algorithm Analysis – Example [17]

EXAMPLE

Analyze the following algorithm-code.

[13]

```
Get n
Assign S = 0
For j = 1 To 3n Do
    For k = 1 To  $\lfloor j/2 \rfloor$  Do
        Increment S
    End For
End For
Display S
```

Algorithm Analysis – Example [18]

EXAMPLE

Analyze the following algorithm-code.

[14]

```
Get n
Assign S = 0
For j = 1 To n Do
    For k = 1 To  $\lfloor j/2 \rfloor$  Do
        Increment S
    End For
End For
Display S
```

Algorithm Analysis – Example [19]

EXAMPLE

Analyze the following algorithm-code.

[15]

```
Get n
Switch (n % 2)
  Case 0 : Increment n : Break
  Case 1 : Decrement n
End Switch
Display n
```


Algorithm Analysis – Example [20]

EXAMPLE

Analyze the following algorithm-code.

[16]

```

Get n
Assign S = 0
If n is even Then
    For j = 1 To 3n Do
        For k = 1 To n Do
            Increment S
        End For
    End For
Else
    For j = 1 To 3n Do
        For k = 1 To n2 Do
            Increment S
        End For
    End For
Display S
    
```

Analysis of Two Sorting Algorithms

Algorithm Analysis – Sorting Algorithms

- **Sort Array A using:**

- Selection-Sort Algorithm
- Insertion-Sort Algorithm.

- Write the pseudocode for each algorithm in your own words.
- Analyze your works by counting the number of comparisons and copying.
- Express the running time for each algorithm using Θ -notation.



Analysis of Sorting Algorithms– Wikipedia

https://en.wikipedia.org/wiki/Sorting_algorithm

Analysis of Selection Sort Algorithm (Worst-Case)

Analysis of Selection-Sort Algorithm

Idea:

- Select the smallest item and put it in its correct place.
- Select the next smallest item and put it in its correct place and son on.

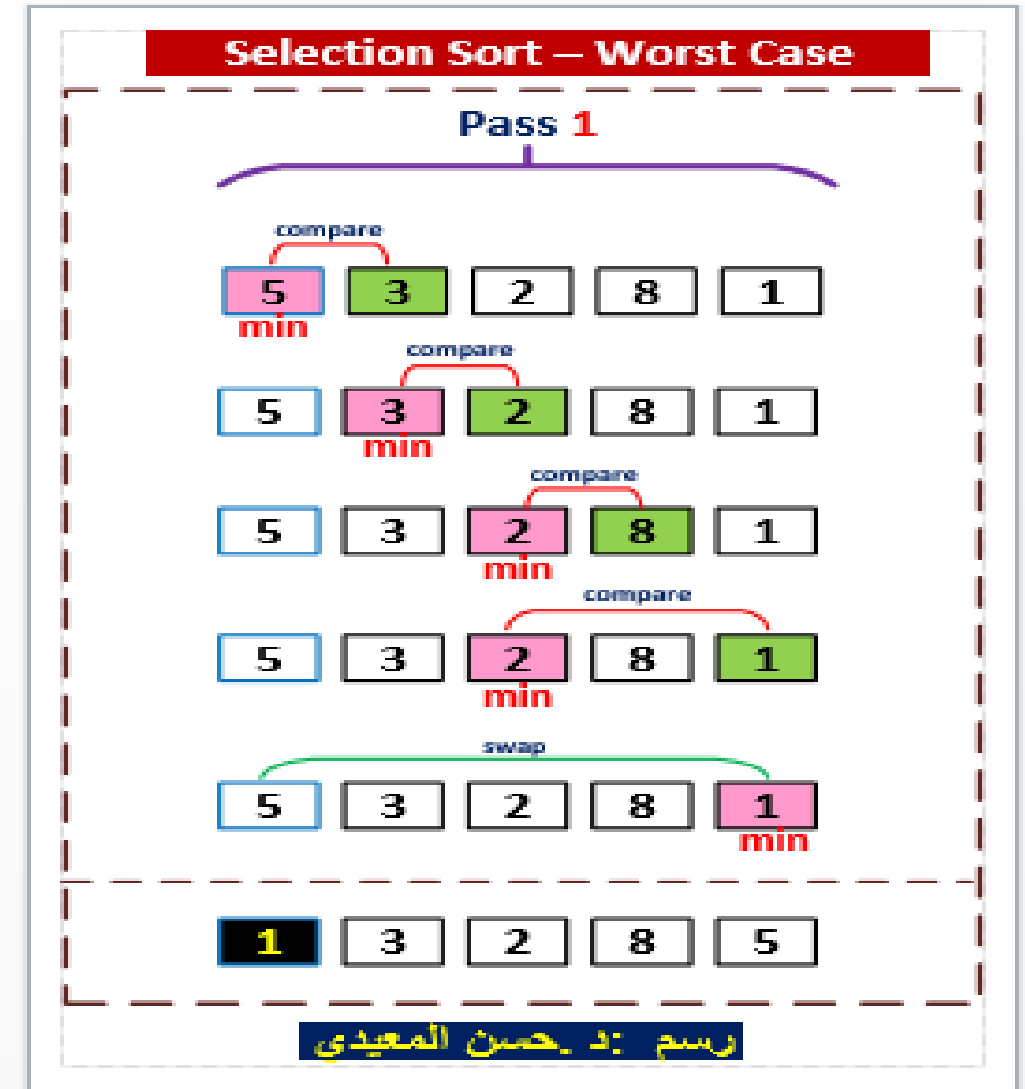
Selection-Sort Algorithm

```
1  def selectionSort(nlist):
2      for fillslot in range(len(nlist)-1,0,-1):
3          maxpos=0
4          for location in range(1,fillslot+1):
5              if nlist[location]>nlist[maxpos]:
6                  maxpos = location
7
8          temp = nlist[fillslot]
9          nlist[fillslot] = nlist[maxpos]
10         nlist[maxpos] = temp
11
12     nlist = [14,46,43,27,57,41,45,21,70]
13     selectionSort(nlist)
14     print(nlist)
```

Analysis of Selection-Sort Algorithm (Worst-Case)

Compare

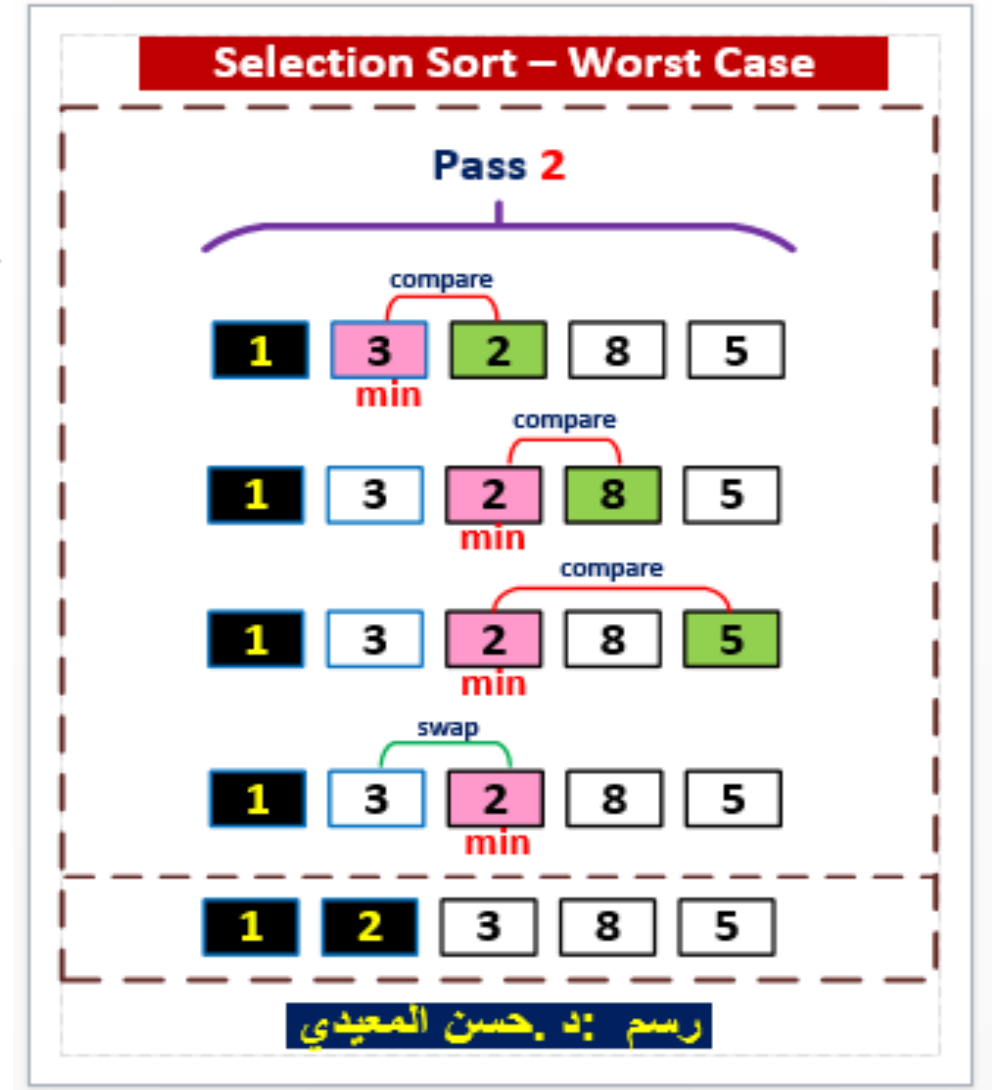
Copy



Analysis of Selection-Sort Algorithm (Worst-Case)

Compare

Copy



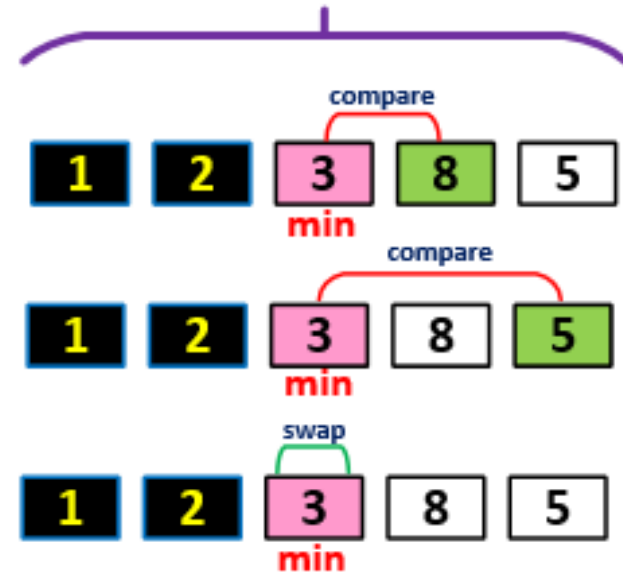
Analysis of Selection-Sort Algorithm (Worst-Case)

Compare

Copy

Selection Sort – Worst Case

Pass 3



1 2 3 8 5

رسم: د. حسن المعيدي

Analysis of Selection-Sort Algorithm (Worst-Case)

Compare

Copy



Analysis of Selection-Sort Algorithm (Worst-Case)

Pass No.	Comparison	Copy
1	4	1
2	3	1
3	2	1
4	1	1
Total		

Analysis of Selection-Sort Algorithm (Worst-Case)

General Equation (if the input size is n)

No. Of Passes

Time to Compare

Time to Copy

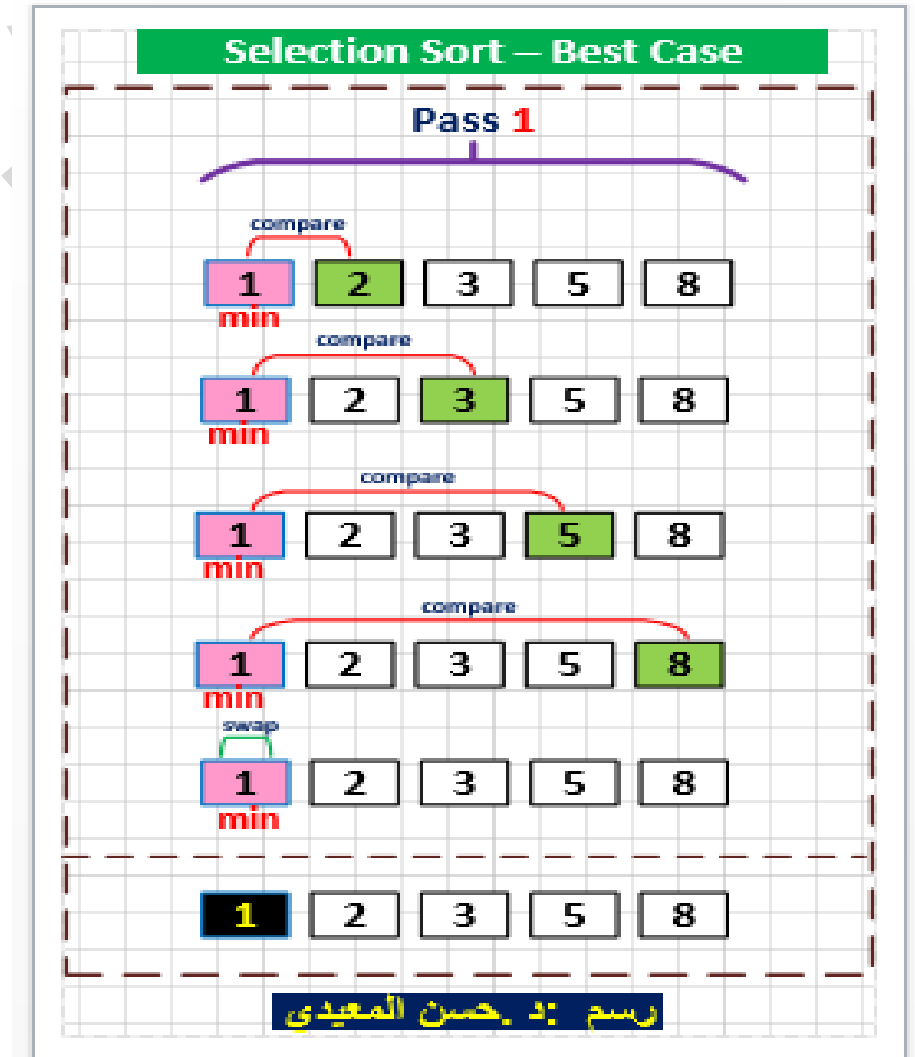
Total Time

Analysis of Selection Sort Algorithm (Best-Case)

Analysis of Selection-Sort Algorithm (Best-Case)

Compare

Copy



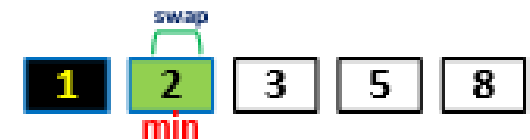
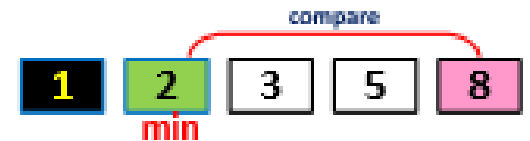
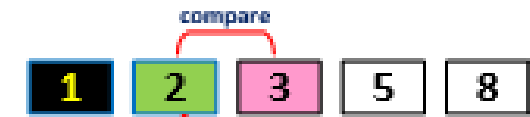
Analysis of Selection-Sort Algorithm (Best-Case)

Compare

Copy

Selection Sort – Best Case

Pass 2

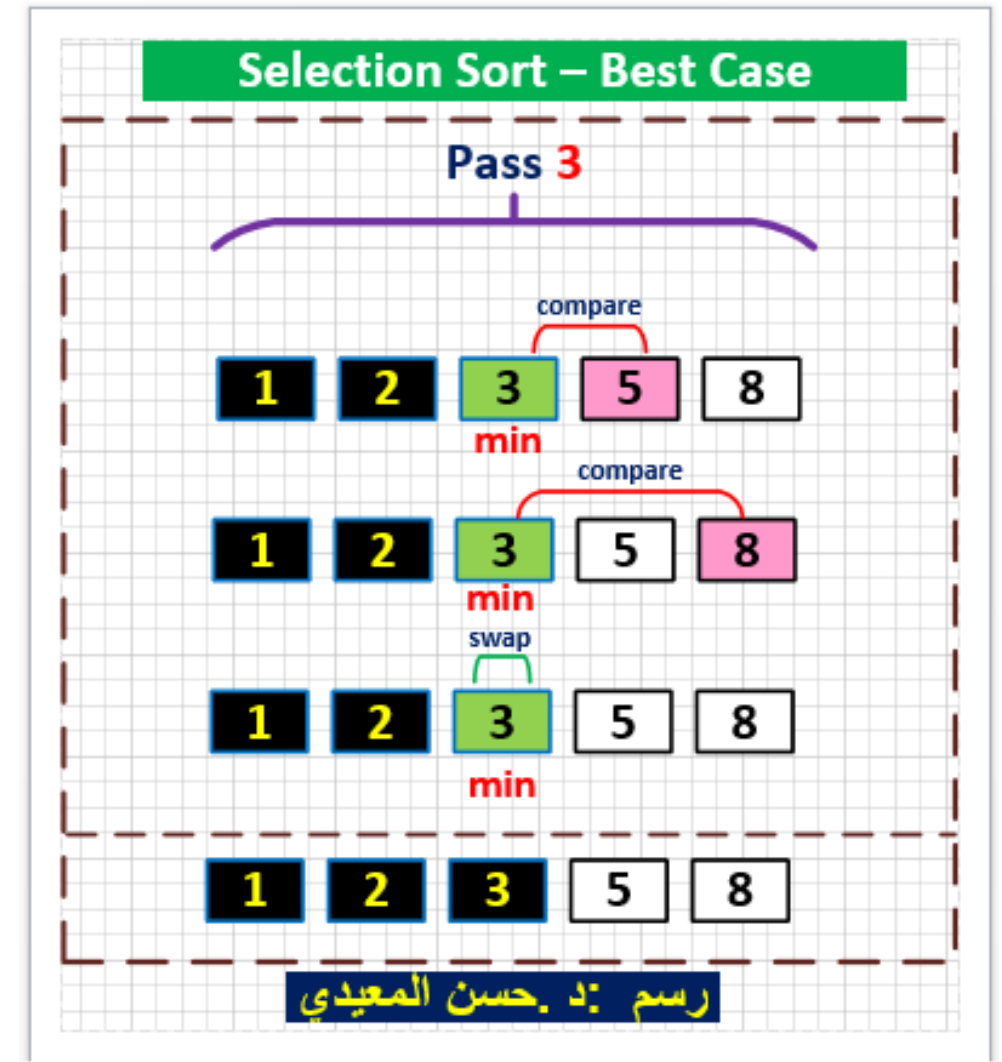


رسم: د. حسن المعيدي

Analysis of Selection-Sort Algorithm (Best-Case)

Compare

Copy



Analysis of Selection-Sort Algorithm (Best-Case)

Compare

Copy



Analysis of Selection-Sort Algorithm (Best-Case)

Pass No.	Comparison	Copy
1	4	1
2	3	1
3	2	1
4	1	1
Total		

Analysis of Selection-Sort Algorithm (Best-Case)

General Equation (if the input size is n)

No. Of Passes

$$n - 1$$

Time to Compare

$$T_1(n) = (n-1)*n/2$$

Time to Copy

$$T_2(n) = n - 1$$

Total Time

$$T(n) = n^2/2 + n/2 - 1$$

$$T(n) \in \Omega(n^2)$$

Summarization of Selection Sort Algorithm

Algorithm	Compare	Copy	Running Time Equation
Best Case			$T(n) =$
			$T(n) \in$
			No. of Passes =
Worst Case			$T(n) =$
			$T(n) \in$
			No. of Passes =

Example - Selection Sorting Algorithm

EXAMPLE

Assume you used the Selection-Sort algorithm to sort an array (A) with an input size is **50**. Answer the following questions.

1- In the **best case**, how many **compare** operations you did?

2- In the **best case**, how many **copy** operations you did?

3- In the **best case**, what is the total no. operations you did?

4- In the **best case**, what is the total no. of passes you did?

Example – Selection Sorting Algorithm

EXAMPLE

Assume you used the Selection-Sort algorithm to sort an array (A) with an input size is **50**. Answer the following questions.

5- In the **worst case**, how many **compare** operations you did?

6- In the **worst case**, how many **copy** operations you did?

7- In the **worst case**, what is the total no. operations you did?

8- In the **worst case**, what is the total no. of passes you did?

Analysis of Insertion Sort Algorithm (Worst-Case)

Analysis – Insertion-Sort Algorithm

Idea:

- Partition the array into two regions: sorted and unsorted.
- Take each item from unsorted region and insert it into its correct order in the sorted region and so on.

Insertion-Sort Algorithm

```
1 def insertionSort(nlist):
2     for index in range(1,len(nlist)):
3
4         currentvalue = nlist[index]
5         position = index
6
7         while position>0 and nlist[position-1]>currentvalue:
8             nlist[position]=nlist[position-1]
9             position = position-1
10
11         nlist[position]=currentvalue
12
13 nlist = [14,46,43,27,57,41,45,21,70]
14 insertionSort(nlist)
15 print(nlist)
```

Analysis of Insertion-Sort Algorithm (Worst-Case)

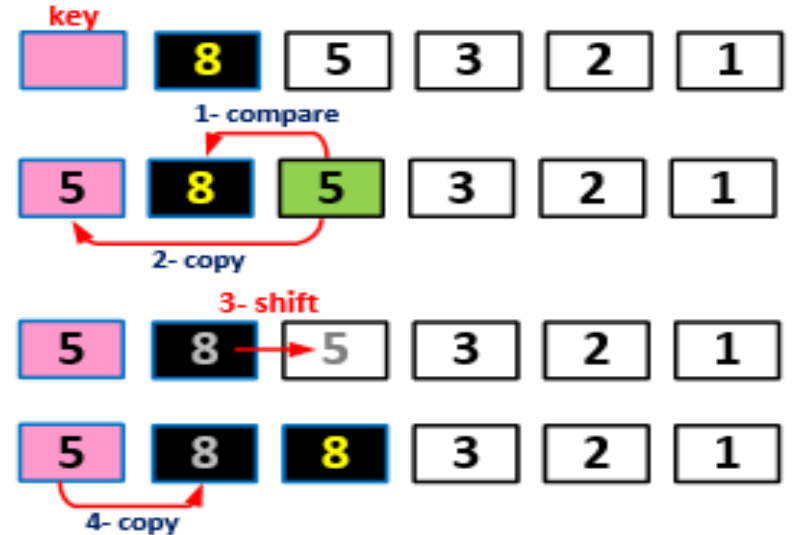
Compare

Shift

Copy

Insertion Sort – Worst Case

Pass 1



اعداد: د. حسن المعيدي

Analysis of Insertion-Sort Algorithm (Worst-Case)

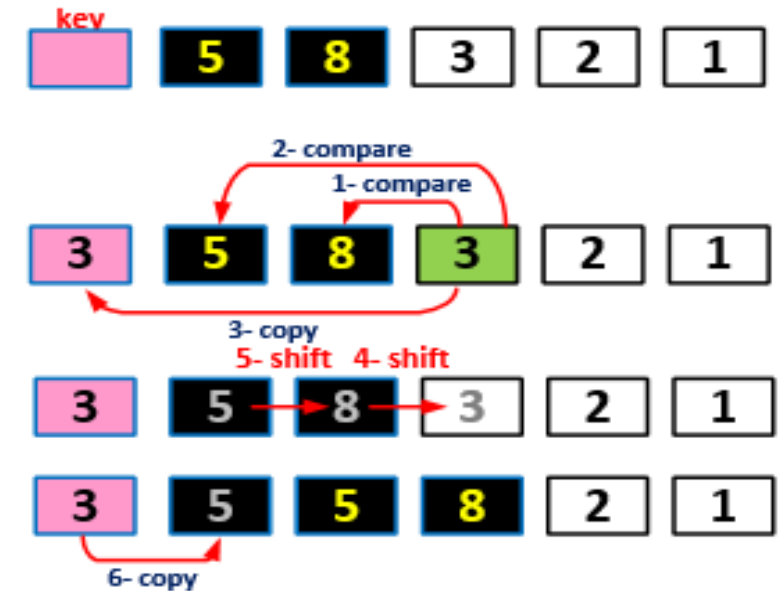
Compare

Shift

Copy

Insertion Sort – Worst Case

Pass 2



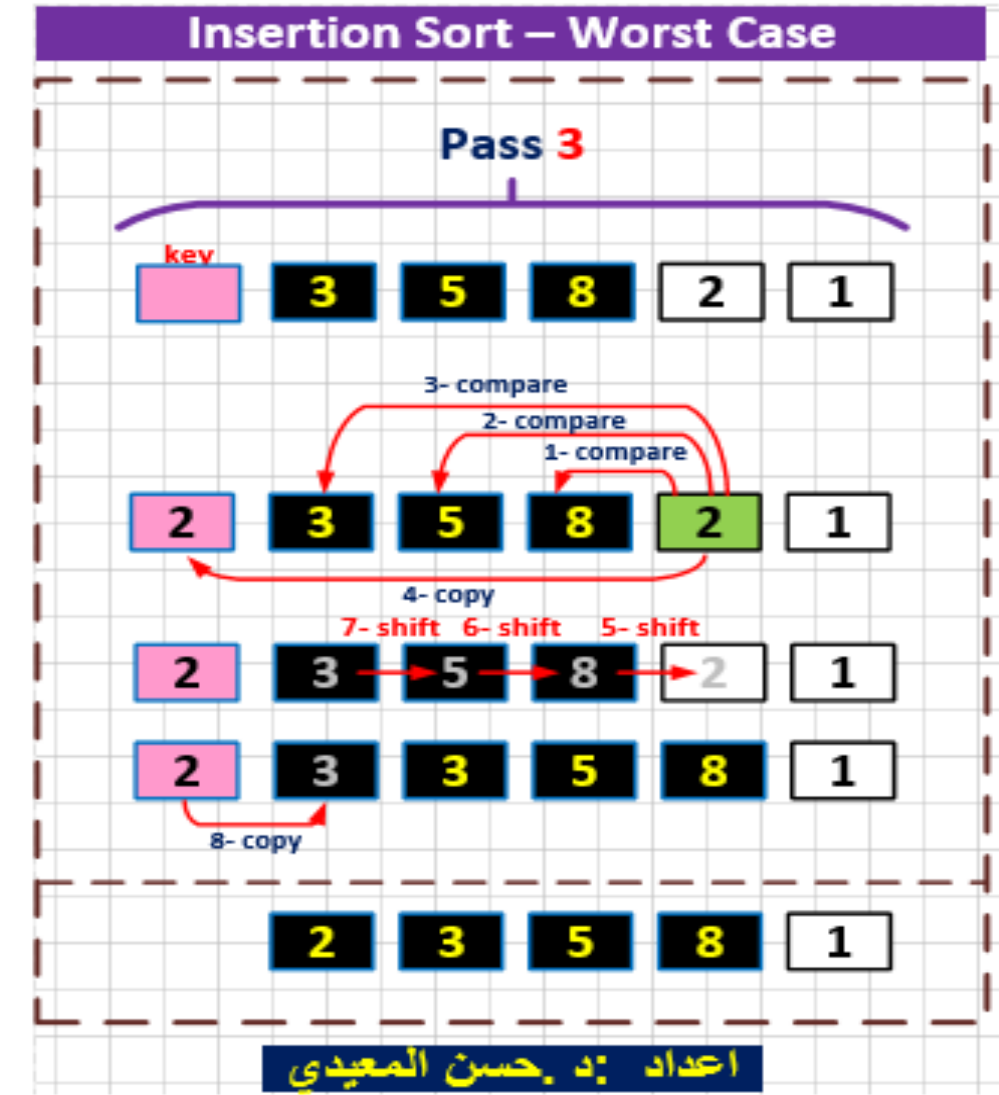
اعداد: د. حسن المعيدي

Analysis of Insertion-Sort Algorithm (Worst-Case)

Compare

Shift

Copy



Analysis of Insertion-Sort Algorithm (Worst-Case)

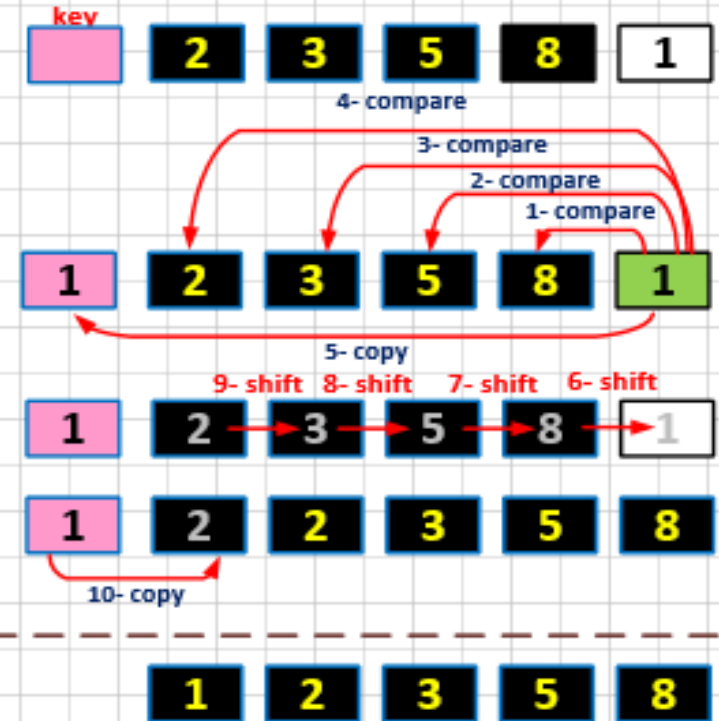
Compare

Shift

Copy

Insertion Sort – Worst Case

Pass 4



اعداد: د. حسن المعيدي

Analysis of Insertion-Sort Algorithm (Worst-Case)

Pass No.	Comparison	Shift	Copy
1	1	1	2
2	2	2	2
3	3	3	2
4	4	4	2
Total			

Analysis of Insertion-Sort Algorithm (Worst-Case)

General Equation (if the input size is n)

No. Of Passes

Time to Compare

Time to Shift

Time to Copy

Total Time

Analysis of Insertion Sort Algorithm (Best-Case)

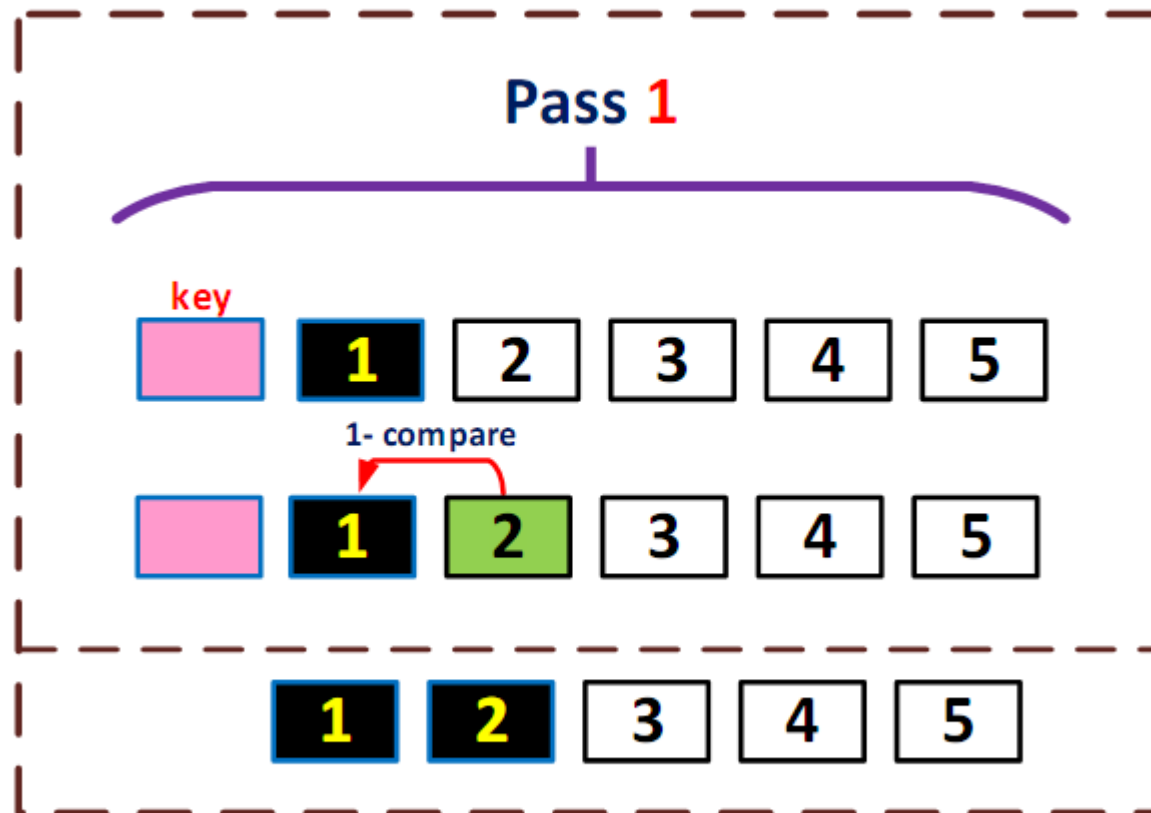
Analysis of Insertion-Sort Algorithm (Best-Case)

Compare

Shift

Copy

Insertion Sort – Best Case



اعداد : د. حسن المعيدي

Analysis of Insertion-Sort Algorithm (Best-Case)

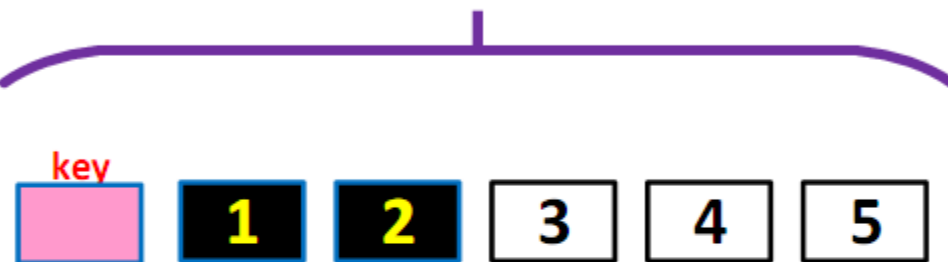
Compare

Shift

Copy

Insertion Sort – Best Case

Pass 2



اعداد : د. حسن المعيدي

Analysis of Insertion-Sort Algorithm (Best-Case)

Compare

Shift

Copy

Insertion Sort – Best Case

Pass 3



اعداد : د. حسن المعيدي

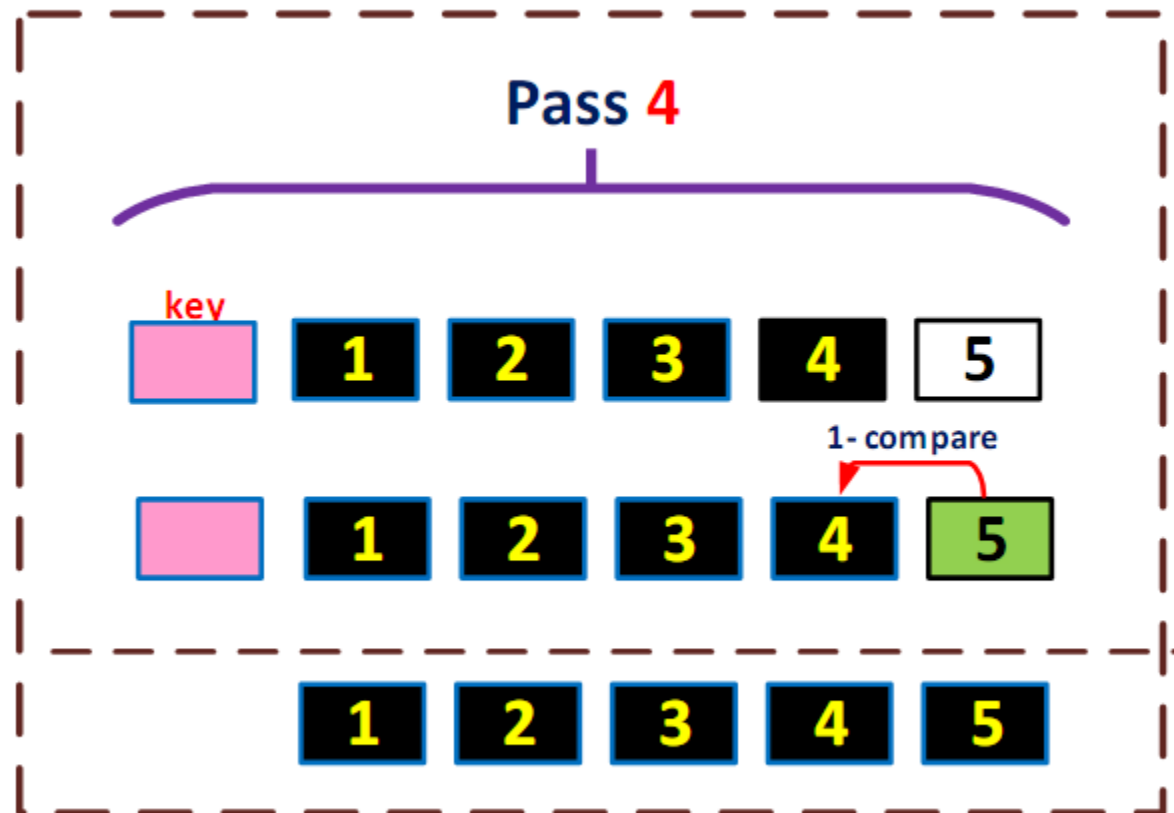
Analysis of Insertion-Sort Algorithm (Best-Case)

Compare

Shift

Copy

Insertion Sort – Best Case



اعداد : د. حسن المعيدي

Analysis of Insertion-Sort Algorithm (Best-Case)

Pass No.	Comparison	Shift	Copy
1	1	1	2
2	2	2	2
3	3	3	2
4	4	4	2
Total			

Analysis of Insertion-Sort Algorithm (Best-Case)

General Equation (if the input size is n)

No. Of Passes

Time to Compare

Time to Shift

Time to Copy

Total Time

Summarization of Insertion Sort Algorithm

Algorithm	Compare	Shift	Copy	Running Time Equation
Best Case				$T(n) =$
				$T(n) \in$
				No. of Passes =
Worst Case				$T(n) =$
				$T(n) \in$
				No. of Passes =

Example - Insertion Sorting Algorithm

EXAMPLE

Assume you used the Insertion-Sort algorithm to sort an array (A) with an input size is **50**. Answer the following questions.

1- In the **best case**, how many **compare** operations you did?

2- In the **best case**, how many **shift** operations you did?

3- In the **best case**, how many **copy** operations you did?

4- In the **best case**, what is the total no. operations you did?

Example - Insertion Sorting Algorithm

EXAMPLE

Assume you used the Insertion-Sort algorithm to sort an array (A) with an input size is **50**. Answer the following questions.

5- In the **worst case**, how many **compare** operations you did?

6- In the **worst case**, how many **shift** operations you did?

7- In the **worst case**, how many **copy** operations you did?

8- In the **worst case**, what is the total no. operations you did?

Summarization of Sorting Algorithms

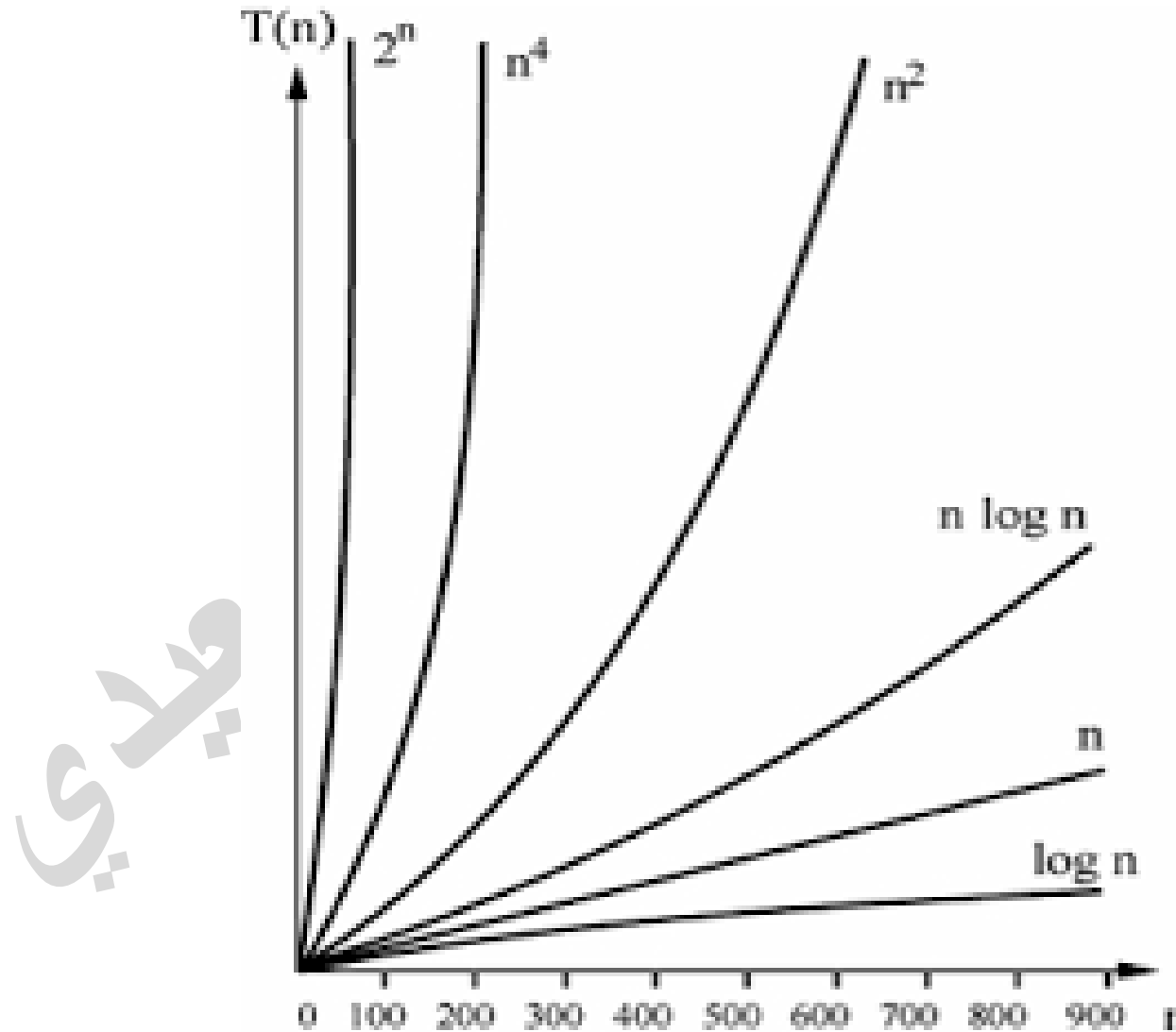
Summarization of Sorting Algorithms

		Algorithm	Best Case	Worst Case
Traditional		Selection	$\Omega(n^2)$	$O(n^2)$
		Insertion	$\Omega(n)$	$O(n^2)$
		Bubble	$\Omega(n^2)$	$O(n^2)$
D & C		Merge	$\Omega(n \log n)$	$O(n \log n)$
		Quick	$\Omega(n \log n)$	$O(n^2)$

N^2 Sorting Algorithm vs. an $N \log N$

N	N^2 Comparisons	N^2 Run Time	$N \cdot \log(N)$ Comparisons	$N \cdot \log(N)$ Run Time
10	100	0.1 sec	33	0.03 sec
20	400	0.4 sec	86	0.09 sec
50	2.5k	2.5 sec	282	0.28 sec
100	10k	10 sec	664	0.66 sec
200	40k	40 sec	1.5k	1.5 sec
500	250k	4 min	4.5k	4.5 sec
1,000	1M	16 min	10k	10 sec
10,000	100M	27 hours	133k	2.2 min
50,000	2.5G	29 days	780k	13 min
100,000	10G	116 days	1.66M	27 min
1,000,000	1T	31 years !	19.9M	5.5 hours

Growth of Functions



Example of : Growth of Functions

N	10	50	100	1000
$\lg(n)$	.0003 sec	.0006 sec	.0007 sec	.001 sec
n	.001 sec	.005 sec	.01 sec	0.1 sec
n^2	.01 sec	.25 sec	1 sec	1.67 min
2^n	.1024 sec	3570 years	$4 * 10^{16}$ centuries	$3 * 10^{287}$ centuries

Common Growth of Functions

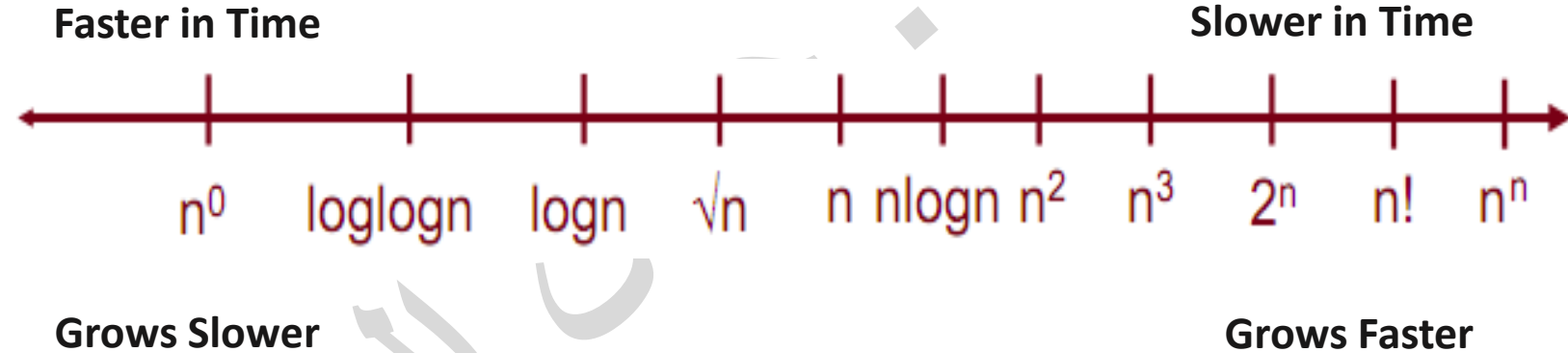
<i>Theta Form</i> [†]	<i>Name</i>
$\Theta(1)$	Constant
$\Theta(\lg \lg n)$	Log log
$\Theta(\lg n)$	Logarithmic
$\Theta(n)$	Linear
$\Theta(n \lg n)$	$n \log n$
$\Theta(n^2)$	Quadratic
$\Theta(n^3)$	Cubic
$\Theta(n^m)$	Polynomial
$\Theta(m^n), m \geq 2$	Exponential
$\Theta(n!)$	Factorial



[†] $\lg = \log$ to the base 2; m is a

fixed, nonnegative integer.

Order of Growth of Functions



n^2 Sorting Algorithm vs. an $n \log n$

Example

If the actual CPU time required to execute the Insertion-Sort algorithm for an array with an input size 100 is 10 second in the worst-case, what is the predicted CPU time in seconds for execution the same array if you use the Merge-Sort algorithm?

Algorithm	Complexity	n	CPU Time
-----------	------------	---	----------

n^2 Sorting Algorithm vs. an $n \log n$

Example

If the actual CPU time required to execute the algorithm (A) for an array with an input size 100 is 30 second in the worst-case, what is the predicted CPU time in seconds for execution the same array if you use another algorithm (B), if you know that the complexity of (A) is n^3 and the complexity of (B) is n^2 ?

Algorithm	Complexity	n	CPU Time
-----------	------------	---	----------

Asymptotic Notations

Asymptotic Notations

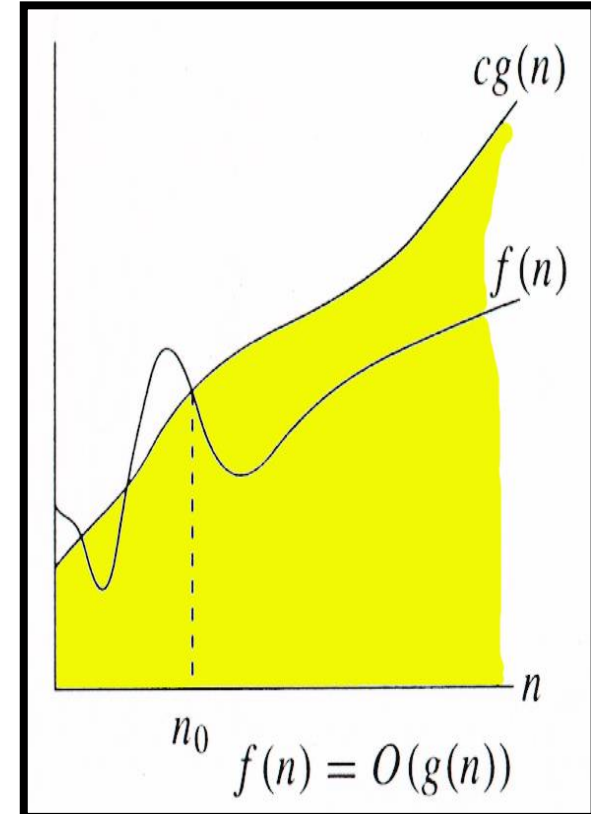
These notations are used to express the complexity of a given algorithm or used to compare two algorithms or more that are designed to solve the same problem.

Big O	Worst Case	$T(n) \leq c g(n)$
Big Ω	Best Case	$T(n) \geq c g(n)$
Big θ	Worst = Best	$c_1 g(n) \leq T(n) \leq c_2 g(n)$
Little o	In Math	$T(n) < c g(n)$
Little ω	In Math	$T(n) > c g(n)$

Big - Oh

Formal Definition of Big Oh

- $T(n) \in O(g(n))$ means there are positive constants c and k , such that $T(n) \leq cg(n); \forall n \geq k$.
- Also known as O , asymptotic upper bound.
- Express the worst case.



Big - Oh

EXAMPLE

If $T(n) = 5n + 4$; prove that $T(n) \in O(n)$ using the definition of big-Oh.

د. حسن المعيني

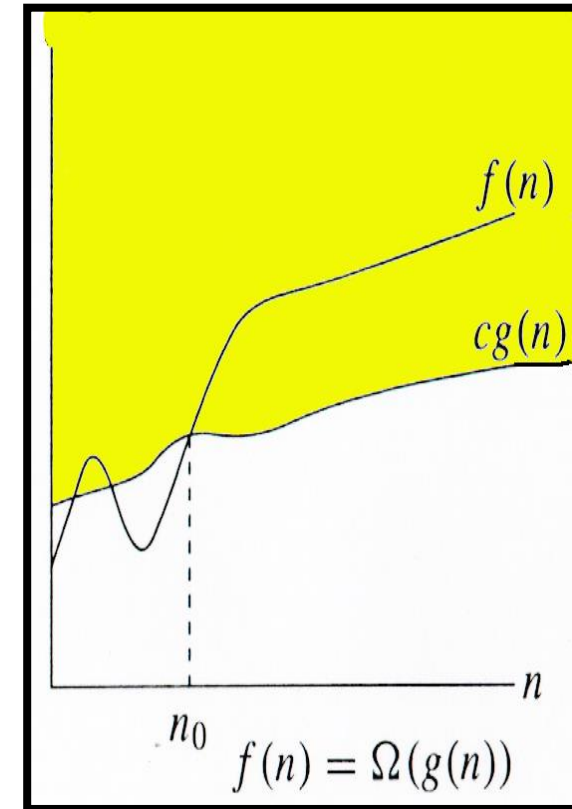
Big - Omega

EXAMPLE

If $T(n) = 7n^3 + 4n + 2$; prove that $T(n) \in \Omega(n^3)$ using the definition of big-Oh.

Formal Definition of Big Omega

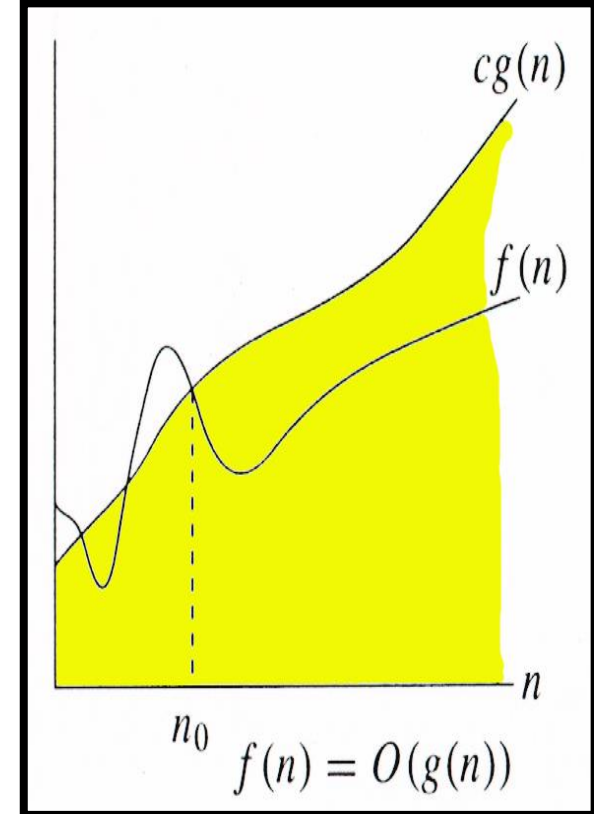
- $T(n) \in \Omega(g(n))$ means there are positive constants c and k , such that $T(n) \geq cg(n)$; $\forall n \geq k$.
- Also known as Ω , asymptotic lower bound.
- Express the best case.



Big - Oh

Formal Definition of Big Oh

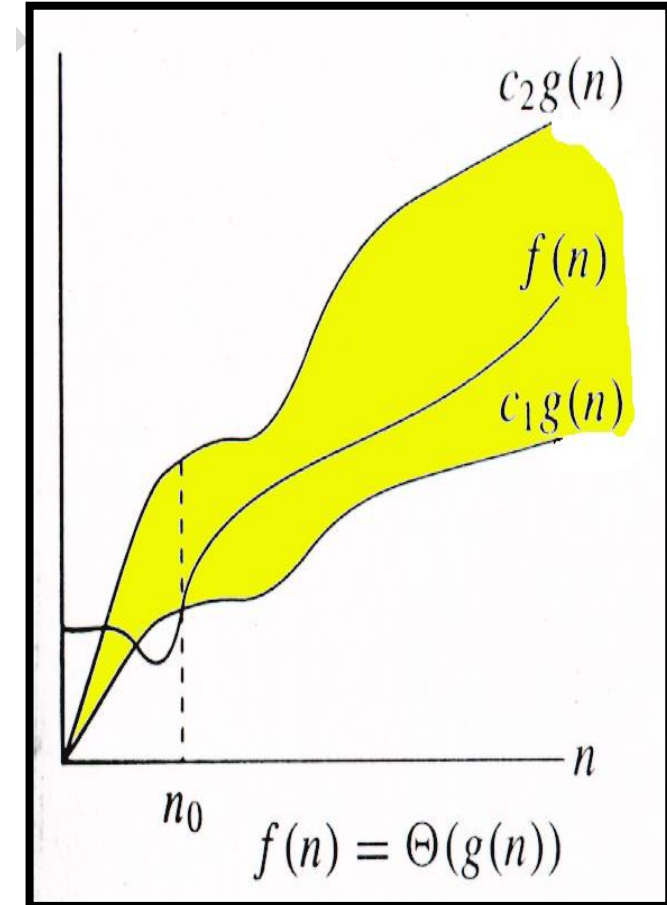
- $T(n) \in O(g(n))$ means there are positive constants c and k , such that $T(n) \leq cg(n); \forall n \geq k$.
- Also known as O , asymptotic upper bound.
- Express the worst case.



Big - Theta

Formal Definition of Big Theta

- $T(n) \in \Theta(g(n))$ means there are positive constants c_1 , c_2 and k , such that $c_1g(n) \leq T(n) \leq c_2g(n)$; $\forall n \geq k$.
- Also known as Θ , asymptotic tight bound.
- Express the best, worst, and avg. cases.
- Also used when best case = worst case.



Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$1- T(n) \in O(n^5)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$2- T(n) \in O(n \log n)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$3- T(n) \in O(n^9)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$4- T(n) \in \Omega(n^5)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$5- T(n) \in \Omega(n^3)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$6- T(n) \in \Omega(n \log n)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$7- T(n) \in \theta(n^5)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$8- T(n) \in \theta(n \log n)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$9 - T(n) \in \theta(n^4)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$10- T(n) \in \theta(2^n)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$11- T(n) \in o(n^9)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$12- T(n) \in o(n^4)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$13- T(n) \in o(n^3)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$14- T(n) \in \omega(n^3)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$15- T(n) \in \omega(n^4)$$



X

Exercise

$$T(n) = 9n^4 + 2n^2 + 6 \log n$$

$$16 - T(n) \in \omega(n^{20})$$



X

Check - Chapter Learning Outcomes

CLO

2

- Distinguish between design and analysis phases.
- Understand the meaning of order of magnitude.
- Analyze the complexity of Linear search algorithm.
- Analyze the complexity of Insertion-sort algorithm.
- Analyze the complexity of Selection-sort algorithm.
- Characterize algorithms with respect to asymptotic growth.
- Apply asymptotic notation to describe the behavior of algorithms in best, average and worst cases.

End of Chapter

2



3

Recursive Algorithms

Chapter Learning Outcomes

CLO

3

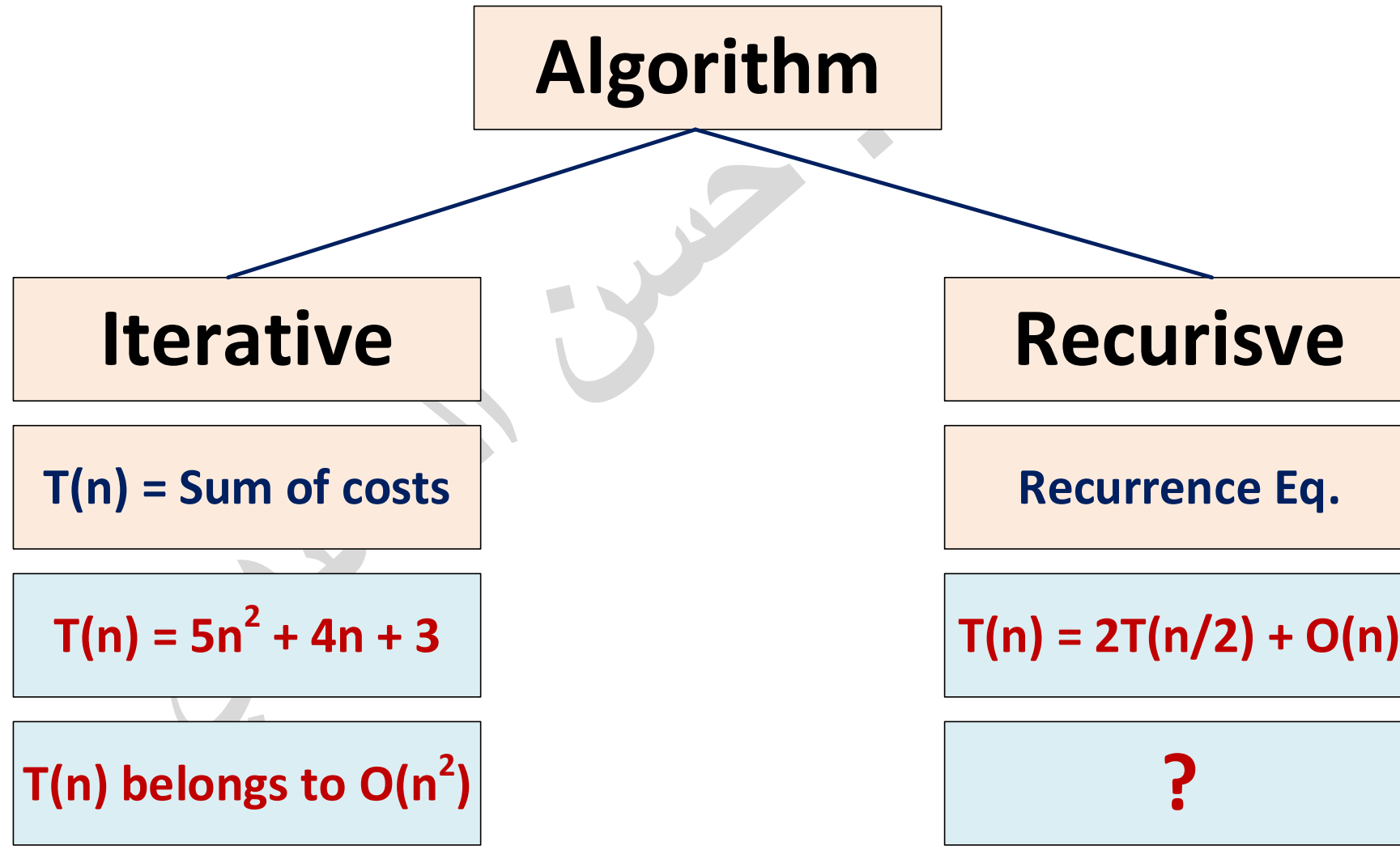
- Understand the meaning of recursive functions.
- Find the recurrence equation for a given recursive function.
- Know some methods to solve the recurrence equations.
- Know and understand the different cases of the master method.
- Solve some recurrence equations using the master method.

Recursive Function

A **recursive function** is a **function** that calls itself during its execution. This enables the **function** to repeat itself several times until it reaches the base step.

```
function Calculate(){  
    ...  
    Calculate();  
    ...  
}
```

Recursive Function



Recursive Function

EXAMPLE

Analyze the following recursive function. (Find the recurrence equation $T(n)$).

```
1. function func1( $A, n$ )  
   /*  $A$  = array of  $n$  integers */  
   1. if  $n \leq 20$  then return ( $A[1]$ );  
   2.  $x \leftarrow \text{func1}(A, \lfloor 3n/4 \rfloor)$ ;  
   3. for  $i \leftarrow \lfloor n/2 \rfloor$  to  $\lfloor n/2 \rfloor + 8$  do  
   4.    $x \leftarrow x + A[i]$ ;  
   5. endfor  
   6. return ( $x$ );
```

Recursive Function

EXAMPLE

Analyze the following recursive function. (Find the recurrence equation $T(n)$).

```

2. function func2( $A, n$ )
   /*  $A$  = array of  $n$  integers */
   1. if  $n \leq 12$  then return ( $A[1]$ );
   2. for  $i \leftarrow 1$  to  $\lfloor n/2 \rfloor$  do
   3.   for  $j \leftarrow 1$  to  $\lfloor n/3 \rfloor$  do
   4.     for  $k \leftarrow 1$  to  $n$  do
   5.        $A[k] \leftarrow A[k] - A[j] + A[i]$ ;
   6.     endfor
   7.   endfor
   8. endfor
   9.  $y \leftarrow \text{func2}(A, n - 5)$ ;
10. return ( $y$ );

```

Recursive Function

EXAMPLE

Analyze the following recursive function. (Find the recurrence equation $T(n)$).

```
3. function func3( $A, n$ )  
   /*  $A$  = array of  $n$  integers */  
   1. if  $n \leq 20$  then return ( $A[n]$ );  
   2.  $x \leftarrow 0$ ;  
   3. for  $i \leftarrow 1$  to 4 do  
   4.   for  $j \leftarrow 1$  to  $n - i$  do  
   5.      $A[j] \leftarrow A[j] - A[n - j]$ ;  
   6.   endfor  
   7.    $x \leftarrow x + \text{func3}(A, \lfloor n/4 \rfloor)$ ;  
   8. endfor  
   9. return ( $x$ );
```

Recursive Function

EXAMPLE

Analyze the following recursive function. (Find the recurrence equation $T(n)$).

```
4. function func4(A, n)
   /* A = array of n integers */
   1. if  $n \leq 3$  then return (A[1]);
   2. for  $i \leftarrow 1$  to  $n$  do
   3.   for  $j \leftarrow \lfloor n/3 \rfloor$  to  $\lfloor 2n/3 \rfloor$  do
   4.      $A[i] \leftarrow A[i] - A[j]$ ;
   5.   endfor
   6. endfor
   7.  $x \leftarrow \text{func4}(A, \lfloor 2n/3 \rfloor)$ ;
   8. return (x);
```

Recursive Function

EXAMPLE

Analyze the following recursive function. (Find the recurrence equation $T(n)$).

```
5. function func5( $A, n$ )  
   /*  $A$  = array of  $n$  integers */  
   1. if  $n \leq 20$  then return ( $A[n]$ );  
   2.  $x \leftarrow \text{func5}(A, n - 5)$ ;  
   3. for  $i \leftarrow 1$  to  $\lfloor n/2 \rfloor$  do  
   4.   for  $j \leftarrow \lfloor n/2 \rfloor$  to  $n$  do  
   5.      $A[i] \leftarrow A[i] - A[j]$ ;  
   6.   endfor  
   7. endfor  
   8.  $x \leftarrow x + \text{func5}(A, n - 8)$ ;  
   9. return ( $x$ );
```

Solving Recurrence Equations

Solving Recurrence Equations

To solve a recurrence relation $T(n)$ we need to derive a form of $T(n)$ that is not a recurrence relation. Such a form is called a closed form of the recurrence relation.

There are five methods to solve recurrence relations that represent the running time of recursive methods:

Solving Recurrence Equations

Solving Recurrence Equations

- 1- Master Method
- 2- Iteration Method
- 3- Recursion Tree Method
- 4- Substitution Method
- 5- Changing Variable Method

د. حسن المعيدي

Master Method

The master method provides bounds for recurrences of the form:

$$T(n) = aT(n/b) + f(n)$$

where :

$a \geq 1$, $b > 1$, $f(n)$ is a given function;

It requires memorization of three cases, but once you do that, determining asymptotic bounds for many simple recurrences is easy.

Master Method

EXAMPLE

Which of the following recurrence equation can be solved using the master method?

حسن المعيني

Master Method

Let $a \geq 1$ and $b > 1$ are two constants, let $f(n)$ be a function, and Let $T(n)$ be defined on nonnegative integers by the recurrence:

$$T(n) = aT(n/b) + f(n).$$

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } f(n) < n^d \\ \Theta(n^d \log n) & \text{if } f(n) = n^d \\ \Theta(f(n)) & \text{if } f(n) > n^d \end{cases}$$

$\text{and } af(n/b) \leq cf(n)$
where $c < 1$

→ **Regularity
Condition**

where $d = \log_b^a$

Master Method

EXAMPLE

Solve the following recurrence equation using the master method?

$$T(n) = 2T(n/2) + n$$

Master Method

EXAMPLE

Solve the following recurrence equation using the master method?

$$T(n) = 9T(n/3) + n$$

حسن المعيني

Master Method

EXAMPLE

Solve the following recurrence equation using the master method?

$$T(n) = 7T(n/2) + n^2$$

Master Method

EXAMPLE

Solve the following recurrence equation using the master method?

$$T(n) = T(n/2) + 1$$

Master Method

EXAMPLE

Solve the following recurrence equation using the master method?

$$T(n) = 3T(n/4) + n \log n$$

حسن المعيني

Master Method

EXAMPLE

Solve the following recurrence equation using the master method?

$$T(n) = T(2n/3) + n$$

Master Method

EXAMPLE

Solve the following recurrence equation using the master method?

$$T(n) = 2T(n/4) + n^{0.51}$$

Check - Chapter Learning Outcomes

CLO

3

- Understand the meaning of recursive functions.
- Find the recurrence equation for a given recursive function.
- Know some methods to solve the recurrence equations.
- Know and understand the different cases of the master method.
- Solve some recurrence equations using the master method.

End of Chapter

3



4

Divide and Conquer

Chapter Learning Outcomes

CLO

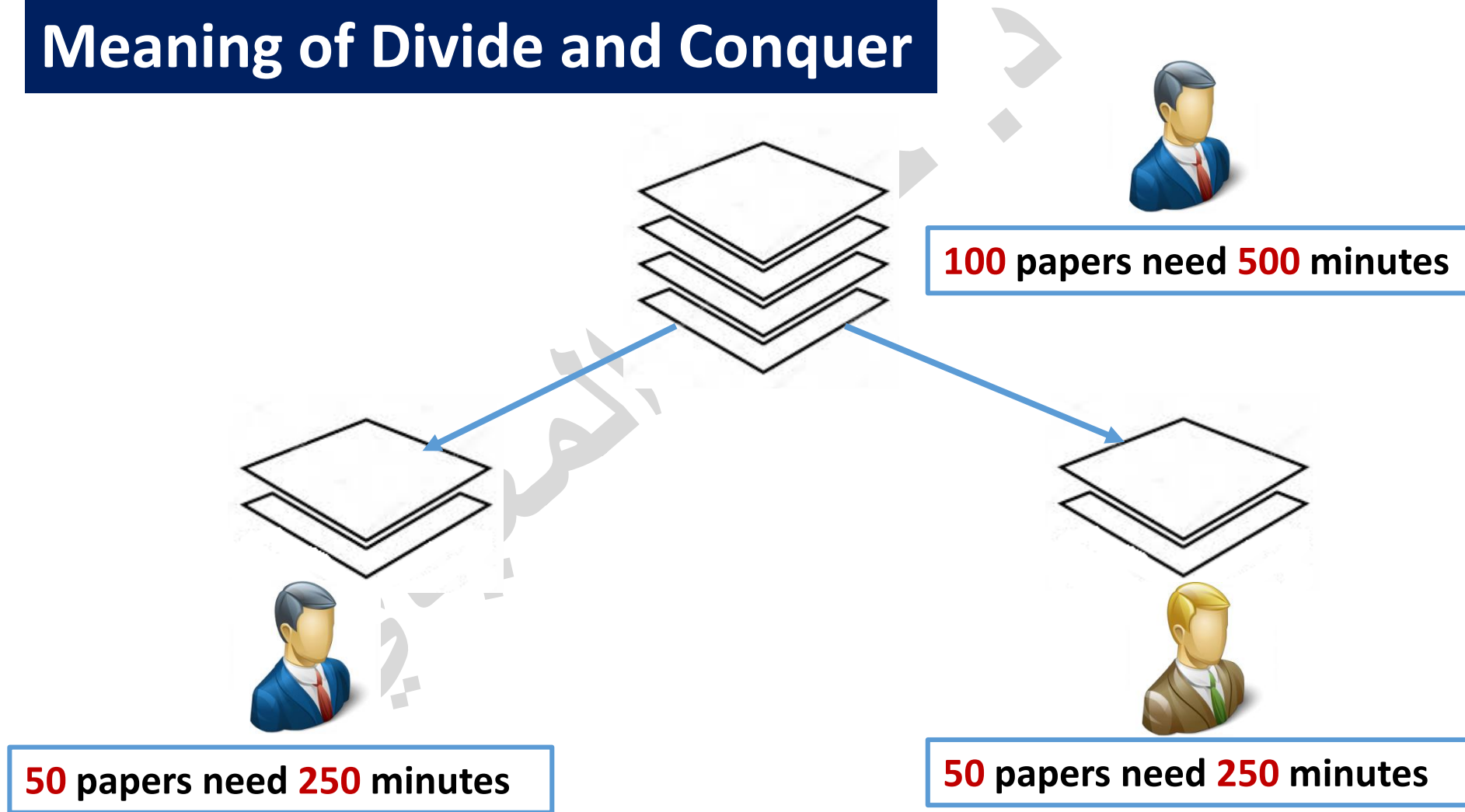
4

- Understand the meaning of Divide and Conquer.
- Analyze the complexity of Merge-sort algorithm.
- Analyze the complexity of Quick-sort algorithm.
- Analyze the complexity of Binary-Search algorithm.

Introduction

Divide and Conquer – An Introduction

Meaning of Divide and Conquer



Divide and Conquer – An Introduction

Meaning of Divide and Conquer

- In military, “**divide and conquer**” is a kind of strategy.
- In computer science, “**divide and conquer**” is a programming technique by breaking down a problem into one or more sub problems.
- “**Divide and conquer**” is usually implemented by recursion.

Divide and Conquer – An Introduction

Recursive Function

A **recursive function** is a **function** that calls itself during its execution. This enables the **function** to repeat itself several times until it reaches the base step.

```
function Calculate(){  
    ...  
    Calculate();  
    ...  
}
```

Divide and Conquer – An Introduction

Divide and Conquer Approach

Many useful algorithms are recursive in structure: to solve a given problem, they call themselves recursively one or more times to deal with closely related subproblems.

These algorithms typically follow a ***divide-and-conquer*** approach: they break the problem into several subproblems that are similar to the original problem but smaller in size, solve the subproblems recursively, and then combine these solutions to create a solution to the original problem.

Divide and Conquer – An Introduction

The General Method

Divide

Divide the problem into a number of subproblems that are themselves smaller instances of the same type of problem.

Conquer

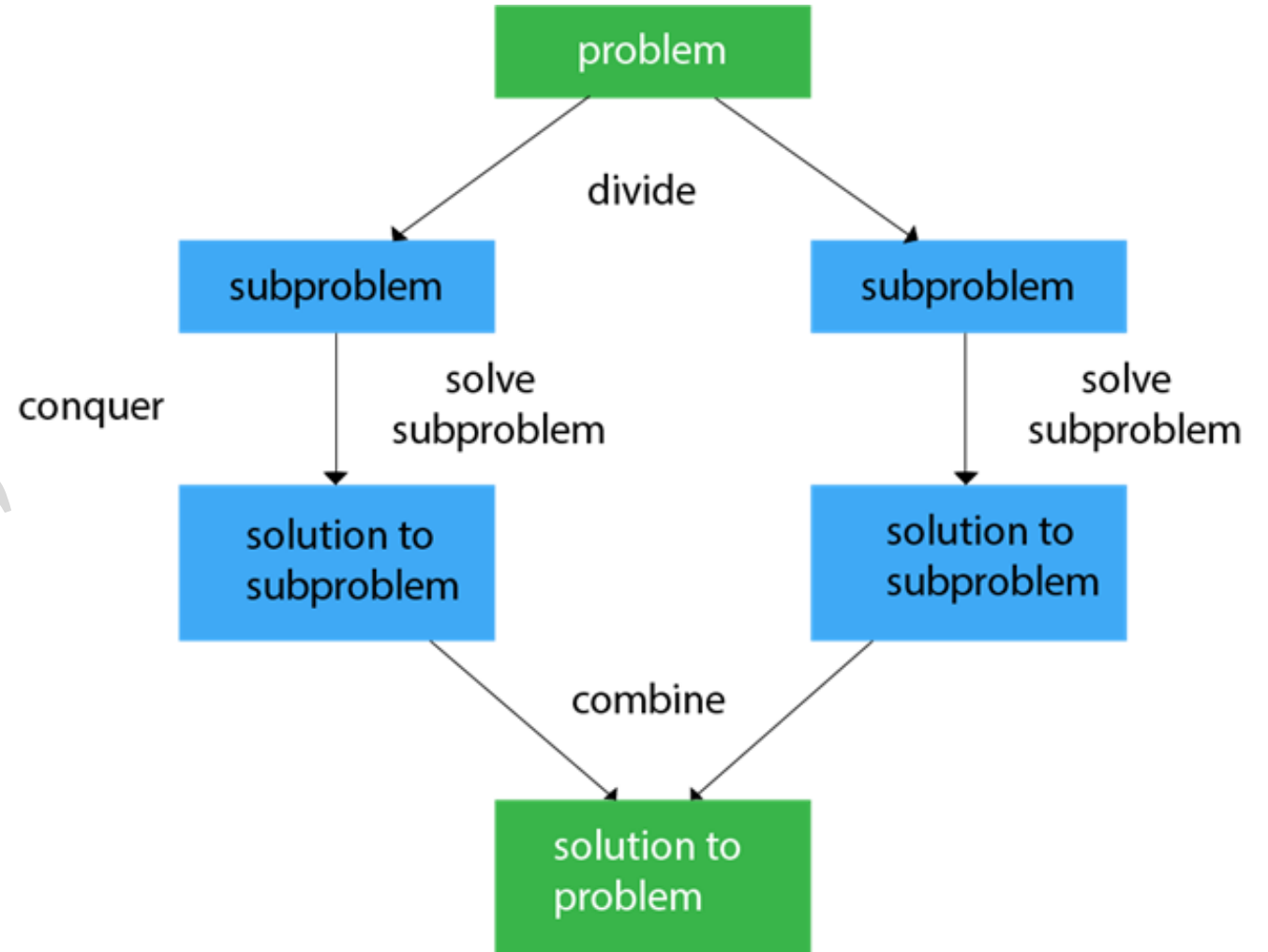
Recursively solving these subproblems. If the subproblems are small enough, solve them straightforward.

Combine

Combine the solutions to the subproblems into the solution of original problem.

Divide and Conquer – An Introduction

The General Method



Divide and Conquer – An Introduction

The Cost Time for Divide and Conquer Approach

$$T(n) = \text{Time to Divide} + \text{Time to Conquer} + \text{Time to Combine}$$



Divide and Conquer – An Introduction

Examples of Problems Solved Using Divide and Conquer Approach

1- Merge Sort

2- Quick Sort

3- Binary Search

4- Strassen's Matrix Multiplications



Divide and Conquer – An Introduction

Methodology

For each problem, we will study :

The Idea

Solve Example

Write the recursive Algorithm

Analyze the recursive Algorithm

Find the Complexity

Merge-Sort Algorithm

Merge-Sort Algorithm



John von Neumann

(1903-1957)

- Developed merge sort for EDVAC in 1945.

Divide and Conquer – Merge-Sort Algorithm

Idea

Merge-sort on an input sequence (A) with (n) elements consists of three steps:

1. **Divide**: partition (A) into two sequences A1 and A2 of about $n/2$ elements each.
2. **Conquer** : recursively sort A1 and A2.
3. **Combine** : merge A1 and A2 into a unique sorted sequence.

Divide and Conquer – Merge-Sort Algorithm

Example

Sort the following array (A) using Merge-Sort algorithm.



Divide and Conquer – Merge-Sort Algorithm

Example

Step 1 : Partition (A) into two sub-arrays of about $n/2$ elements each.

p=0	1	2	q=3	4	5	6	r=7
6	3	5	1	8	2	4	7

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{0 + 7}{2} \right\rfloor$$

$$q = \lfloor 3.5 \rfloor = 3$$

Divide and Conquer – Merge-Sort Algorithm

6	3	5	1	8	2	4	7
---	---	---	---	---	---	---	---

p=0	q=1	2	r=3
6	3	5	1

p=4	q=5	6	r=7
8	2	4	7

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{0 + 3}{2} \right\rfloor$$

$$q = \lfloor 1.5 \rfloor = 1$$

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{4 + 7}{2} \right\rfloor$$

$$q = \lfloor 5.5 \rfloor = 5$$

Divide and Conquer – Merge-Sort Algorithm

6 3 5 1 8 2 4 7

6 3 5 1 8 2 4 7

p=0 r=1

6 3

p=2 r=3

5 1

p=4 r=5

8 2

p=6 r=7

4 7

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{0 + 1}{2} \right\rfloor$$

$$q = \lfloor 0.5 \rfloor = 0$$

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{2 + 3}{2} \right\rfloor$$

$$q = \lfloor 2.5 \rfloor = 2$$

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{4 + 5}{2} \right\rfloor$$

$$q = \lfloor 4.5 \rfloor = 4$$

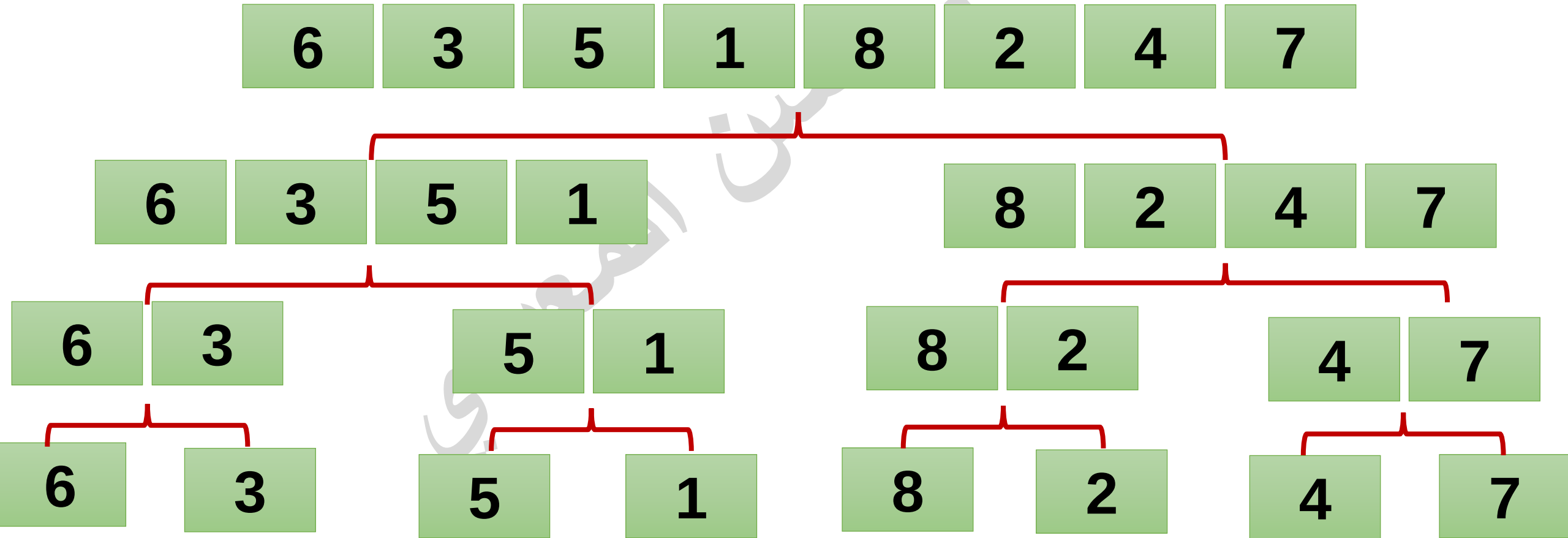
$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{6 + 7}{2} \right\rfloor$$

$$q = \lfloor 6.5 \rfloor = 6$$

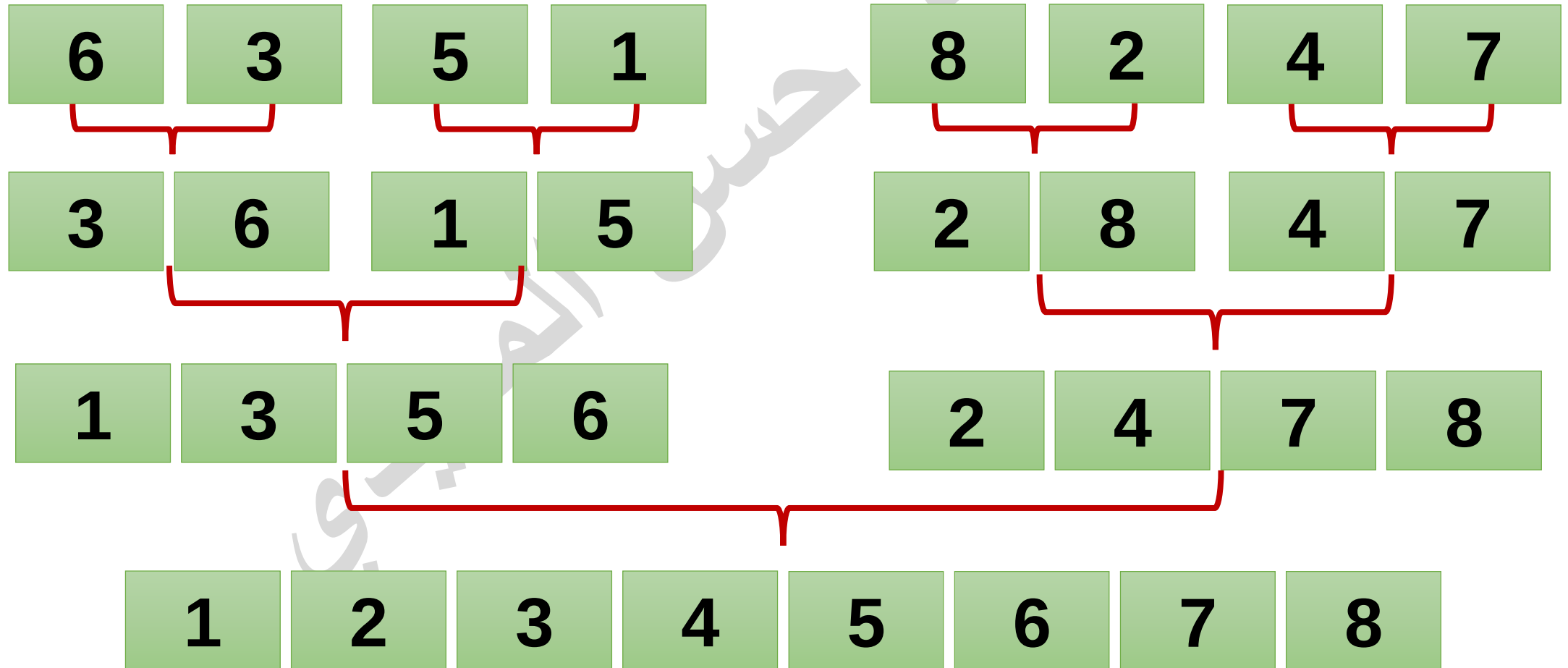
Divide and Conquer – Merge-Sort Algorithm

Divide - Section



Divide and Conquer – Merge-Sort Algorithm

Combine - Section



Divide and Conquer – Merge-Sort Algorithm

How to Combine ?

Here are two lists to be merged:

First: (12, 16, 17, 20, 21, 27)

Second: (9, 10, 11, 12, 19)

Compare 12 and 9

First: (12, 16, 17, 20, 21, 27)

Second: (10, 11, 12, 19)

New: (9)

Compare 12 and 10

First: (12, 16, 17, 20, 21, 27)

Second: (11, 12, 19)

New: (9, 10)

Compare 12 and 11

Divide and Conquer – Merge-Sort Algorithm

How to Combine ?

First: (12, 16, 17, 20, 21, 27)

Second: (12, 19)

New: (9, 10, 11)

Compare 12 and 12

First: (16, 17, 20, 21, 27)

Second: (12, 19)

New: (9, 10, 11, 12)

Compare 16 and 12

Divide and Conquer – Merge-Sort Algorithm

How to Combine ?

First: (16, 17, 20, 21, 27)

Second: (19)

New: (9, 10, 11, 12, 12)

Compare 16 and 19

First: (17, 20, 21, 27)

Second: (19)

New: (9, 10, 11, 12, 12, 16)

Compare 17 and 19

Divide and Conquer – Merge-Sort Algorithm

How to Combine ?

First: (20, 21, 27)

Second: (19)

New: (9, 10, 11, 12, 12, 16, 17)

Compare 20 and 19

First: (20, 21, 27)

Second: ()

New: (9, 10, 11, 12, 12, 16, 17, 19)

Checkout 20 and empty list

First: ()

Second: ()

New: (9, 10, 11, 12, 12, 16, 17, 19, 20, 21, 27)

Divide and Conquer – Merge-Sort Algorithm

Algorithm Merge-Sort(A, p, r)

1. IF ($p < r$) THEN
 - (a) Assign $q = \lfloor p+r \rfloor / 2$
 - (b) Call Merge-Sort(A, p, q)
 - (c) Call Merge-Sort($A, q+1, r$)
 - (d) Call Combine(A, p, q, r)
2. End If

Divide and Conquer – Merge-Sort Algorithm

Algorithm Merge-Sort(A, p, r)

1. IF ($p < r$) THEN

(a) Assign $q = \lfloor p+r \rfloor / 2$

(b) Call Merge-Sort(A, p, q)

(c) Call Merge-Sort($A, q+1, r$)

(d) Call Combine(A, p, q, r)

2. End If

Divide and Conquer – Merge-Sort Algorithm

$$T(n) = \text{Time to Divide} + \text{Time to Conquer} + \text{Time to Combine}$$



Divide and Conquer – Merge-Sort Algorithm

Solve the recurrence equation using the Master Method

د. حسن المعيني



Divide and Conquer – Merge-Sort Algorithm

Summary

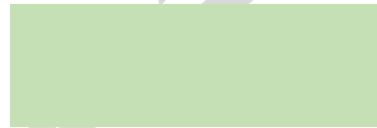
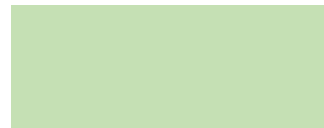
Divide

Conquer

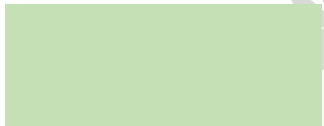
Combine

Recurrence Equation

Best Case



Worst Case



Quick-Sort Algorithm

Quick-Sort Algorithm



The algorithm was developed by a British computer scientist **Tony Hoare** in 1959.

Divide and Conquer – Quick-Sort Algorithm

Idea

Quick-sort is **a randomized sorting** algorithm based on the Divide-and-Conquer paradigm :

1. Divide: Partition the array into two subarrays around **a pivot p** such that:

elements in lower subarray $\leq p \leq$ elements in upper subarray.

1. Conquer: Recursively sort the two subarrays.

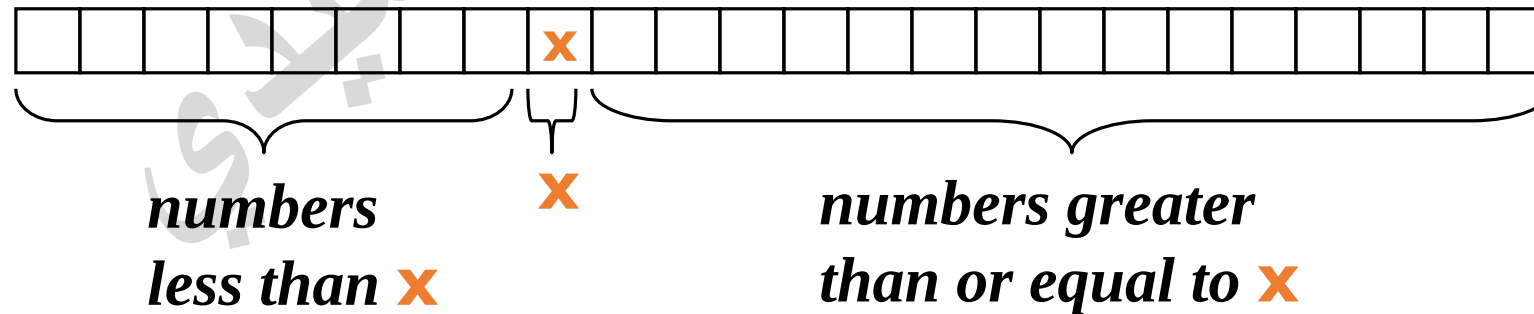
2. Combine: Trivial.

Divide and Conquer – Quick-Sort Algorithm

Idea

A key step in the Quicksort algorithm is **partitioning** the array.

- We choose some (any) number **x** in the array to use as **a pivot**
- We partition the array into three parts:



Quick-Sort Algorithm

A	6	10	5	8	13	3	2	11
---	---	----	---	---	----	---	---	----

How to Partition the array A ?

First Scenario, create a new array (B), and partition (A) around pivot, put the result in (B). Repeat the steps until (B) is sorted.

B								
---	--	--	--	--	--	--	--	--

Pivot
6

B	5	3	2	6	10	8	13	11
---	---	---	---	---	----	---	----	----

Disadvantage ? Extra places, Space Complexity $O(n)$

Quick-Sort Algorithm

A	6	10	5	8	13	3	2	11
---	---	----	---	---	----	---	---	----

How to Partition the array A ?

Second Scenario: In-Place Partition.

We need to sort this array in the most efficient manner without using extra place (in-place sorting).

Quick-Sort Algorithm (In-Place Partition)

A

6	10	5	8	13	3	2	11
---	----	---	---	----	---	---	----

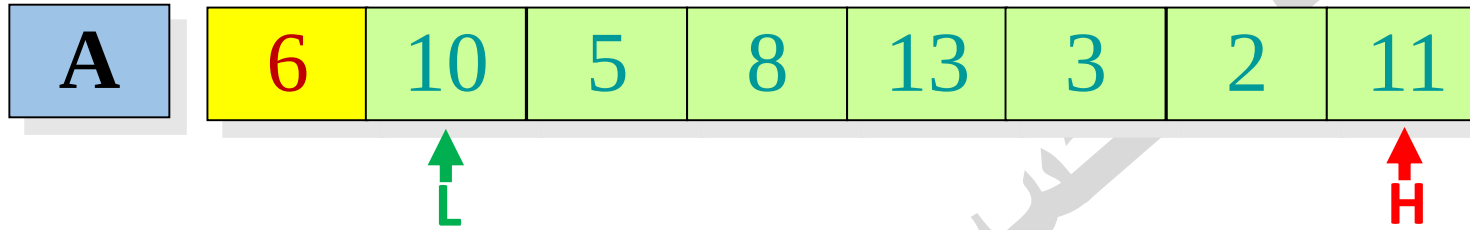
Pivot
6

1- Make any element as pivot: For convenience of code and for the purpose of this course, we will use the first element as pivot.

2- Partition the array on the basis of pivot: Call for partitioning which rearranges the array in such a way that pivot (6) comes to its actual position (of the sorted array). And to the left of the pivot, the array has all the elements less than it, and to the right greater than it.

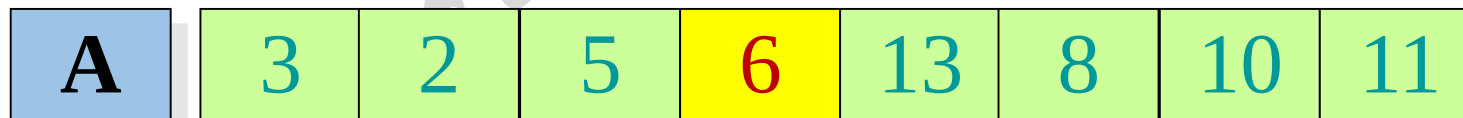
Quick-Sort Algorithm (In-Place Partition)

At the **Beginning** of 1st Partitioning



- If $low < pivot$, then move low to the right.
- If $high > pivot$, then move high to the left.
- Swap if needed.
- Swap (pivot, $A[high]$).

At the **End** of 1ST Partitioning



Quick-Sort Algorithm (In-Place Partition)

At the **Beginning** of 2nd Partitioning

A	3	2	5	6	13	8	10	11
----------	---	---	---	---	----	---	----	----

At the **End** of 2nd Partitioning

A	2	3	5	6	13	8	10	11
----------	---	---	---	---	----	---	----	----

Quick-Sort Algorithm (In-Place Partition)

At the **Beginning** of 3rd Partitioning

A	2	3	5	6	13	8	10	11
----------	---	---	---	---	----	---	----	----

At the **End** of 3rd Partitioning

A	2	3	5	6	11	8	10	13
----------	---	---	---	---	----	---	----	----

Quick-Sort Algorithm (In-Place Partition)

At the **Beginning** of 4th Partitioning

A	2	3	5	6	11	8	10	13
----------	---	---	---	---	----	---	----	----

At the **End** of 4th Partitioning

A	2	3	5	6	10	8	11	13
----------	---	---	---	---	----	---	----	----



Quick-Sort Algorithm (In-Place Partition)

At the **Beginning** of 5th Partitioning

A	2	3	5	6	10	8	11	13
----------	---	---	---	---	----	---	----	----

At the **End** of 5th Partitioning

A	2	3	5	6	8	10	11	13
----------	---	---	---	---	---	----	----	----



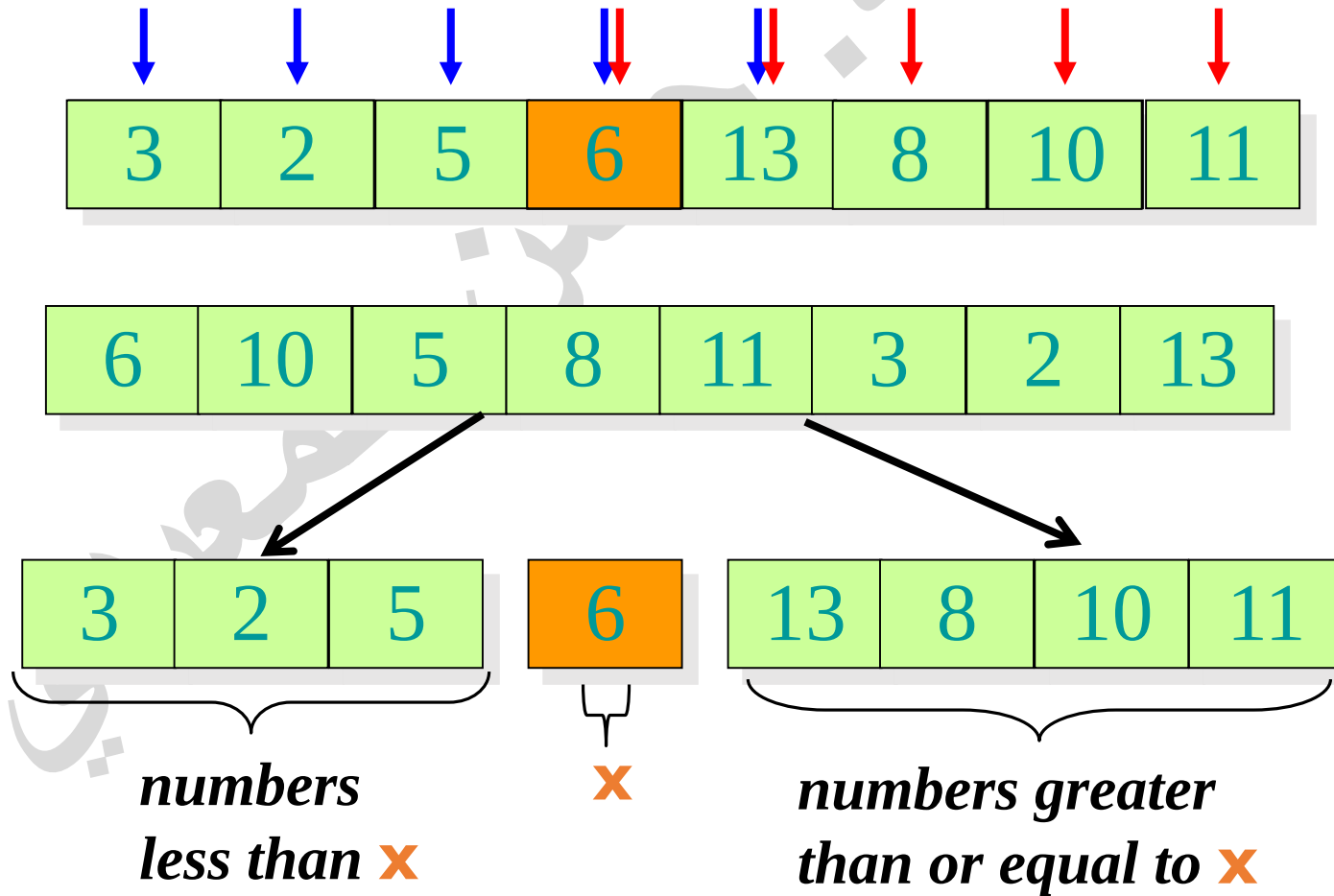
Quick-Sort Algorithm (In-Place Partition)

Self Check

Given the following list of numbers [14, 17, 13, 15, 19, 10, 3, 16, 9, 12] which answer shows the contents of the list after the second partitioning according to the quicksort algorithm?

- A. [9, 3, 10, 13, 12]**
- B. [9, 3, 10, 13, 12, 14]**
- C. [9, 3, 10, 13, 12, 14, 17, 16, 15, 19]**
- D. [9, 3, 10, 13, 12, 14, 19, 16, 15, 17]**
- E. None of the above**

Quick-Sort Algorithm (In-Place Partition)



Quick-Sort Algorithm (In-Place Partition)

Partition(A, p, r):

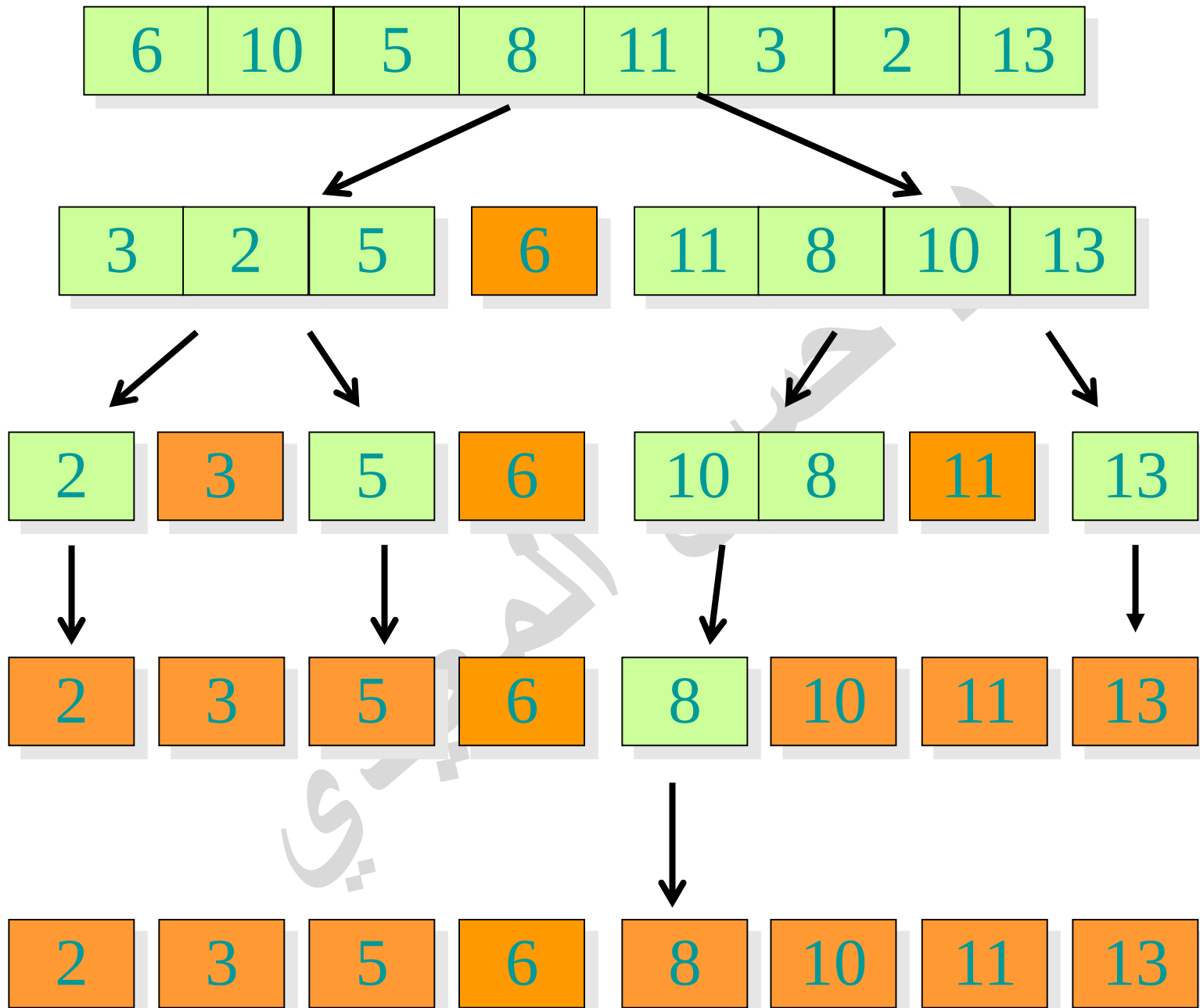
Select an element to act as the “pivot” (*which?*)

Grow two regions, A[p..i] and A[j..r]

All elements in A[p..i] $\leq$ pivot

All elements in A[j..r] $\geq$ pivot

→ Increment i until A[i] > pivot
Decrement j until A[j] < pivot
Swap A[i] and A[j]
Repeat until $i \geq j$
Swap A[j] and A[p]
Return j



Divide and Conquer – Quick-Sort Algorithm

Algorithm Quick-Sort(A, p, r)

1. IF ($p < r$) THEN
 - (a) Assign $q = \text{partition}(A, p, r)$
 - (b) Call Quick-Sort($A, p, q-1$)
 - (c) Call Quick-Sort($A, q+1, r$)
2. End If

Divide and Conquer – Quick-Sort Algorithm

Analysis of Quick-Sort Algorithm (Best-Case)

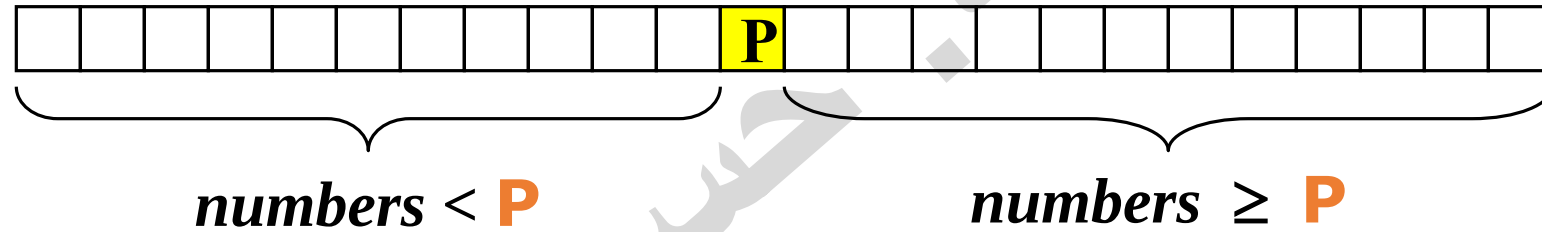
Algorithm Quick-Sort(A, p, r)

1. IF ($p < r$) THEN
 - (a) Assign $q = \text{partition}(A, p, r)$
 - (b) Call Quick-Sort($A, p, q-1$)
 - (c) Call Quick-Sort($A, q+1, r$)
2. End If

- Occurs when the subarrays are completely **balanced**.
- Have **$n/2$** elements in one subarray and **$n/2$** elements in the other subarray.

Divide and Conquer – Quick-Sort Algorithm

(Best-Case)



- Occurs when the subarrays are completely balanced.
- Have $n/2$ elements in one subarray and $n/2$ elements in the other subarray.

Divide and Conquer – Quick-Sort Algorithm

Analysis of Quick-Sort Algorithm (Best-Case)

$$T(n) = \text{Time to Divide} + \text{Time to Conquer} + \text{Time to Combine}$$

Divide and Conquer – Quick-Sort Algorithm

Analysis of Quick-Sort Algorithm (Worst-Case)

Algorithm Quick-Sort(A, p, r)

1. IF ($p < r$) THEN

(a) Assign $q = \text{partition}(A, p, r)$

(b) Call Quick-Sort($A, p, q-1$)

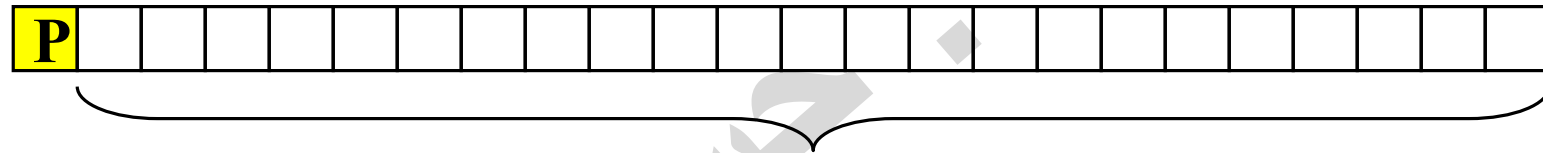
(c) Call Quick-Sort($A, q+1, r$)

2. End If

- Occurs when the subarrays are **completely unbalanced**.
- Have **zero** elements in one subarray and **(n-1)** elements in the other subarray.

Divide and Conquer – Quick-Sort Algorithm

(Worst-Case)



numbers $\geq$ P

- Occurs when the subarrays are completely unbalanced.
- Have zero elements in one subarray and **(n-1)** elements in the other subarray.

Divide and Conquer – Quick-Sort Algorithm

Analysis of Quick-Sort Algorithm (Worst-Case)

$T(n) =$

Time to Divide

+

Time to Conquer

+

Time to Combine

Divide and Conquer – Quick-Sort Algorithm

The running time of quick sort depends on the partitioning of the subarrays:

- If the subarrays are, then quick sort can run as fast as
- If they are, , then quick sort can run as slowly as

Divide and Conquer – Quick-Sort Algorithm

Summary	Divide	Conquer	Combine	Recurrence Equation
Best Case				
Worst Case				

Summarization of Sorting Algorithms

		Algorithm	Best Case	Worst Case
Traditional		Selection	$\Omega(n^2)$	$O(n^2)$
		Insertion	$\Omega(n)$	$O(n^2)$
		Bubble	$\Omega(n^2)$	$O(n^2)$
D & C		Merge	$\Omega(n \log n)$	$O(n \log n)$
		Quick	$\Omega(n \log n)$	$O(n^2)$

Binary-Search Algorithm

Searching Problem

Given a **collection of elements (array)** and an **key to find...**

Output

Print a message ("Found", "Not Found")

Return a value (position of key)

Don't modify the collection in the search!

Linear Search Algorithm

A search **traverses** the array until:

- The desired element is **found**
- Or the array is **exhausted**

If the array is **ordered**, I might not have to look at all elements.

- I can **stop looking when I know the element cannot be in the collection.**

Linear Search Algorithm

Un-Ordered Iterative Array Search

Search (A, Key)

Scan the array

```
for i = 1 ... N:  
    if A[i] = Key:  
        return "Found"
```

```
return ("Not found")
```


Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key:  
        return "Found"  
  
return ("Not found")
```

Key = 13

A	7	12	5	22	13	32
	1	2	3	4	5	6

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N: ←  
    if A[i] = Key:  
        return "Found"  
  
return ("Not found")
```

Key = 13

A	7	12	5	22	13	32
	1	2	3	4	5	6

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key: ←  
        return "Found"  
  
return ("Not found")
```

A	7	12	5	22	13	32
	1	2	3	4	5	6

Key = 13

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key: ←  
        return "Found"  
  
return ("Not found")
```

A	7	12	5	22	13	32
	1	2	3	4	5	6

Key = 13

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key: ←  
        return "Found"  
  
return ("Not found")
```

Key = 13

A	7	12	5	22	13	32
	1	2	3	4	5	6

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key: ←  
        return "Found"  
  
return ("Not found")
```

Key = 13

A	7	12	5	22	13	32
	1	2	3	4	5	6

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key: ←  
        return "Found"  
  
return ("Not found")
```

Key = 13

A	7	12	5	22	13	32
	1	2	3	4	5	6

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key: ←  
        return "Found"  
  
return ("Not found")
```

A	7	12	5	22	13	32
	1	2	3	4	5	6

Key = 13

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key:  
        return "Found" ←
```

```
return ("Not found")
```

Key is found

Key = 13

A	7	12	5	22	13	32
	1	2	3	4	5	6

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key:  
        return "Found"  
  
return ("Not found")
```

Best Case:
1 Comparison

Best Case: match with the first item

A	7	12	5	22	13	32
	1	2	3	4	5	6

Key = 7

Linear Search Algorithm

Search (A, Key)

```
for i = 1 ... N:  
    if A[i] = Key:  
        return "Found"  
  
return ("Not found")
```

Worst Case:
n Comparisons

Worst Case: match with the last item (or no match)

A	7	12	5	22	13	32
	1	2	3	4	5	6

Key = 32



Divide and Conquer – Binary Search Algorithm

Binary-Search Algorithm

Divide and Conquer – Binary Search Algorithm

Idea

- We have a **sorted array**.
- We want to determine if a **particular element** is in the array or not.
 - Once **found**, print or return (index).
 - If **not found**, indicate the element is not in the array.

Divide and Conquer – Binary Search Algorithm

Idea

- We have a **sorted array**.
- Cuts the “search space” **in half** each time.
- It works by comparing a search **key x** with the array's middle element $A[q]$.
 - If $x = A[q]$, successful search and return found
 - If $x > A[q]$, search $[q+1 \dots \text{top}]$
 - If $x < A[q]$, search $[\text{bottom} \dots q-1]$

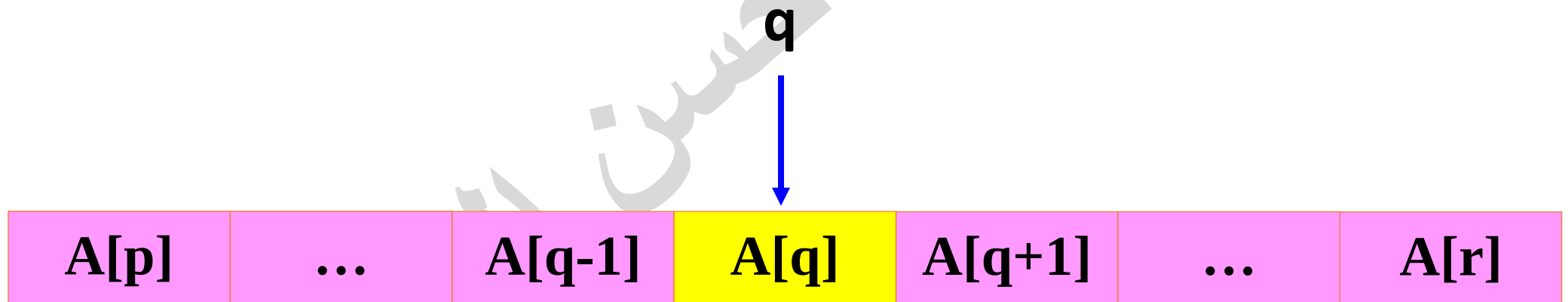
Divide and Conquer – Binary Search Algorithm

Idea

$A[p]$	...	$A[q-1]$	$A[q]$	$A[q+1]$	...	$A[r]$
--------	-----	----------	--------	----------	-----	--------

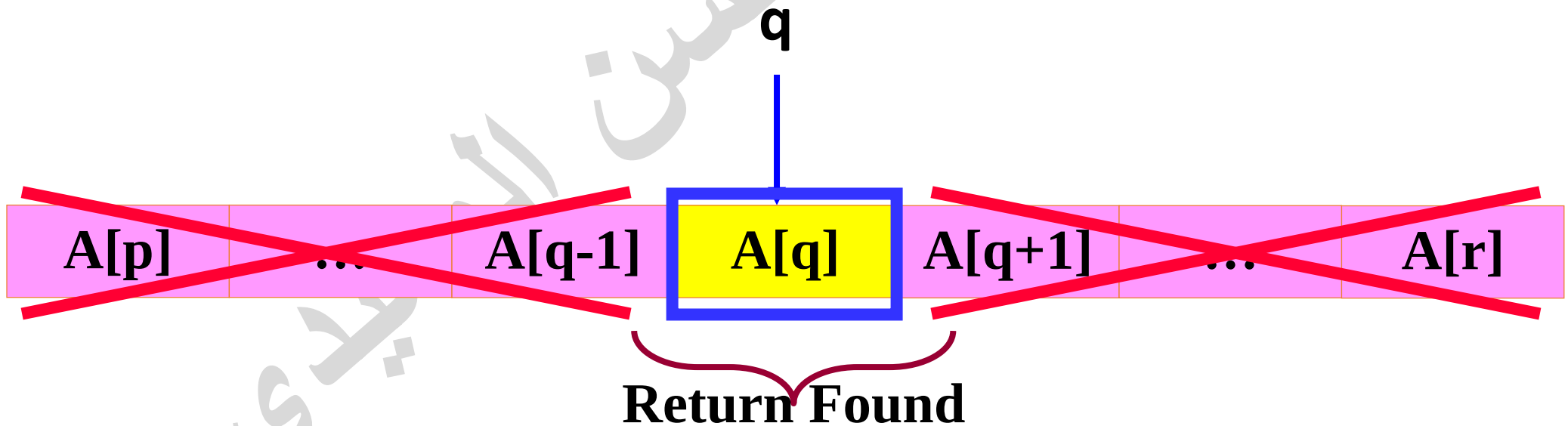
Divide and Conquer – Binary Search Algorithm

Find the middle element in the array (q)



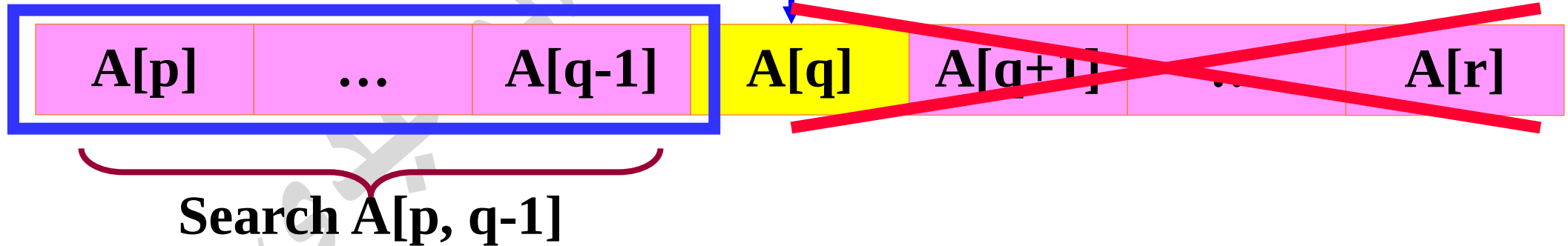
Divide and Conquer – Binary Search Algorithm

If $x = A[q]$



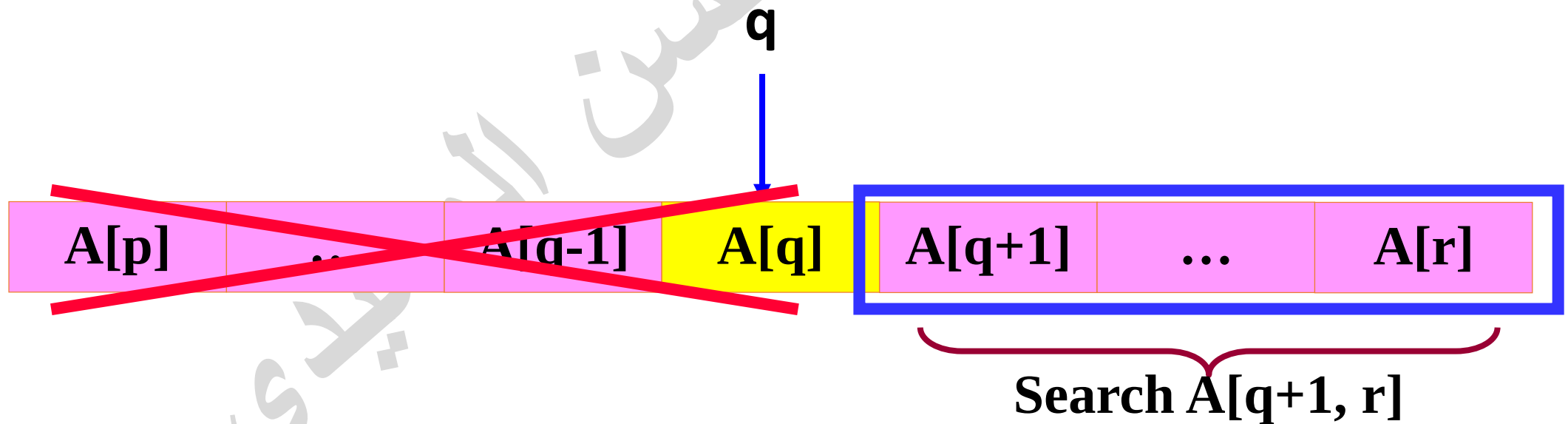
Divide and Conquer – Binary Search Algorithm

If $x < A[q]$



Divide and Conquer – Binary Search Algorithm

If $x > A[q]$



Divide and Conquer – Binary Search Algorithm

Full Example

Apply a binary-Search algorithm to check if a **key=34** is found or not.

p=0	1	2	3	4	5	6	r=7
2	4	7	10	18	22	34	57

X= 57

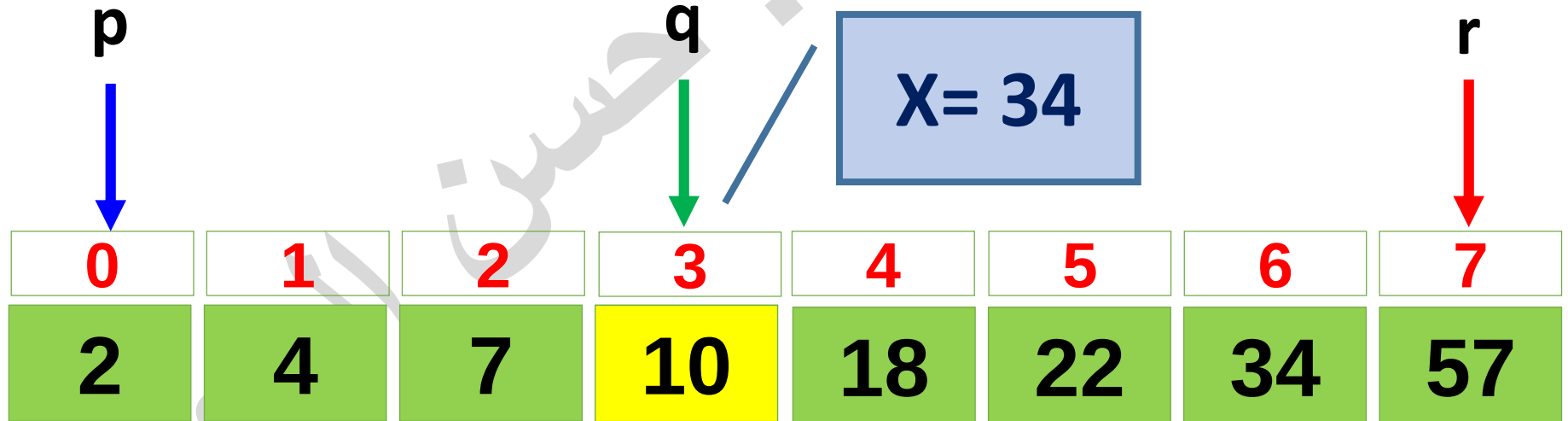
Divide and Conquer – Binary Search Algorithm

Iteration 1

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{0 + 7}{2} \right\rfloor$$

$$q = \lfloor 3.5 \rfloor = 3$$



If $x = A[q]$

If $x > A[q]$

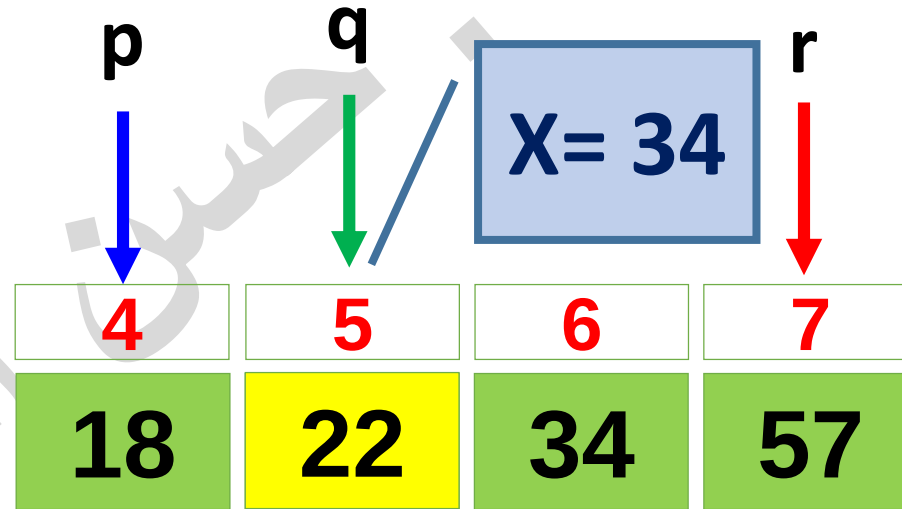
Divide and Conquer – Binary Search Algorithm

Iteration 2

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{4 + 7}{2} \right\rfloor$$

$$q = \lfloor 5.5 \rfloor = 5$$



If $x = A[q]$

If $x > A[q]$

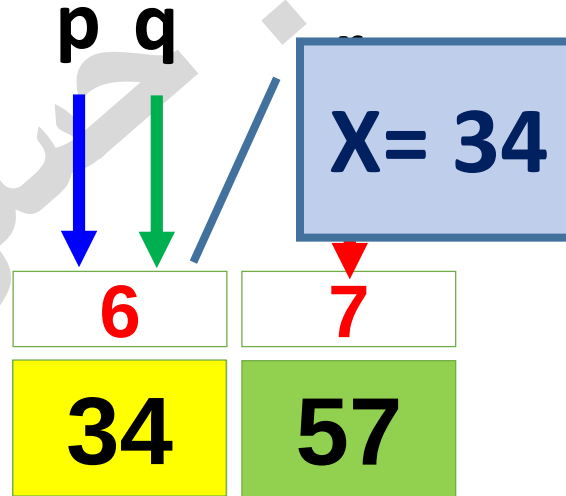
Divide and Conquer – Binary Search Algorithm

Iteration 3

$$q = \left\lfloor \frac{p + r}{2} \right\rfloor$$

$$q = \left\lfloor \frac{6 + 7}{2} \right\rfloor$$

$$q = \lfloor 6.5 \rfloor = 6$$



If $x = A[q]$

Divide and Conquer – Binary Search Algorithm

Algorithm BINARYSEARCH(A, p, r, x)

SET $q \leftarrow \lfloor (p+r)/2 \rfloor$

IF ($x = A[q]$) THEN

 RETURN true

ELSE

 IF ($x < A[q]$) THEN

 BINARYSEARCH (A, p, q-1, x)

 ELSE

 BINARYSEARCH(A, q+1, r, x)

 ENDIF

ENDIF

Divide and Conquer – Binary Search Algorithm

Analysis of Binary-Search Algorithm (Best-Case)

Algorithm BINARYSEARCH(A, p, r, x)

SET $q \leftarrow \lfloor (p+r)/2 \rfloor$

IF ($x = A[q]$) THEN

 RETURN true

ELSE

 IF ($x < A[q]$) THEN

 BINARYSEARCH (A, p, q-1, x)

 ELSE

 BINARYSEARCH(A, q+1, r, x)

 ENDIF

ENDIF

Divide and Conquer – Binary Search Algorithm

Analysis of Binary-Search Algorithm (Best-Case)

$T(n) =$

Time to Divide

+

Time to Conquer

+

Time to Combine

Divide and Conquer – Binary Search Algorithm

Analysis of Binary-Search Algorithm (Worst-Case)

Algorithm BINARYSEARCH(A, p, r, x)

SET $q \leftarrow \lfloor (p+r)/2 \rfloor$

IF ($x = A[q]$) THEN

 RETURN true

ELSE

 IF ($x < A[q]$) THEN

 BINARYSEARCH (A, p, q-1, x)

 ELSE

 BINARYSEARCH(A, q+1, r, x)

 ENDIF

ENDIF

Divide and Conquer – Binary Search Algorithm

Analysis of Binary-Search Algorithm (Worst-Case)

$T(n) =$

Time to Divide

+

Time to Conquer

+

Time to Combine

Divide and Conquer – Binary Search Algorithm

Solve the recurrence equation using the Master Method

$$T(n) = T\left(\frac{n}{2}\right) + \theta(1)$$

Summary of Search Algorithms

Linear Search

Best-Case

Worst-Case

Binary Search

Example

Assume you have a sorted array (A) with size=8.

Approximate the number of iterations did you need to find a key (x) in (A) in worst-case if you applied:

1. Linear Search Algorithm
2. Binary Search Algorithm.

المعدي

Example

Assume you have a sorted array (A) with size=**100**.

Approximate the number of iterations did you need to find a key (x) in (A) in worst-case if you applied:

1. Linear Search Algorithm
2. Binary Search Algorithm.

المعدي

Check - Chapter Learning Outcomes

CLO

4

- Understand the meaning of Divide and Conquer.
- Analyze the complexity of Merge-sort algorithm.
- Analyze the complexity of Quick-sort algorithm.
- Analyze the complexity of Binary-Search algorithm.

End of Chapter

4



5

Graphs Algorithms

Chapter Learning Outcomes

- Define a graph as a set of vertices and a set of edges.
- Determine the size of a graph.
- Distinguish between directed and undirected graphs.
- Use graphs to model situations in which the vertices represent objects, and edges represent a particular relationship between objects.
- Understand the following graph terminologies: (weighted graph, loop graph, multigraph, pseudograph, regular graph, complete graph, null graph).
- Find the degree of each vertex in directed and undirected graphs.
- Understand the handshaking theorem.
- Distinguish between connected and unconnected graphs.
- Apply graph traversal algorithms.
- Apply topological sort algorithm.
- Apply minimum spanning algorithms (Kruskal and Prim).
- Apply Dijkstra's algorithm to find the Single Source Shortest-Paths.

Introduction

Graphs – An Introduction

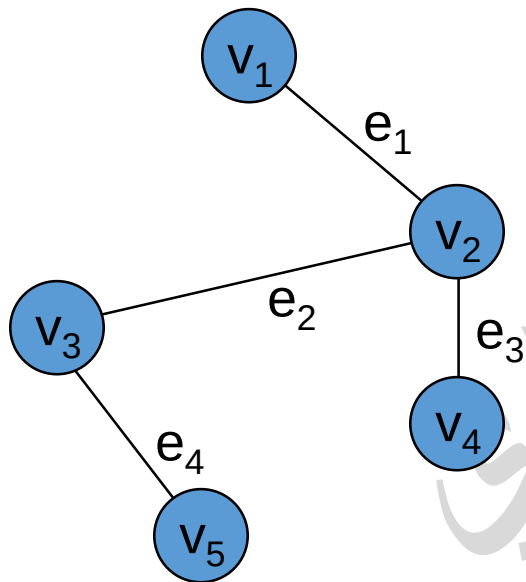
In many cases we are faced with a problem that is defined in terms of **a number of objects or entities** that have some **relationships among them**. The following are examples:

Examples

- Social Networks
- Travelling Salesperson
- Circuit Layout
- Project Scheduling
- Computer Networks
- Flight Scheduling

What is a Graph?

A Graph **G** consists of a set **V** of vertices or nodes and a set **E** of edges (arcs) that connect the vertices. We write $G=(V,E)$.



G

Define graph (G)

$G=(V,E)$

$V=\{v_1, v_2, v_3, v_4, v_5\}$

$E=\{e_1, e_2, e_3, e_4\}$

$e_1=(v_1, v_2)$

$e_2=(v_2, v_3)$

$e_3=(v_2, v_4)$

$e_4=(v_3, v_5)$

What is a Graph?

A graph G is defined as follows:

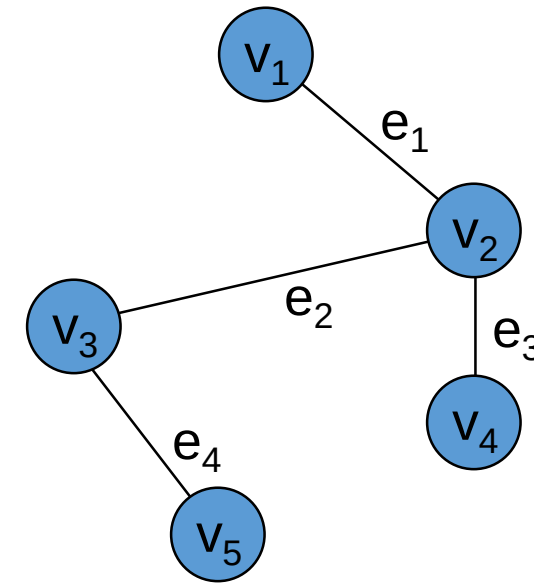
$$G=(V,E)$$

- $V(G)$: a finite, nonempty set of vertices.
- $E(G)$: a set of edges (pairs of vertices).
- An edge is a path between two nodes

Size of a Graph

Given Graph $G = (V, E)$ then :

- $n = |V| = \# \text{ vertices}$
- $m = |E| = \# \text{ edges}$
- size of G is $n + m$

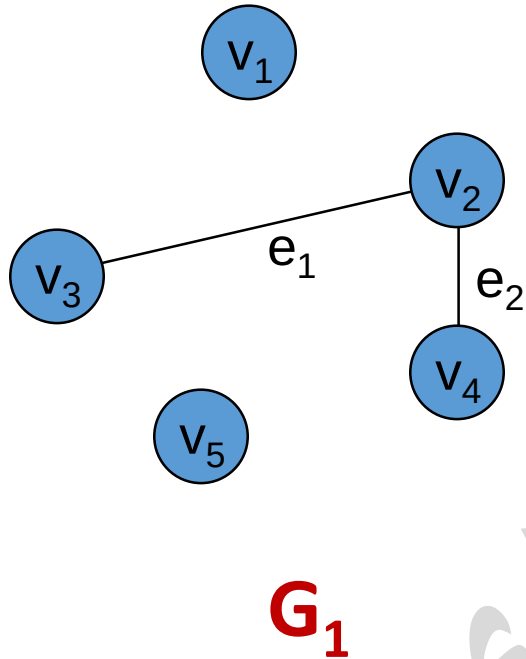


G

- $n = |V| = 5$
- $m = |E| = 4$
- $|G| = 5 + 4 = 9$

What is a Graph?

Define the following graphs using formal definition.



Define graph (G_1)

$$G_1 = (V, E)$$

$$V = \{v_1, v_2, v_3, v_4, v_5\}$$

$$E = \{e_1, e_2\}$$

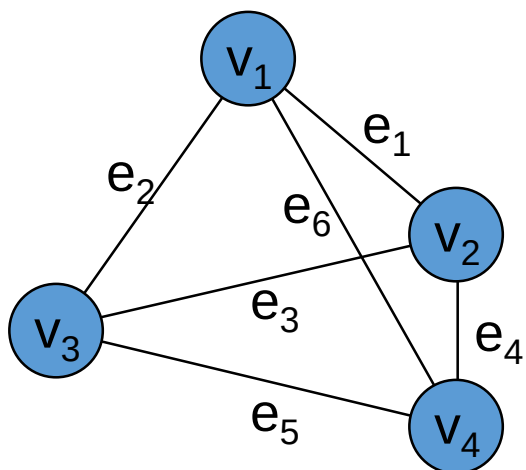
$$e_1 = (v_2, v_3)$$

$$e_2 = (v_2, v_4)$$

- $n = |V| = 5$
- $m = |E| = 2$
- $|G_1| = 5 + 2 = 7$

What is a Graph?

Define the following graphs using formal definition.



G_2

Define graph (G_2)

$G_2 = (V, E)$

$V = \{v_1, v_2, v_3, v_4\}$

$E = \{e_1, e_2, e_3, e_4, e_5, e_6\}$

$e_1 = (v_1, v_2)$ $e_2 = (v_1, v_3)$

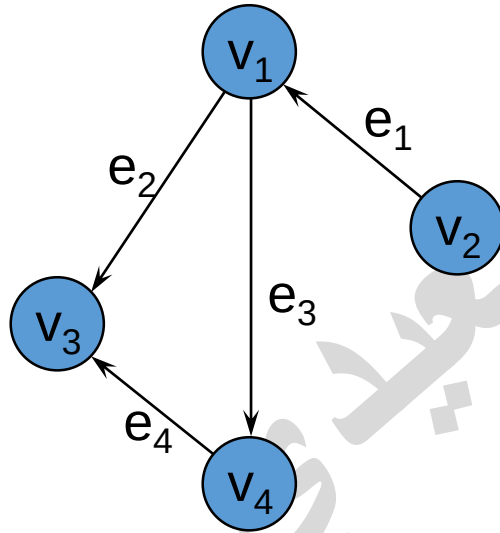
$e_3 = (v_2, v_3)$ $e_4 = (v_2, v_4)$

$e_5 = (v_3, v_4)$ $e_6 = (v_1, v_4)$

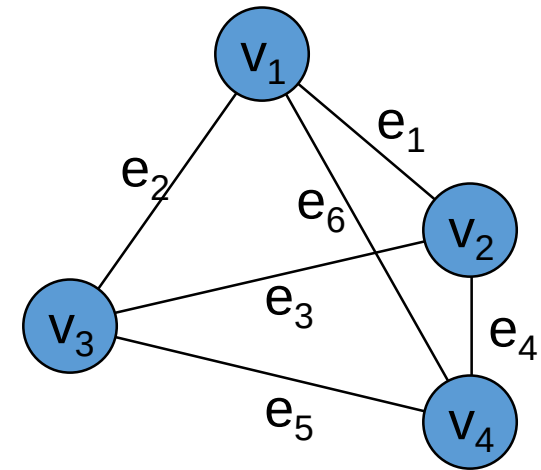
- $n = |V| = 4$
- $m = |E| = 6$
- $|G_2| = 4 + 6 = 10$

Graph Types (Directed & Undirected)

In some cases we want the edges to have directions associated with them; we call such a graph a directed graph or a digraph.



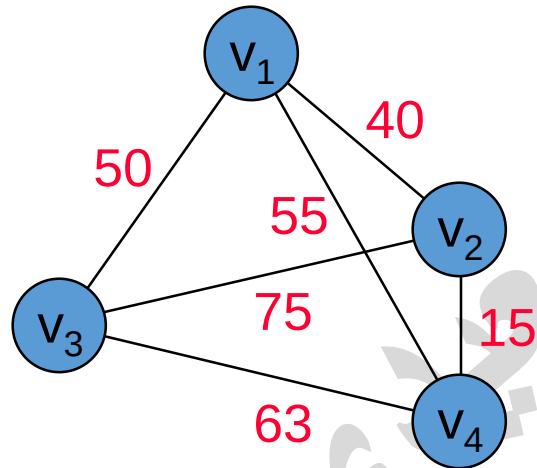
Directed (Digraph)



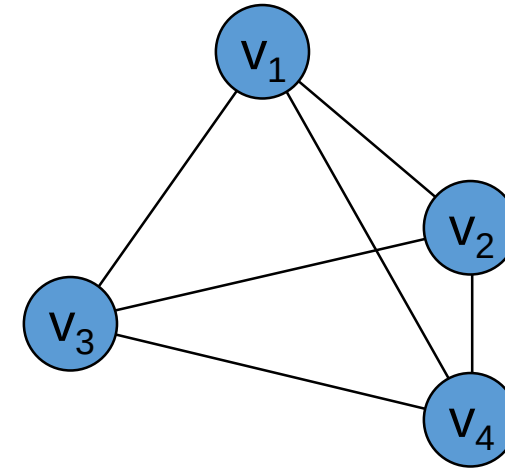
Undirected

Graph Types (Weighted & Unweighted)

In some cases, we want to associate a weight with each edge in the graph. Such a graph is known as a weighted graph.



Weighted Graph

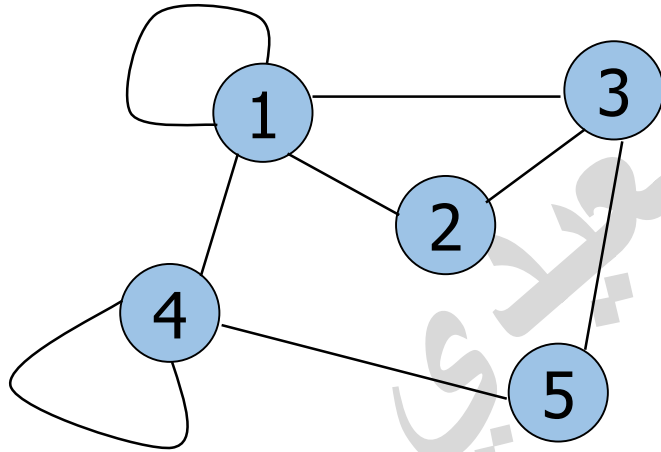


Unweighted Graph

Graph Types (Loop Graph & Multigraph)

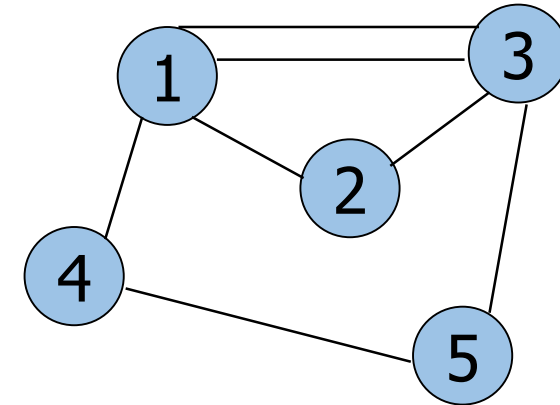
A **loop graph** is a graph that has a **loop**.

a **loop** is an edge that **connects a vertex to itself**



Loop Graph

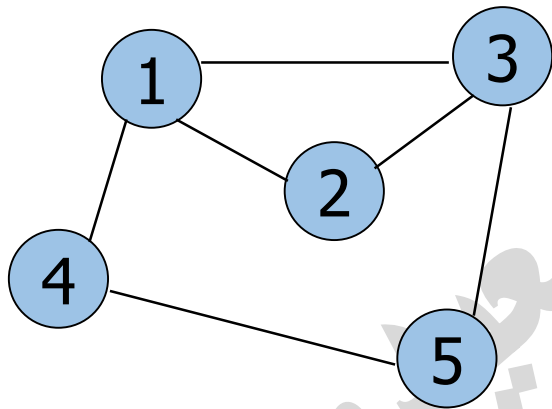
A **multigraph** is a graph that can have **more than one edge between a pair of vertices**.



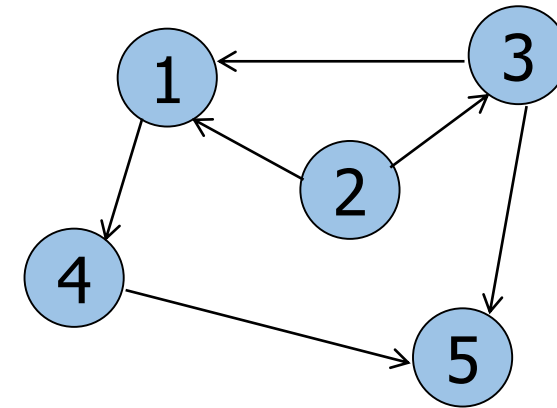
Multigraph

Graph Types (Simple Graph)

A **simple graph** (proper graph) has **no loops** and **no multiple edges**.



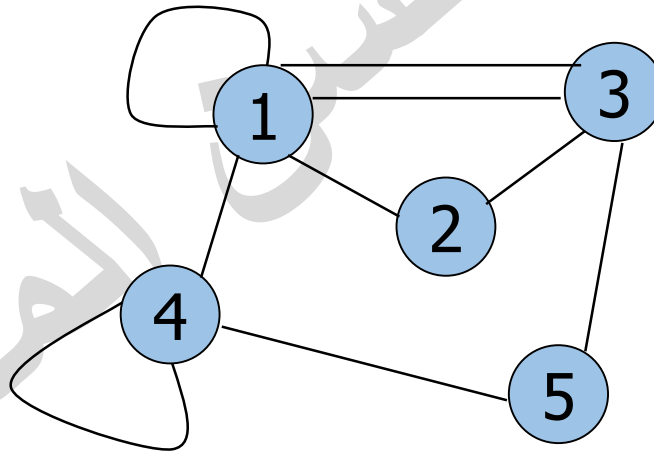
Simple Graph



Simple Digraph

Graph Types (Pseudograph)

A **pseudograph graph** may include **loops** as well as **multiple edges**.



Pseudograph

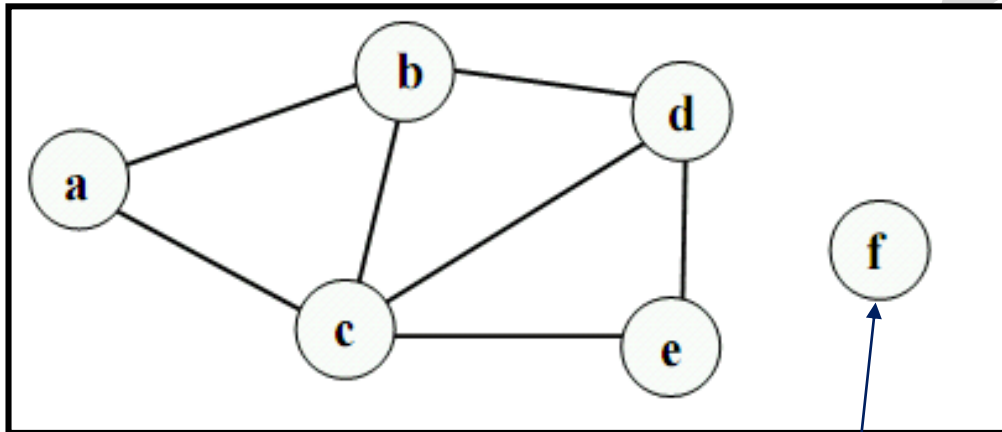
Summary

Term	Edge Type	Multiple Edges Ok?	Loop Ok?
Simple Graph	Undirected	No	No
Loop Graph	Undirected	No	Yes
Multigraph	Undirected	Yes	No
Pseudograph	Undirected	Yes	Yes
Simple Directed Graph	Directed	No	NO
Directed Loop Graph	Directed	No	Yes
Directed Multigraph	Directed	Yes	Yes
Mixed Graph	Directed & Undirected	Yes	Yes

Undirected Node Degree

Degree in undirected graphs:

Degree(v_i) = # of adjacent (incident) edges to vertex v_i in G .



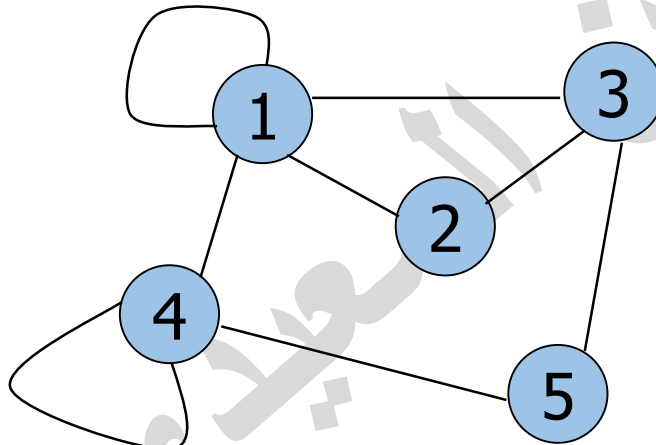
Isolated Vertex

Vertex	Degree
a	
b	
c	
d	
e	
f	

Undirected Node Degree

Example:

Find all the degrees for each vertex in the following graph.

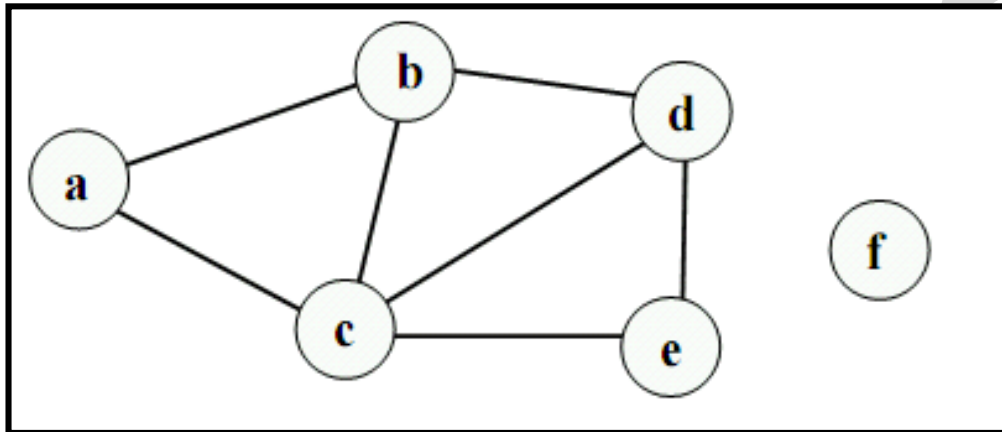


Vertex	Degree
1	
2	
3	
4	
5	
Total	

Neighborhoods of Vertices

Degree in undirected graphs:

$N(v_i)$ = Set of all neighbors of a vertex (v_i).



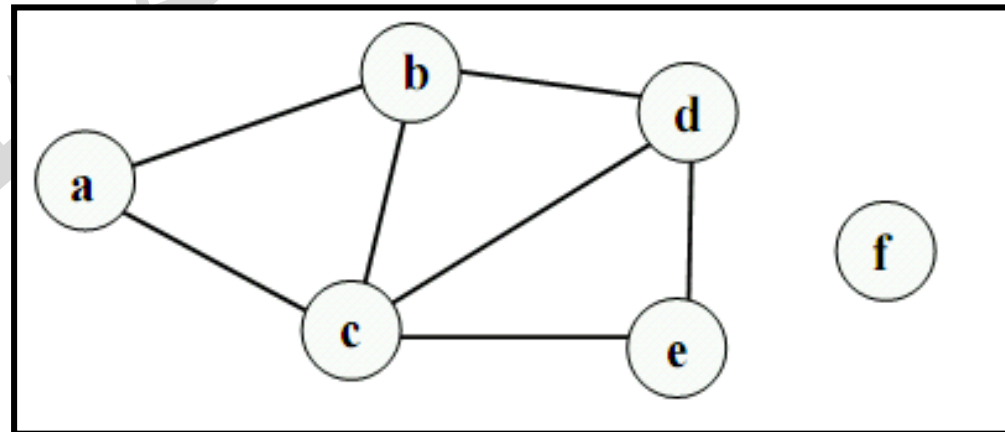
Vertex	Neighborhoods
a	
b	
c	
d	
e	
f	

Handshaking Theorem

(Handshaking Theorem): If $G = (V, E)$ is an undirected graph with m edges, then

$$2m = \sum_{v \in V} \deg(v)$$

must be even



Vertex	Degree
a	2
b	3
c	4
d	3
e	2
f	0
Total	14

Handshaking Theorem (Examples)

Example 1

How many edges are there in a graph with **10** vertices of degree **6**?

$$2m = \sum_{v \in V} \deg(v)$$

Solution

Handshaking Theorem (Examples)

Example 2

If a graph has **5** vertices, can each vertex have degree **3**?

$$2m = \sum_{v \in V} \deg(v)$$

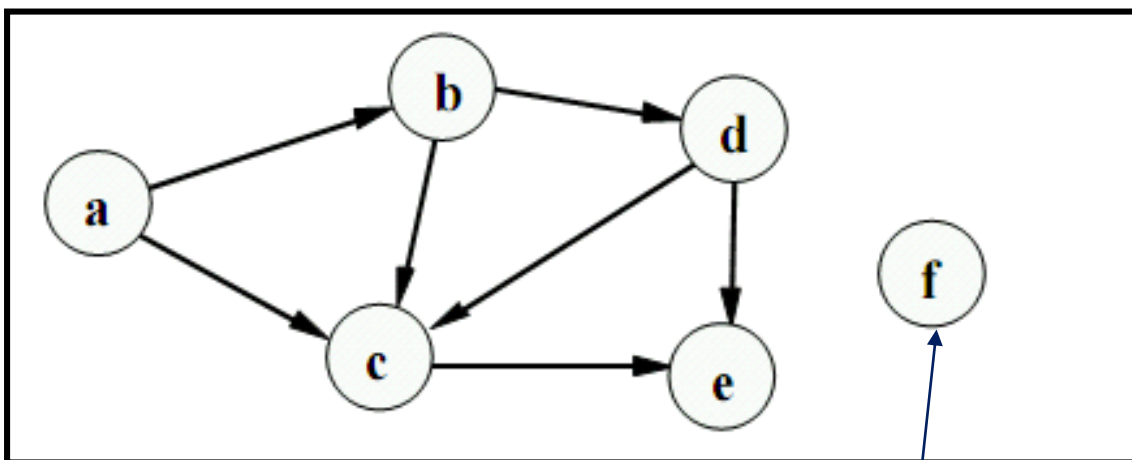
Solution

This is **not possible** by the handshaking theorem, because the sum of the degrees of the vertices **$3 * 5 = 15$** is odd.

Directed Node Degree

Degree in directed graphs:

- **In-Deg(V)** or **Deg⁻(V)** = # of incoming edges.
- **Out-Deg(V)** or **Deg⁺(V)** = # of outgoing edges.



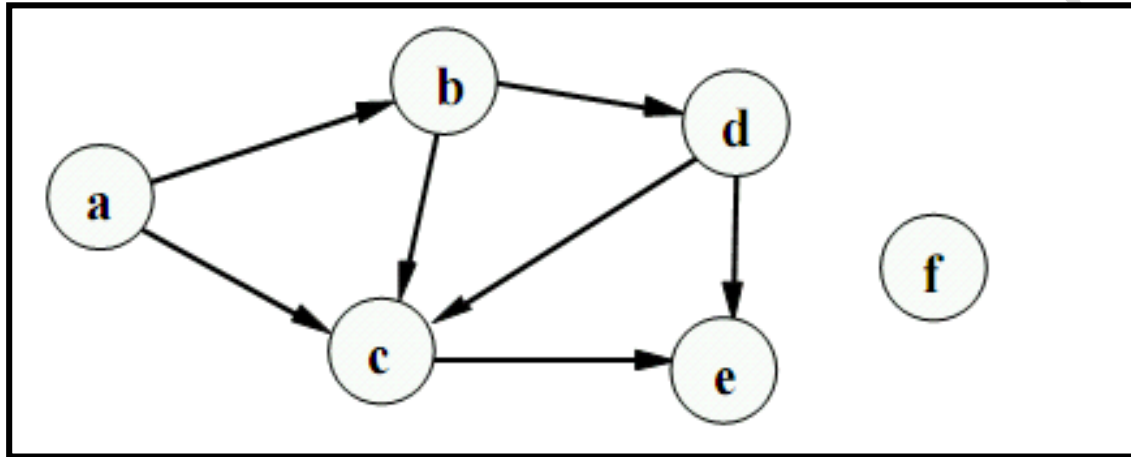
Isolated Vertex

Vertex	In-Degree	Out-Degree
a		
b		
c		
d		
e		
f		

Directed Node Degree

If $G = (V, E)$ is a directed graph with m edges, then

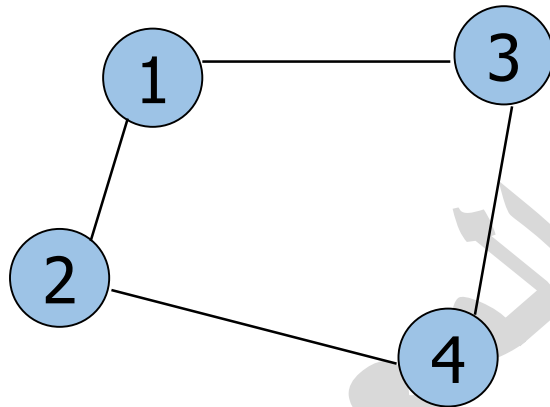
$$m = |E| = \sum_{v \in V} \deg^+(v) = \sum_{v \in V} \deg^-(v)$$



Vertex	Deg ⁻ (v)	Deg ⁺ (v)
a		
b		
c		
d		
e		
f		
Total		

Graph Types (Regular Graph)

Regular graph (G_R) is an undirected graph where each vertex has the same number of neighbors (**every vertex has the same degree**).

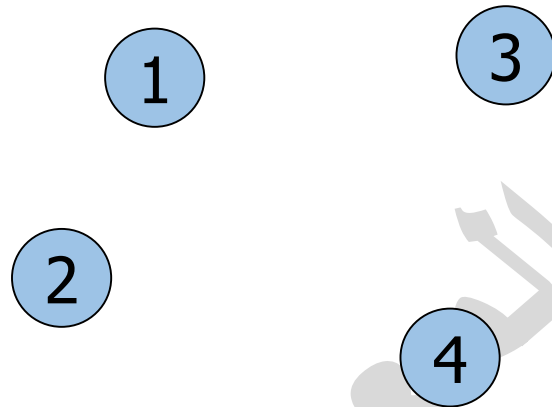


Regular Graph

Vertex	Degree
1	
2	
3	
4	
Total	

Graph Types (Null Graph)

Null graph (G_N) is a graph containing no edges

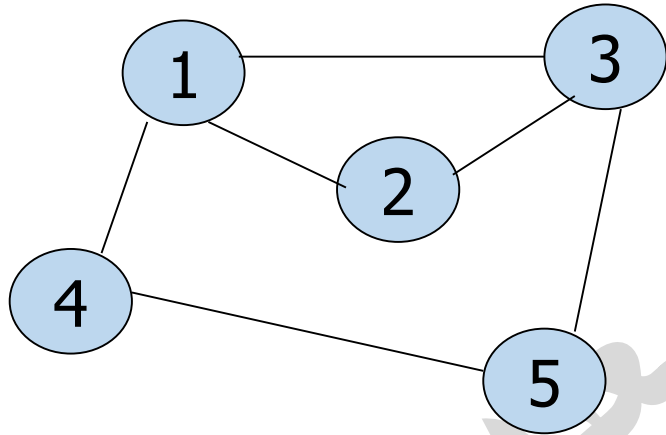


Null Graph

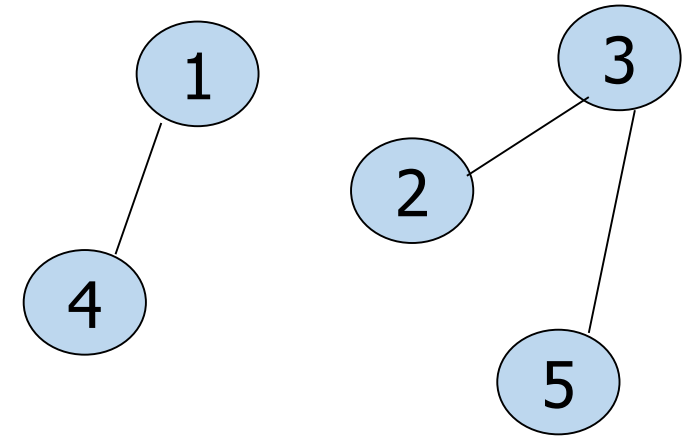
Vertex	Degree
1	
2	
3	
4	
Total	

Graph Types (Connected Graph)

A graph G is called **connected** if there is at least one path from every vertex to every other vertex in the graph



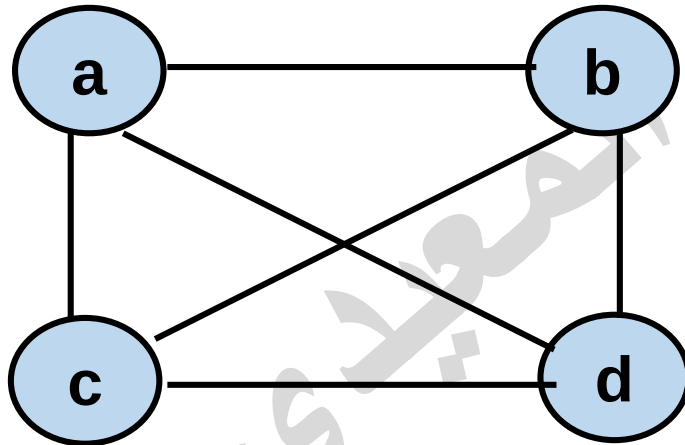
Connected Graph



Unconnected Graph

Graph Types (Complete Graph)

- A graph G is **Completed Graph** (G_c) iff every node u in G is adjacent to every other node v in G .
- A completed graph is already connected.
- It has $n(n-1)/2$ edges.
- The degree for each vertex = $(n-1)$



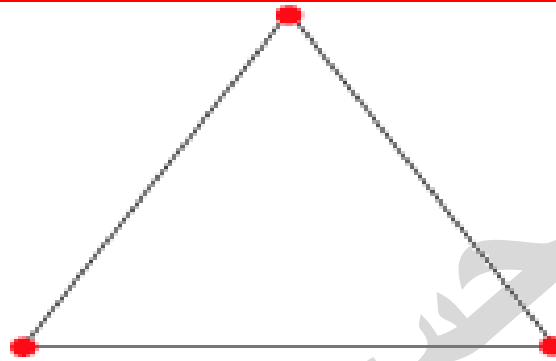
Completed Graph

Vertex	Degree
a	
b	
c	
d	
Total	

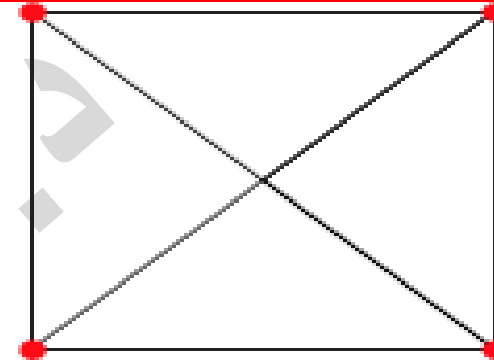
Graph Types (Complete Graph)



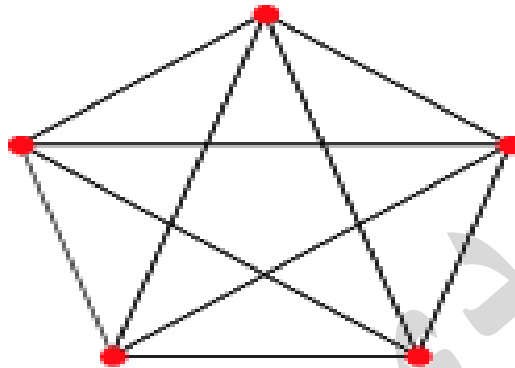
K_2



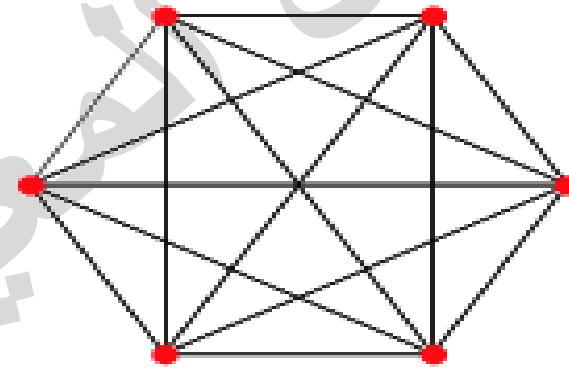
K_3



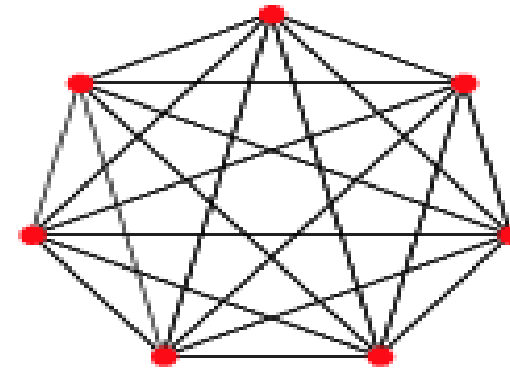
K_4



K_5



K_6



K_7

Completed Graphs

Graph Representation

Graph Representation

There are two ways to represent a graph:

1. **Adjacency Matrix** (Array-Based Implementation).
2. **Adjacency List** (Linked-List Implementation).

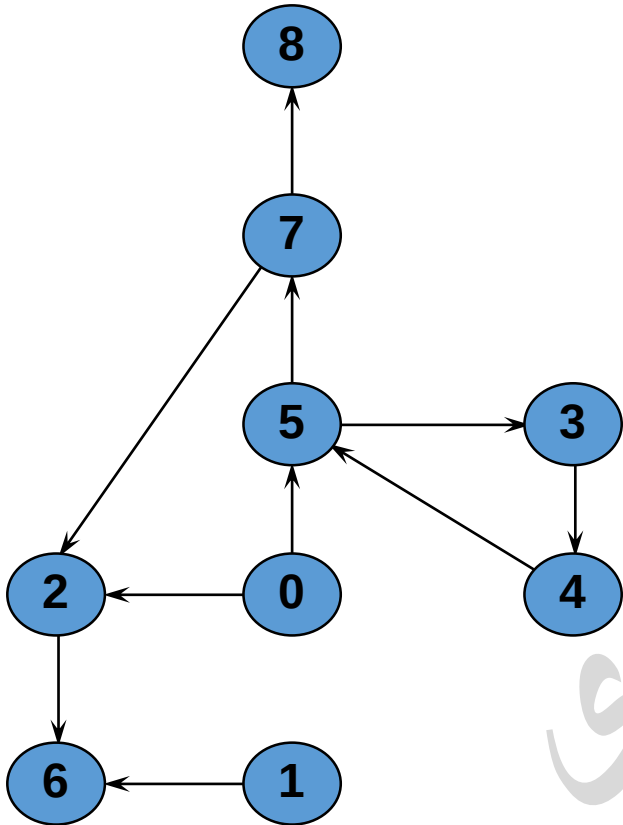
المعيني

Graph Representation

The **adjacency matrix** for a graph $G=(V,E)$ with n (or $|V|$) vertices numbered $0, 1, \dots, n-1$ is an **$n \times n$ array M** such that $M[i][j] = 1$ if and only if there is an edge from vertex i to vertex j .

The **adjacency list** for a graph $G=(V,E)$ with n vertices numbered $0, 1, \dots, n-1$ consists of **n linked lists**. The i^{th} linked list has a node for vertex j if and only if the graph contains an edge from vertex i to vertex j .

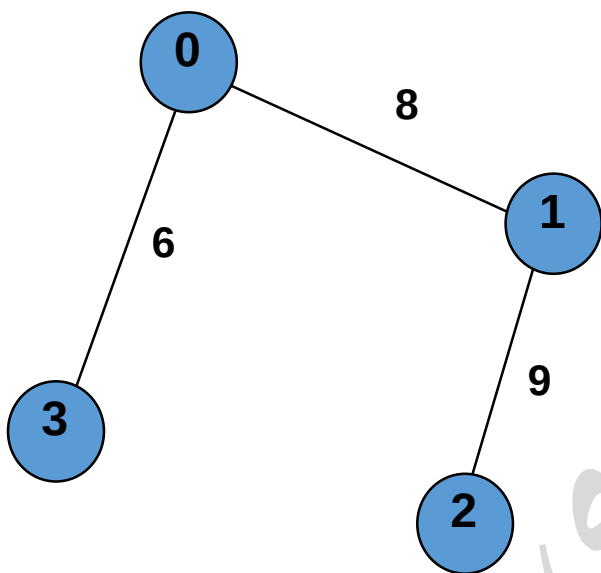
Adjacency Matrix



	0	1	2	3	4	5	6	7	8
0	0	0	1	0	0	1	0	0	0
1	0	0	0	0	0	0	1	0	0
2	0	0	0	0	0	0	1	0	0
3	0	0	0	0	1	0	0	0	0
4	0	0	0	0	0	1	0	0	0
5	0	0	0	1	0	0	0	1	0
6	0	0	0	0	0	0	0	0	0
7	0	0	1	0	0	0	0	0	1
8	0	0	0	0	0	0	0	0	0

Adjacency Matrix

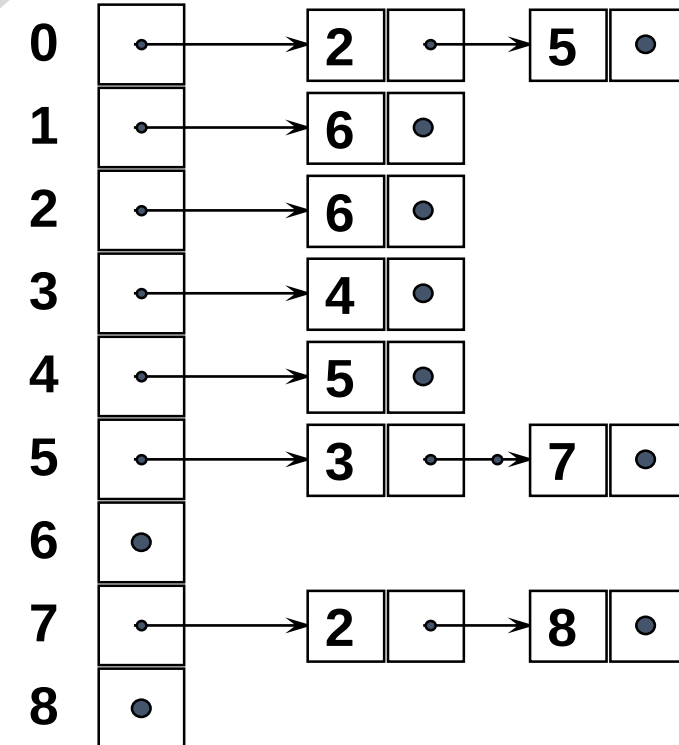
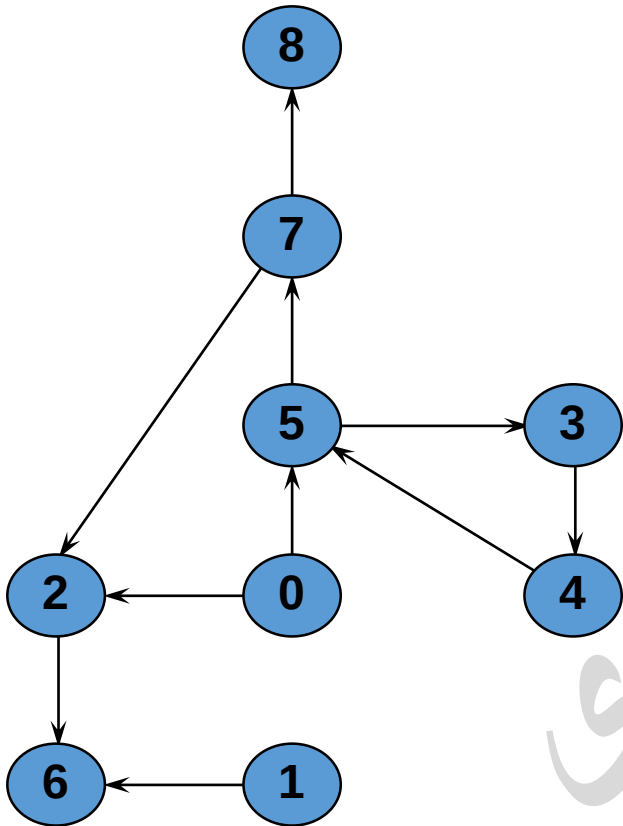
Adjacency Matrix



	0	1	2	3
0	∞	8	∞	6
1	8	∞	9	∞
2	∞	9	∞	∞
3	6	∞	∞	∞

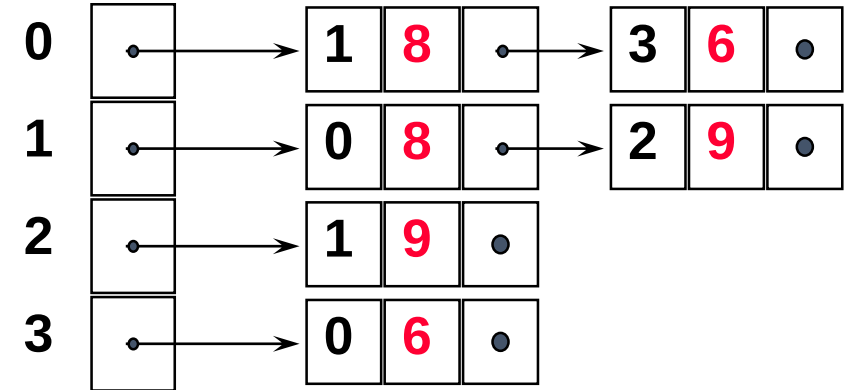
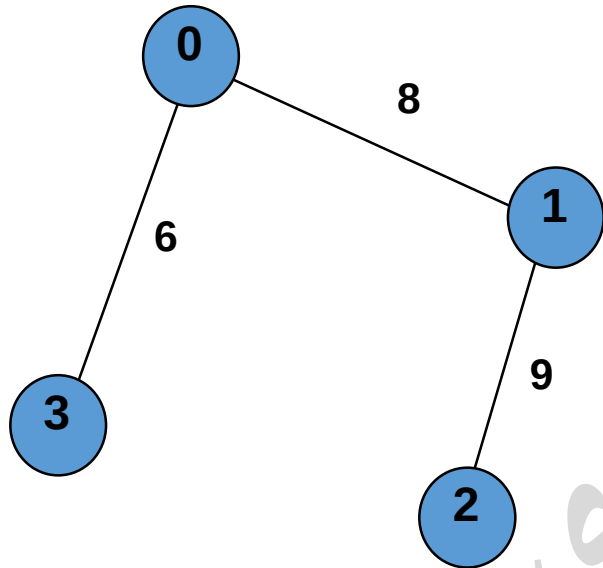
Adjacency Matrix

Adjacency List



Adjacency List

Adjacency List



Adjacency List

Graph Representation (Efficiency)

Space (Memory)

Matrix: n rows * n columns $\rightarrow O(n^2)$.

List : size of vertices + size of pointers (no. of edges)
 $O(|V| + |E|)$

Graph Representation (Summary)

Method	Graph Type	Size
Adjacency Matrix	Undirected Graph	n^2
	Directed Graph	n^2

Method	Graph Type	Size
Adjacency List	Undirected Graph	$n + 2m$
	Directed Graph	$n + m$

Graph Representation

EXAMPLE

If an undirected graph (G) has **5** vertices and **4** edges.

1- How many nodes did you need to represent G using an adjacency matrix?

2- How many nodes did you need to represent G using an adjacency list?

Graph Representation

EXAMPLE

If a directed graph (G) has **5** vertices and **4** edges.

1- How many nodes did you need to represent G using an adjacency matrix?

2- How many nodes did you need to represent G using an adjacency list?

Graph Traversal

Graph Searching (Traversal)

Problem:

Find a path between two nodes of the graph

Standard Graph-Searching Algorithms:

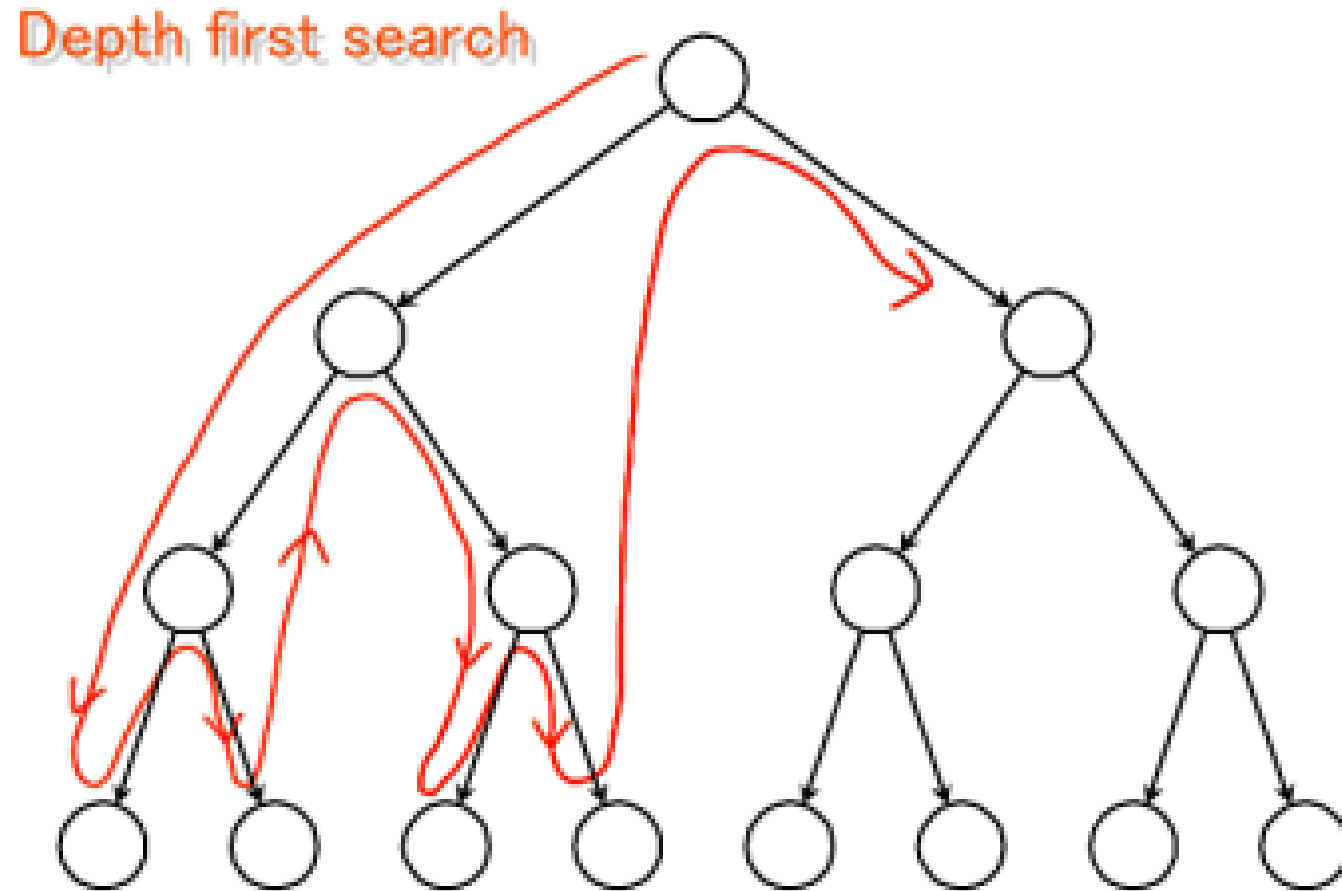
- **Depth-First-Search (DFS)**
- **Breadth-First-Search (BFS)**

Depth-First Search

DFS

Depth-First Search (DFS)

- What is the idea behind DFS?
 - Travel as far as you can down a path.
 - Back up *as little as possible* when you reach a "dead node":
 - » Next vertex has been "visited".
 - » There is no next vertex.
- DFS can be implemented efficiently using a **stack**.
- Works with directed and undirected graphs
- Works with weighted and unweighted graphs



Depth-First Search - Algorithm

Procedure DFS (L : adjacencyList, v : vertex)

initialize *stack*.

visit, mark and push(v).

while *stack* is nonempty do

while there is an unmarked vertex w adjacent to top(*stack*) do

visit, mark and push(w).

end

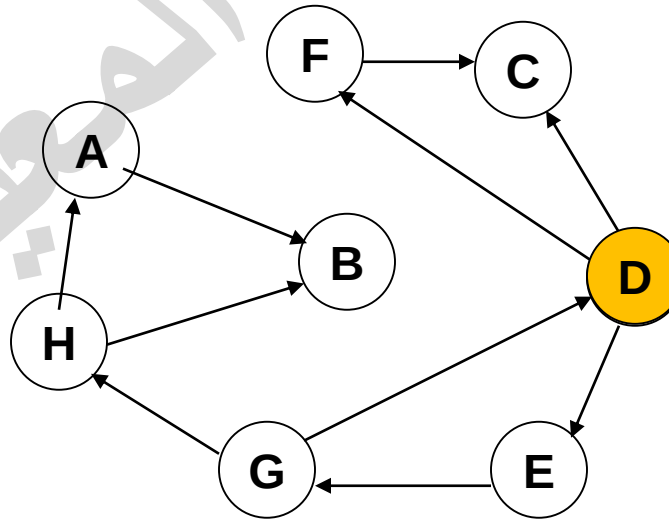
pop(*stack*)

end.

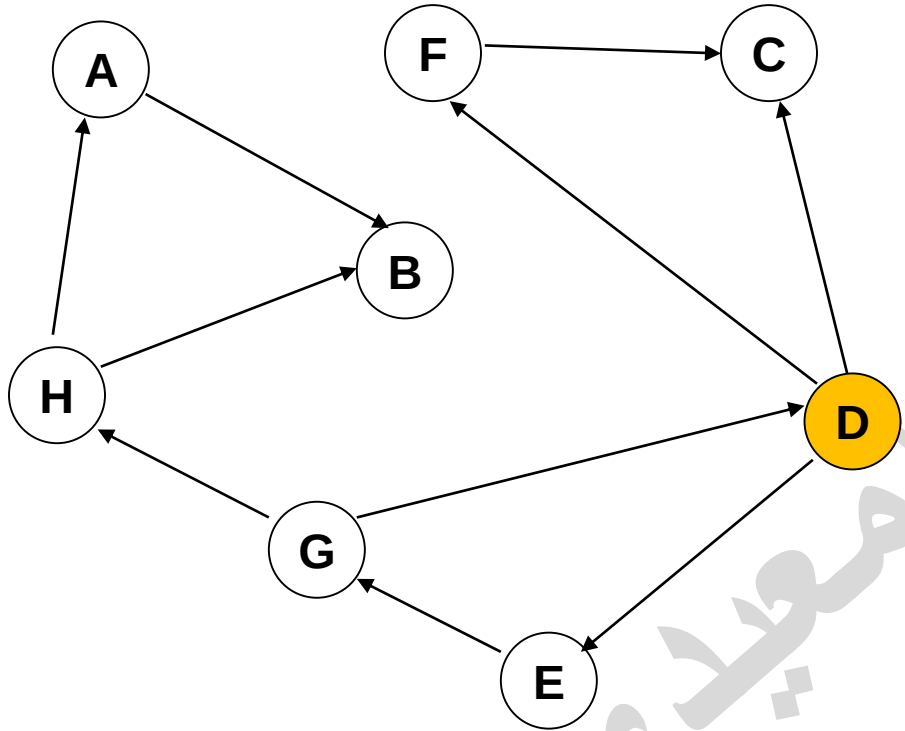
♦ Time complexity = $O(|V|+|E|)$

Depth-First Search - Example

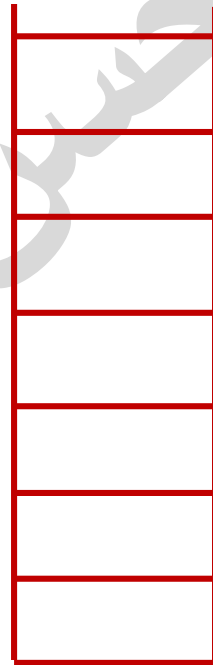
Apply the DFS algorithm to the following graph starting with vertex **D**. Determine the **discover (start)** and **finish times** for each vertex? (Always visit adjacent nodes in alphabetical order)



Depth-First Search - Example



Give the visited node order



Stack

Vertex	Discovery Time	Finish Time
A		
B		
C		
D		
E		
F		
G		
H		

Breadth-First Search

BFS

Breadth-First Search (BFS)

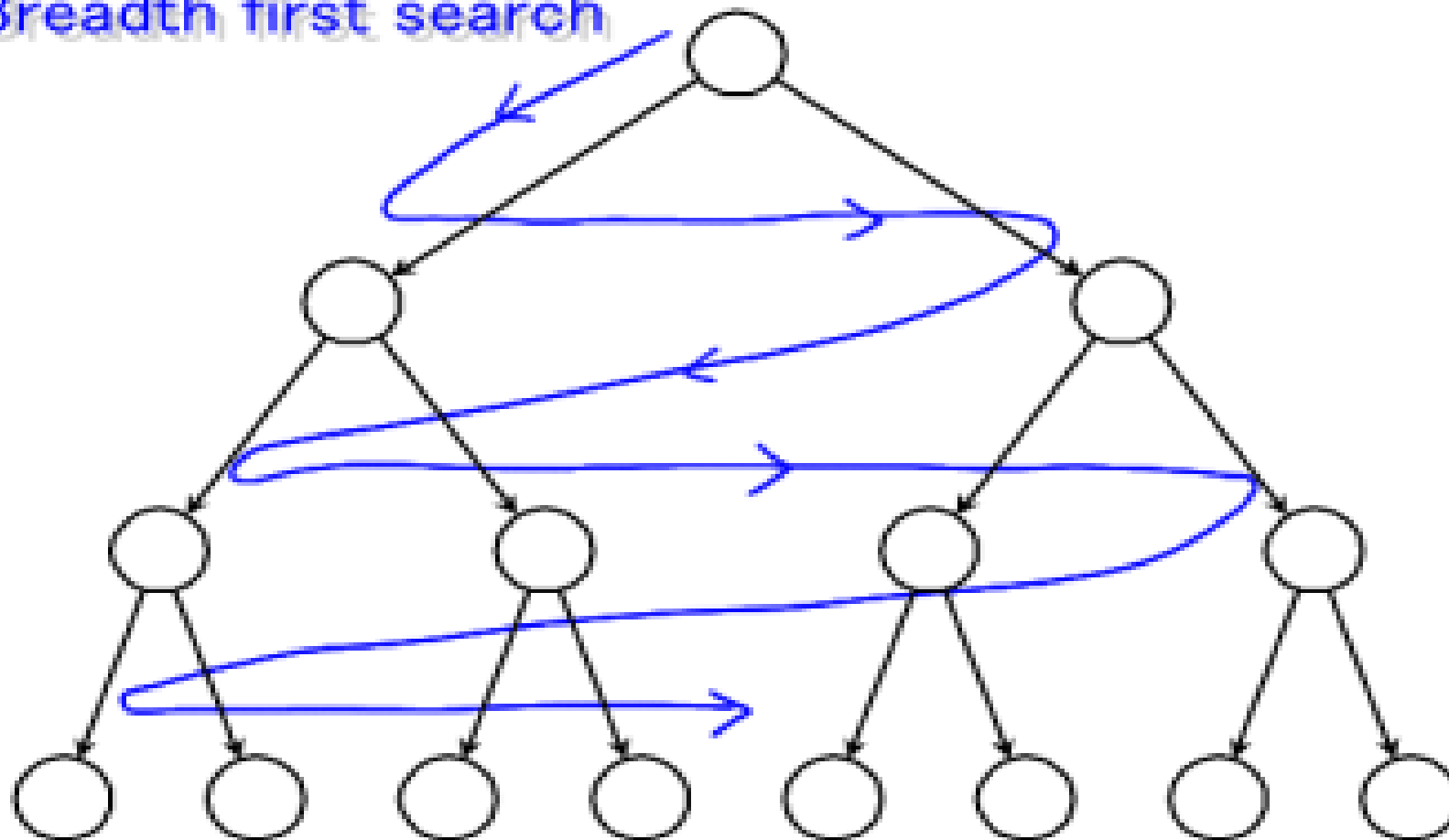
What is the idea behind BFS?

- Look at all possible paths at the same depth before you go at a deeper level
- Back up *as far as possible* when you reach a "dead end" (i.e., next vertex has been "marked" or there is no next vertex)

BFS can be implemented efficiently using a **queue**

Breadth-First Search (BFS)

Breadth first search



Breadth-First Search - Algorithm

Procedure BFS (L : adjacencyList, v : vertex)

initialize the queue Q .

visit and mark v .

enqueue (v, Q).

while Q is nonempty **do**

x = the next vertex in the queue Q

for each unmarked vertex w adjacent to x **do**

visit and mark w

enqueue(w, Q)

end

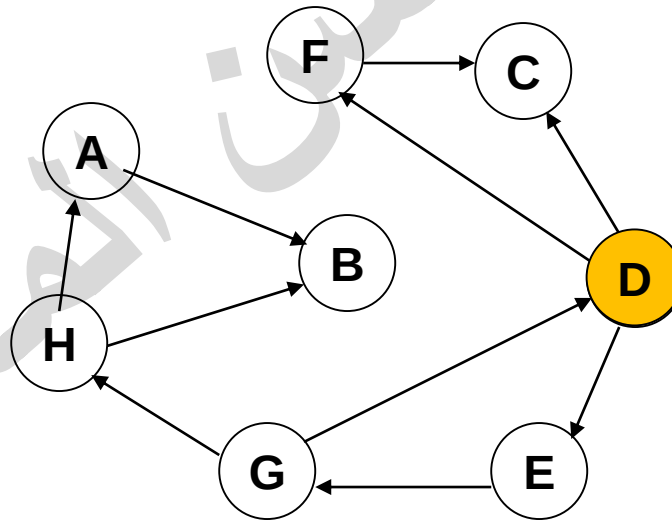
remove x from the queue Q

end.

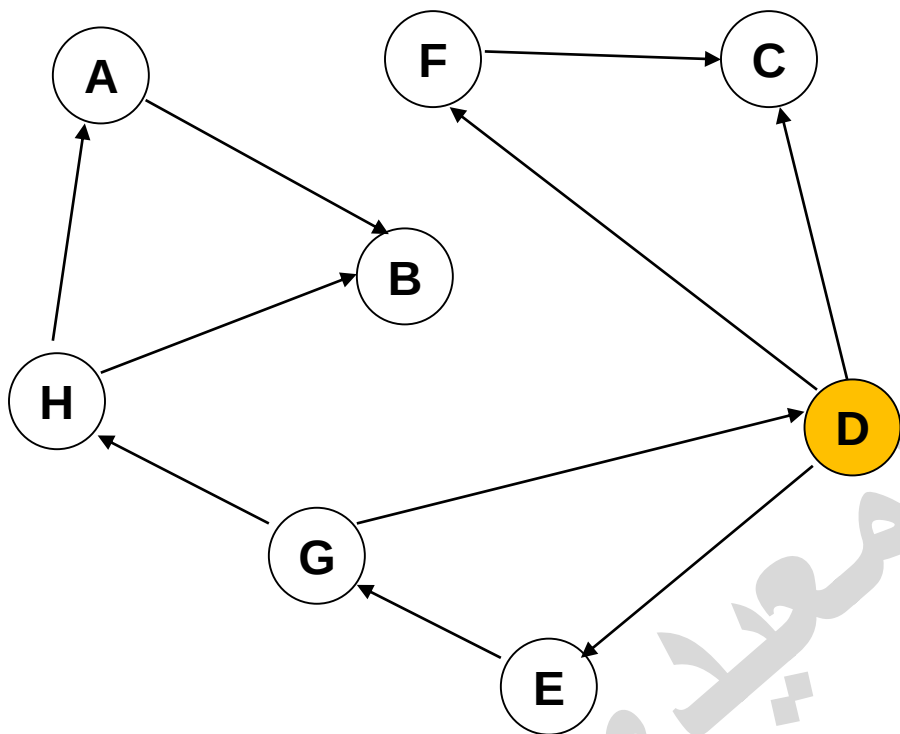
♦ Time complexity = $O(|V|+|E|)$

Breadth-First Search - Example

Apply the BFS algorithm to the following graph starting with vertex **D**. Determine the **discover (start) time** for each vertex?
(Always visit adjacent nodes in alphabetical order)



Breadth-First Search - Example



Give the visited node order



Queue

Vertex	Discovery Time
A	
B	
C	
D	
E	
F	
G	
H	

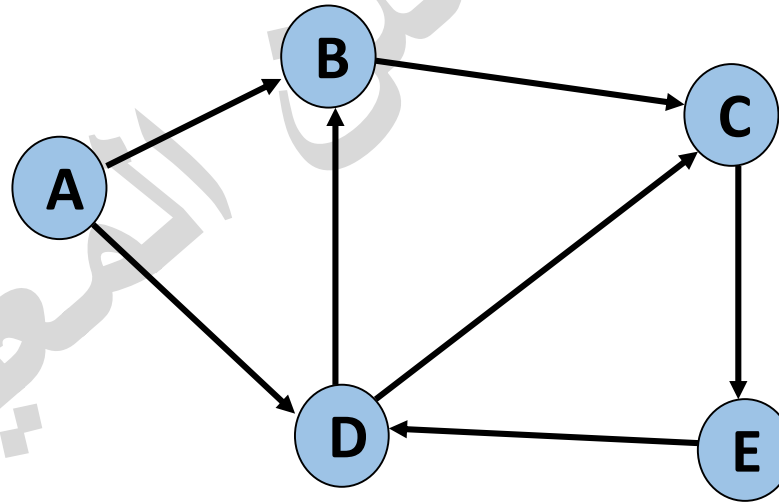
Topological Sort Algorithm

Path in Graph Theory

Path

A **path** is a sequence of vertices with the property that each vertex in the sequence is adjacent to the vertex next to it.

A path that **does not repeat vertices** is called a **simple path**.



Path : A, B, C, E, D, C, E

Simple Path : A, B, C, E, D

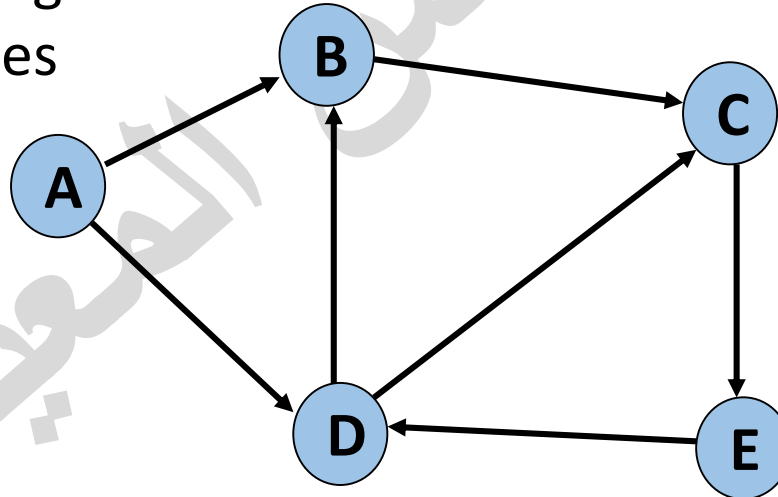
Circuit & Cycle in Graph Theory

Circuit

A **circuit** is path that begins and ends at the same vertex.

Cycle

A **cycle** is path that begins and ends at the same vertex, and doesn't repeat vertices

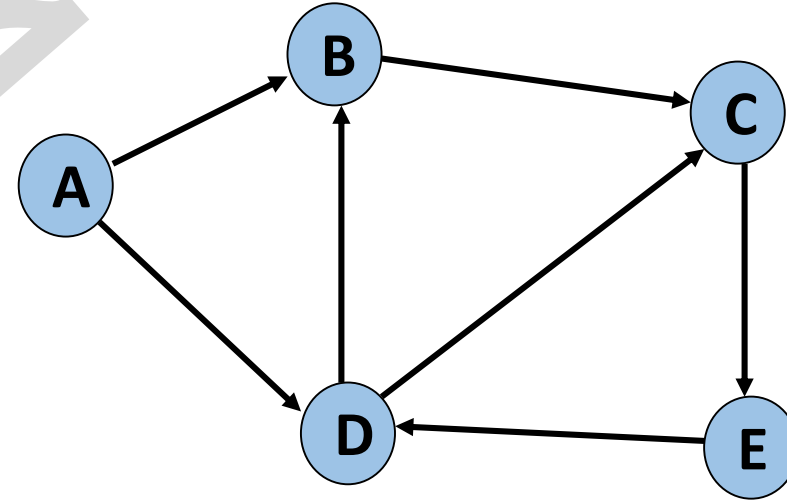
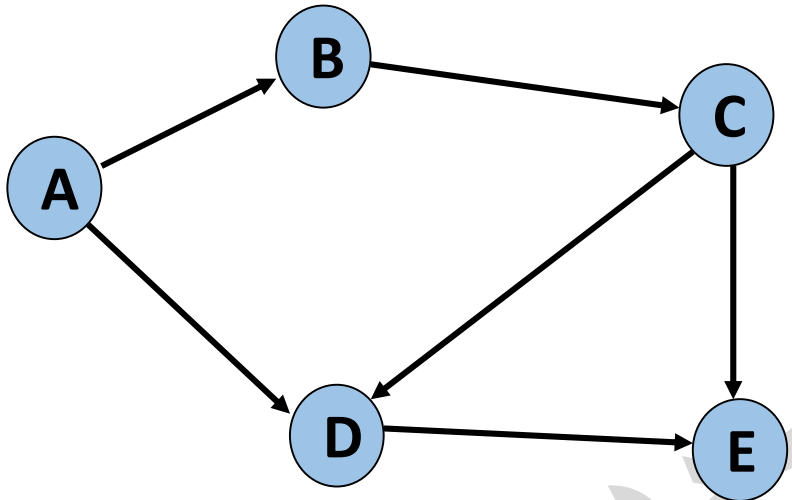


Circuit : B, C, E, D, C, E, D, B

Cycle : B, C, E, D, B

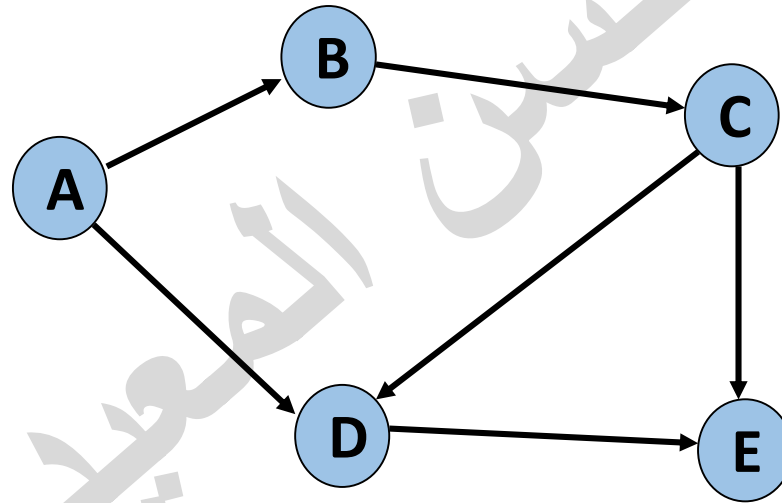
Topological Sort

- Performed on a **DAG** (**D**irected **A**cyclic **G**raph (directed graph with no cycles)).
- If the graph is not acyclic, then **no linear ordering (topological sort) is possible**.



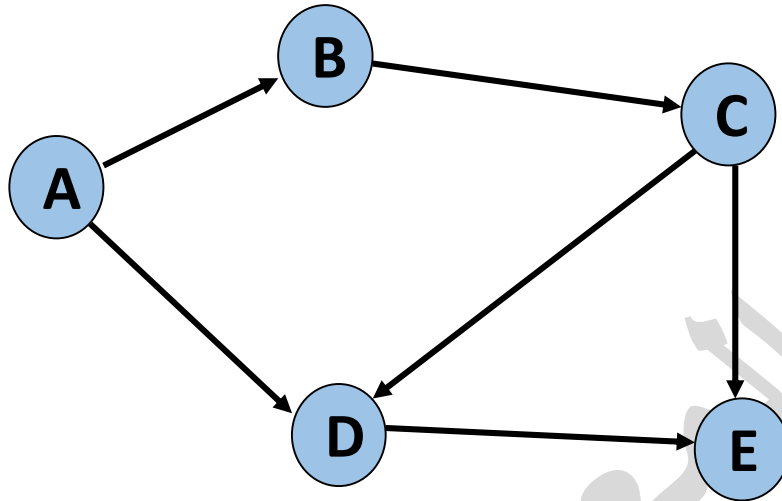
Topological Sort

- Linear ordering of the vertices of G such that if $(u, v) \in E$, then u appears somewhere before v .
- In general, there may be many legal topological orders for a given **DAG**.



Topological Sort

- Sort a directed acyclic graph (**DAG**) by the nodes' **finishing times**.



Topological Sort

Vertex	Discovery Time	Finish Time
A		
B		
C		
D		
E		

Topological Sort Algorithm

Topological-Sort(G)

{

1. call **DFS(G)** to compute **finishing time** $f[v]$ for each vertex v .
2. as a vertex is finished, **insert** it onto the **front** of a linked list.
3. **return** the linked list of vertices.

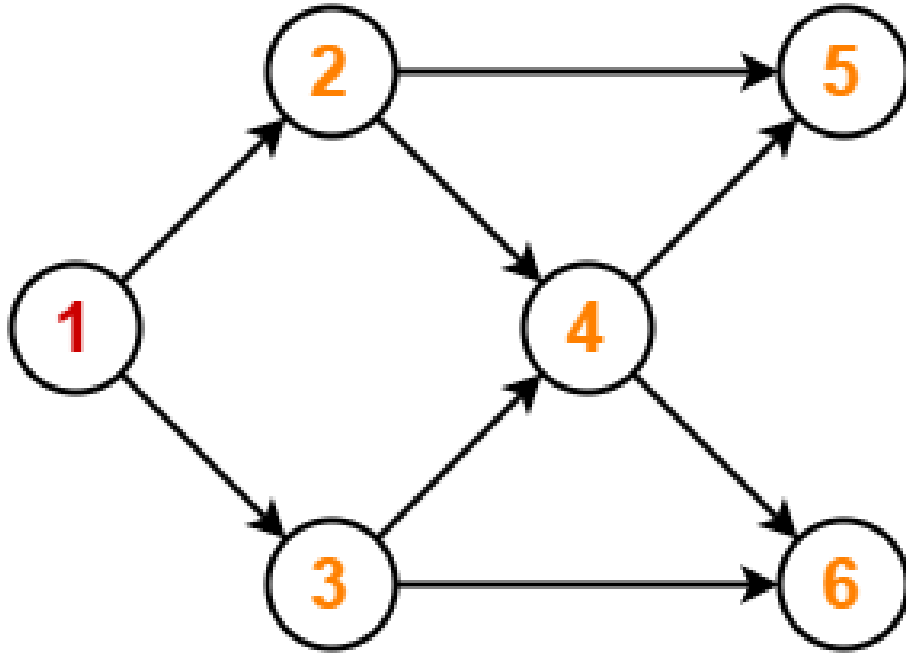
}

Topological Sort Algorithm Analysis

- Time: $O(V+E)$
 - DFS algorithm is called once with a cost $O(V+E)$ plus
 - $O(1)$ time to insert each of $|V|$ vertices onto the front of the linked list

Example 2

- Apply the topological sort algorithm for the following graph?

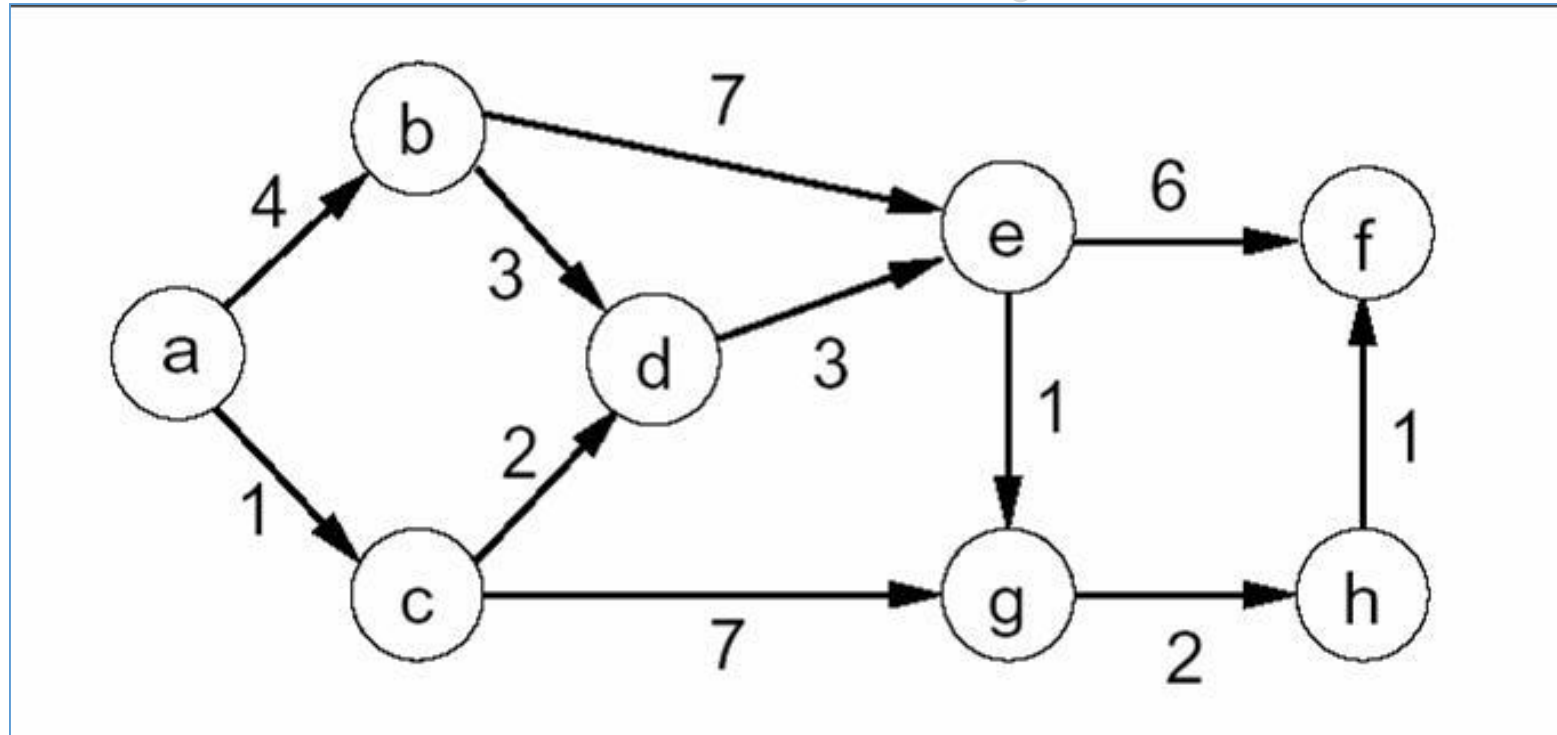


Topological Sort

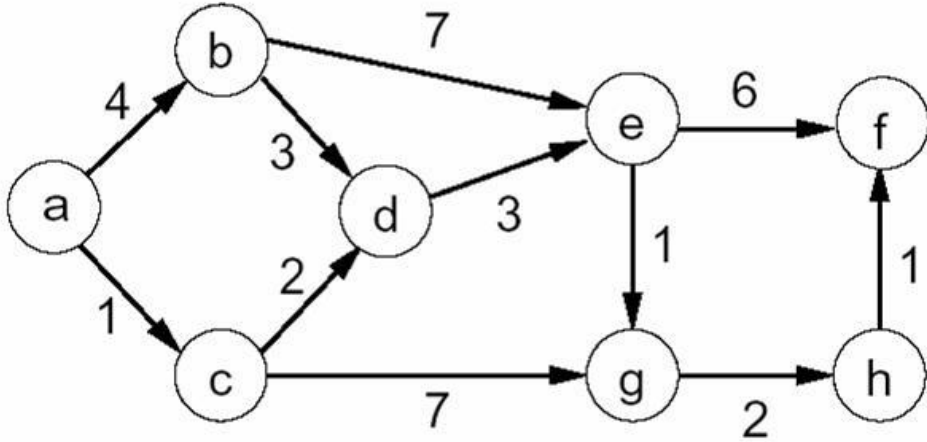
Vertex	Discovery Time	Finish Time
1		
2		
3		
4		
5		
6		

Example 3

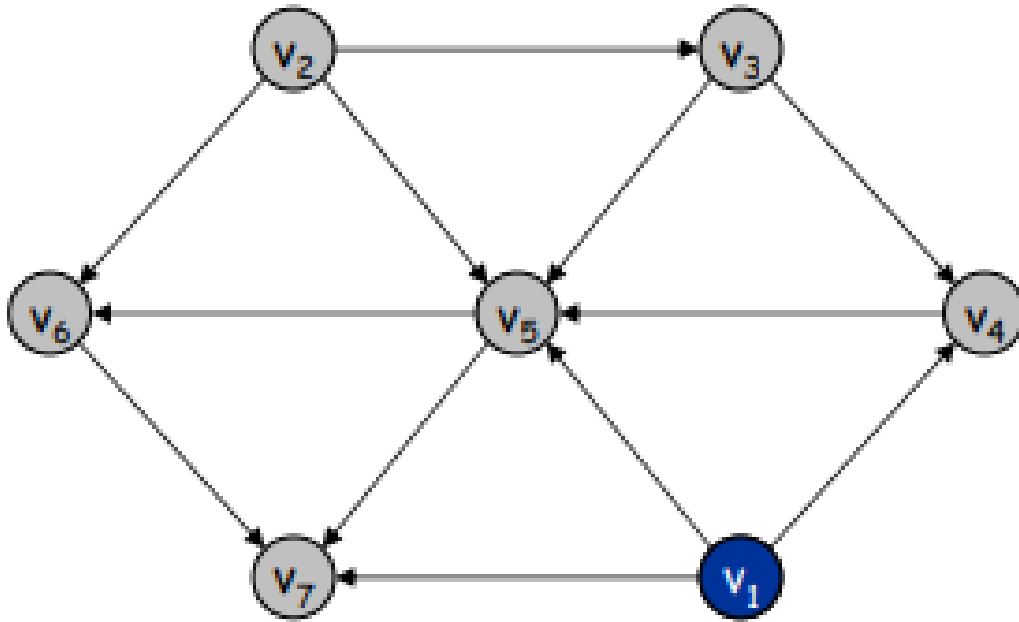
Example: Apply the topological sort algorithm for the following graph.



Example 3



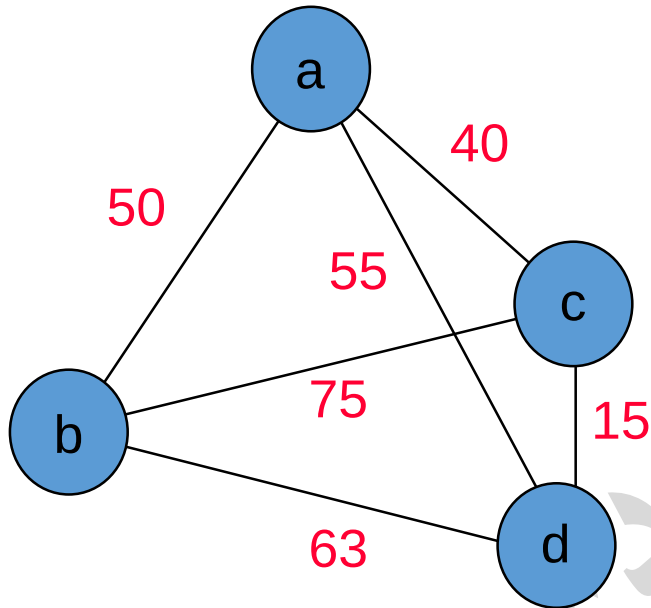
Example 4



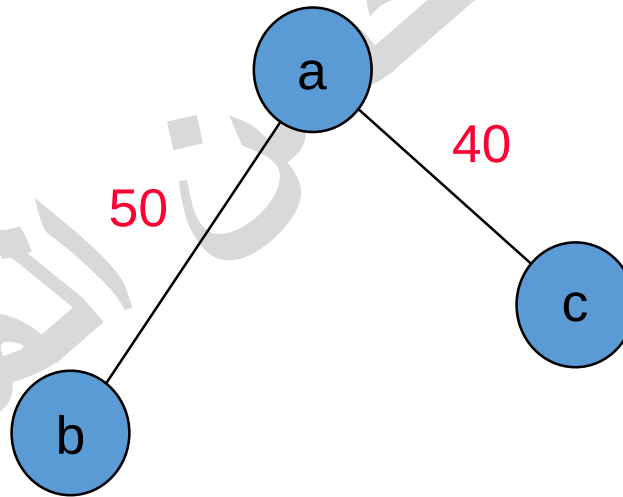
Minimum Spanning Tree

A Tree

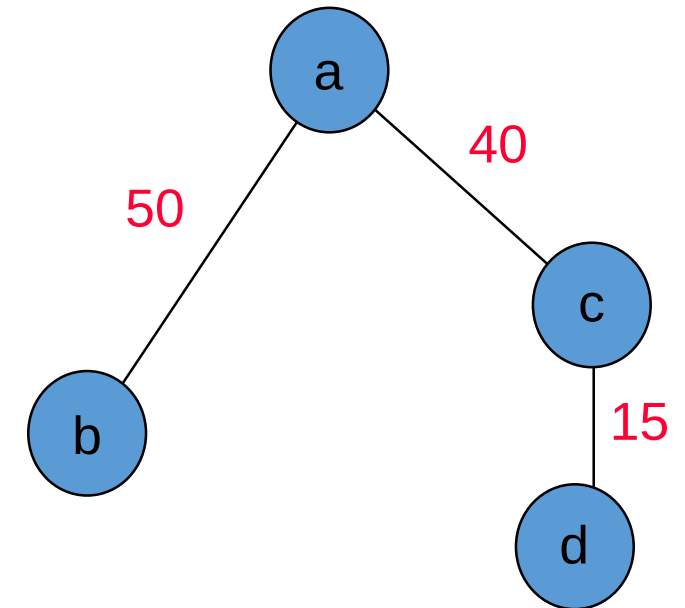
A **tree** is a connected undirected graph that contains no cycles



Graph



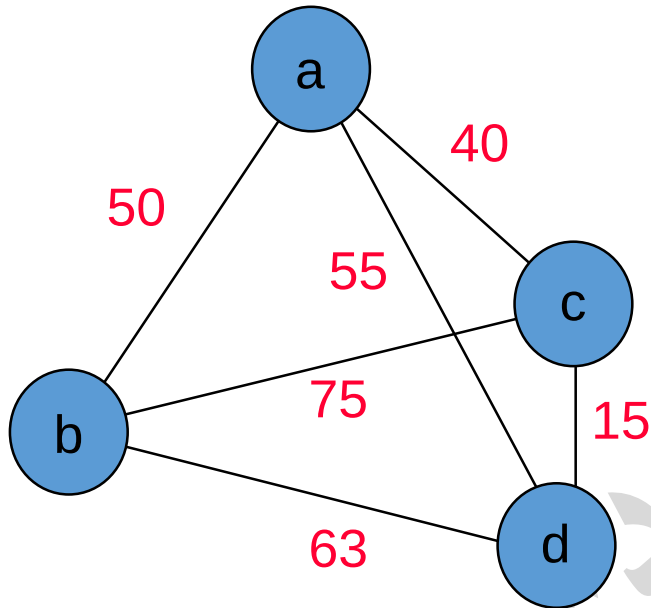
Tree T1



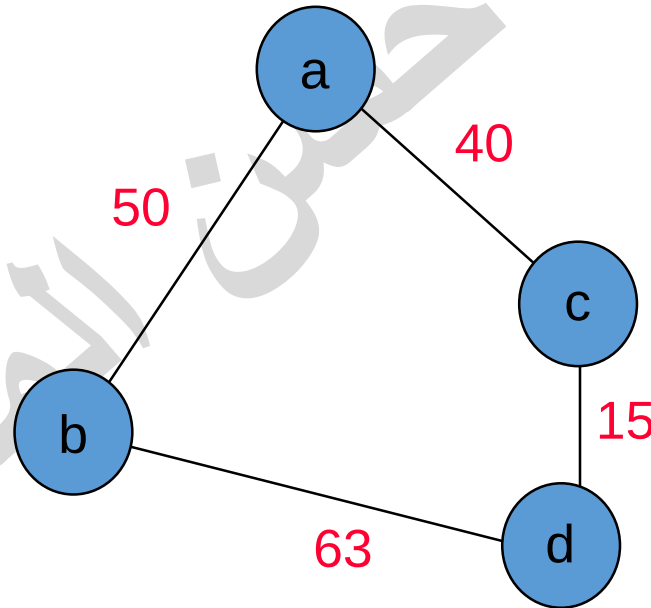
Tree T2

A Tree

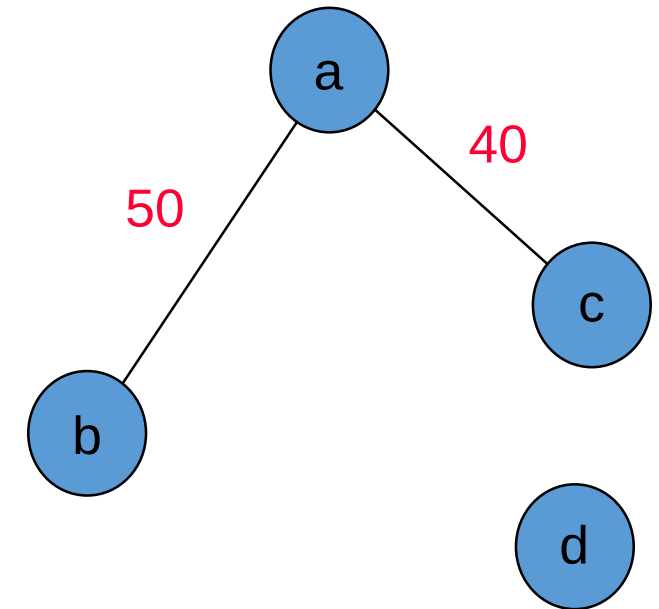
A **tree** is a connected undirected graph that contains no cycles



Graph



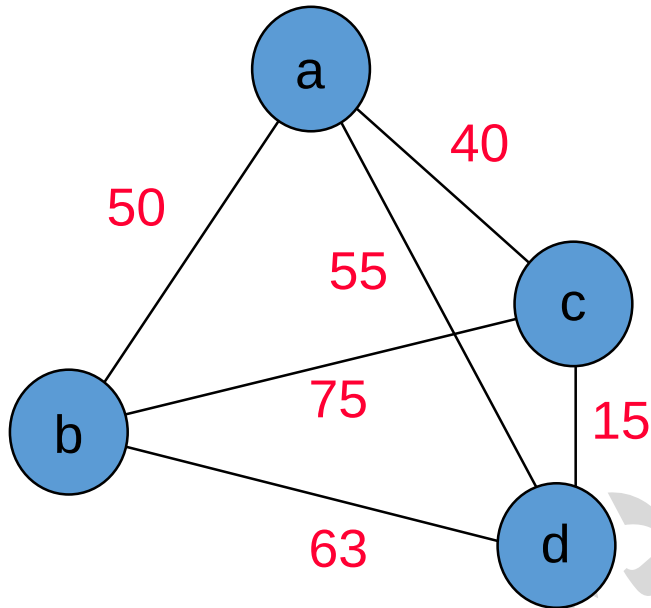
Not Tree



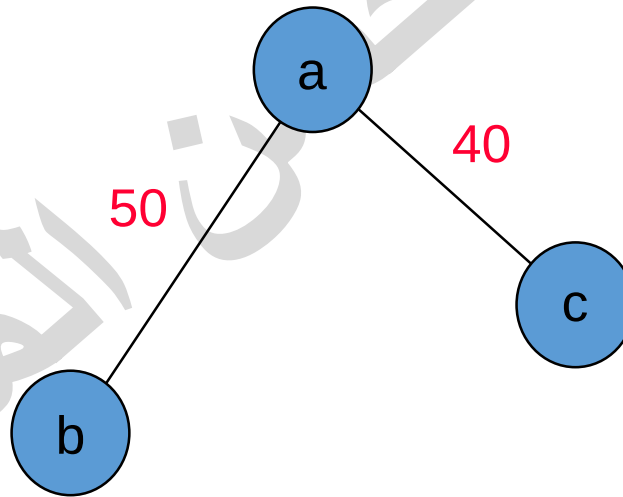
Not Tree

A Spanning Tree

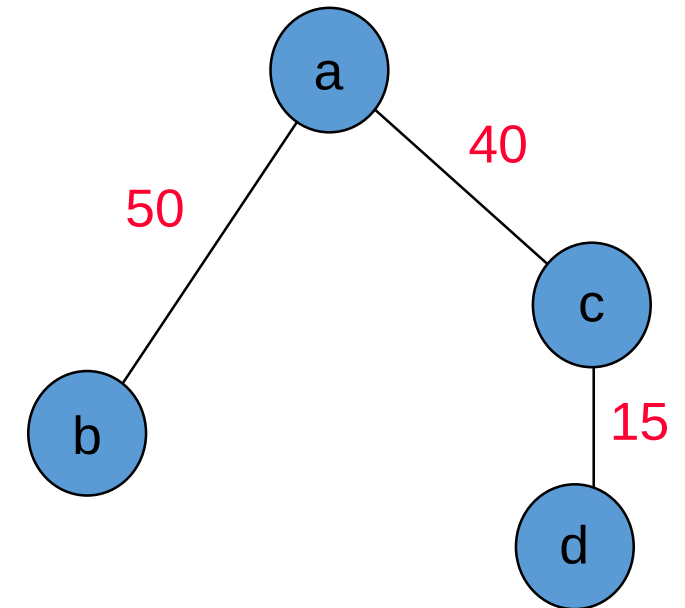
A **spanning tree** of a graph G is a subgraph of G that is a tree and contains all the vertices of G



Graph



Tree but Not ST



Spanning Tree

Properties of a Spanning Tree

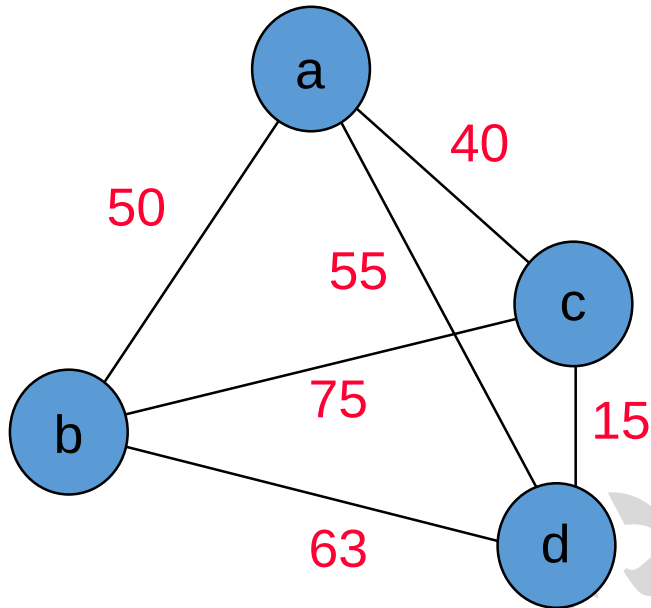
- The spanning tree of a **n -vertex** undirected graph has exactly **$n - 1$** edges
- It connects all the vertices in the graph
- It has ***no cycles***

Example: Assume an undirected graph contains **50** vertices and **1225** edges. We want to construct a MST from it.

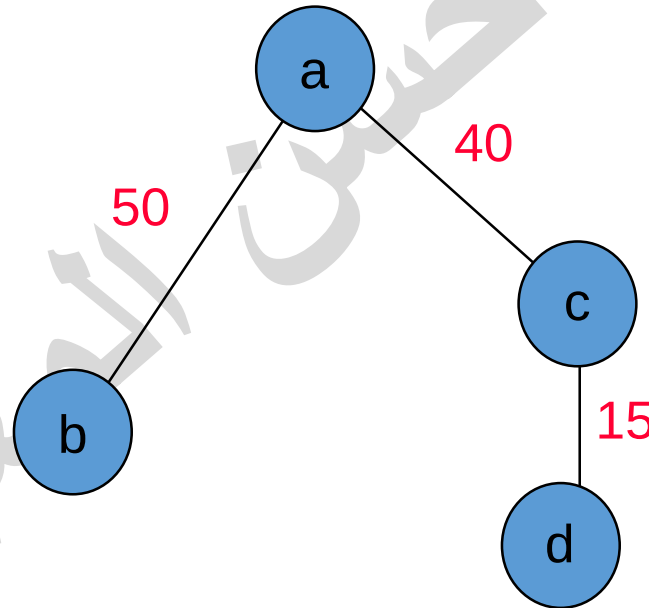
1. How many vertices are there in a MST?
2. How many edges are there in a MST?

A Minimum Spanning Tree (MST)

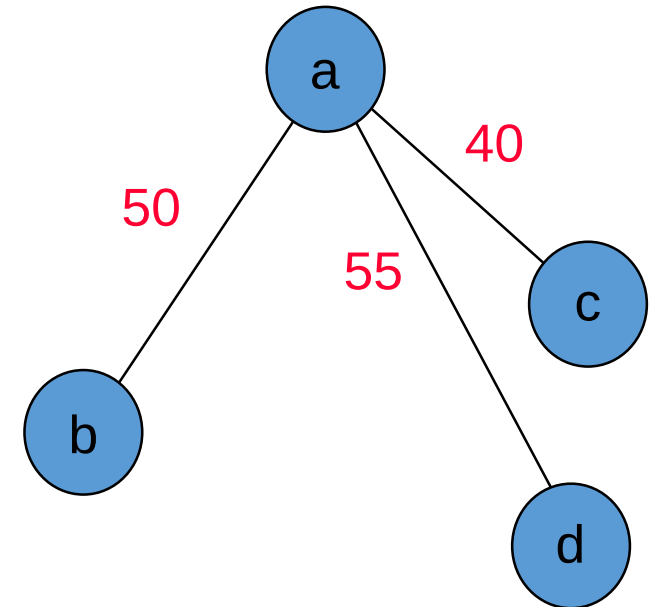
- A **minimum spanning tree (MST)** is the one among all the spanning trees **with the lowest cost**



Graph



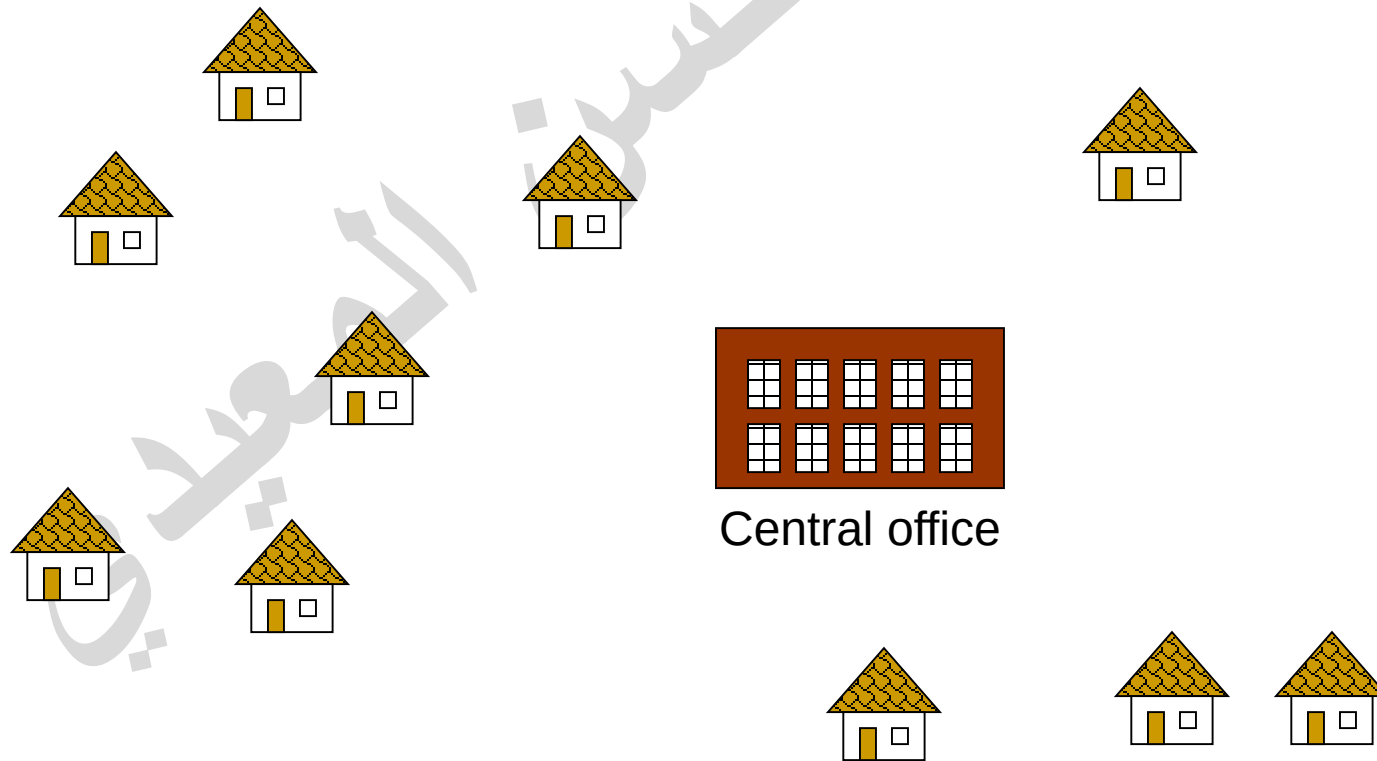
**Spanning Tree
ST1**



**Spanning Tree
ST2**

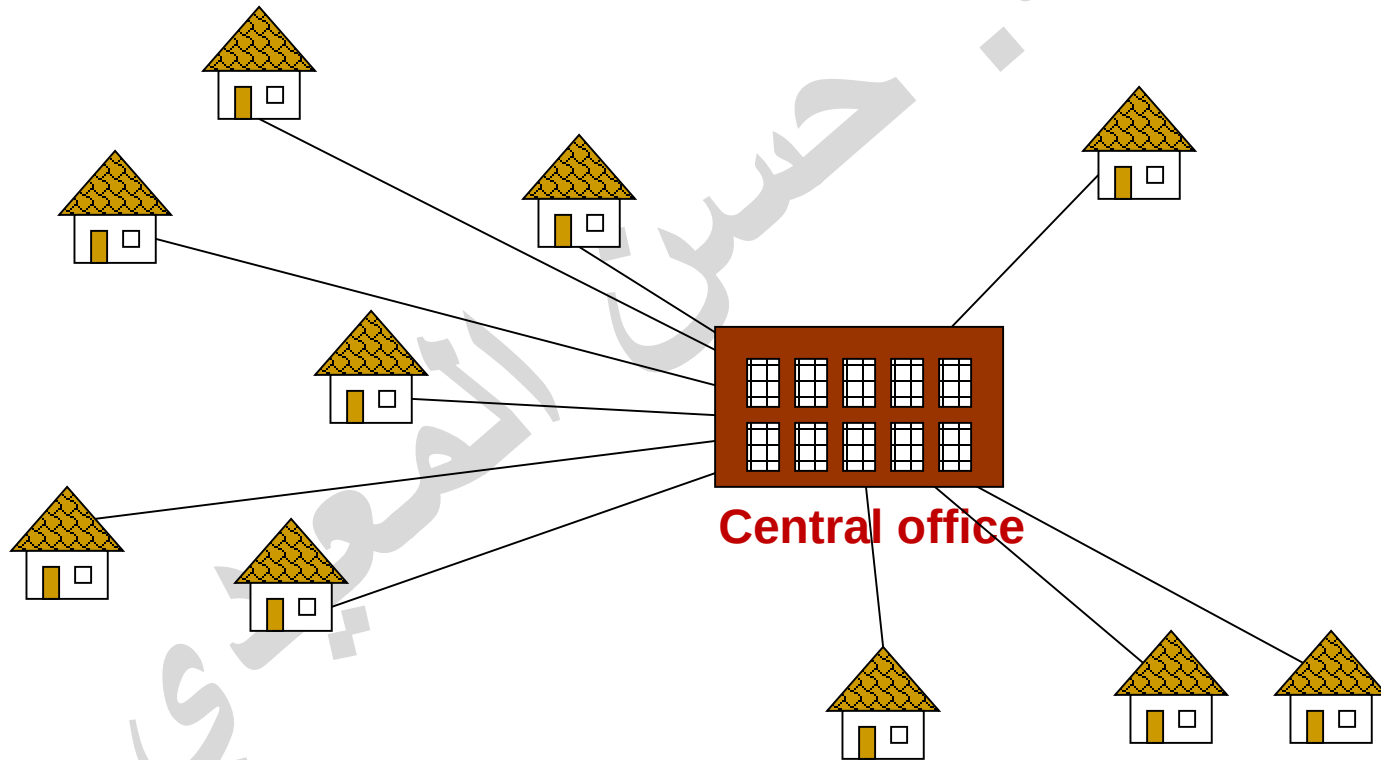
Some Applications of MST

Laying Telephone Wire



Some Applications of MST

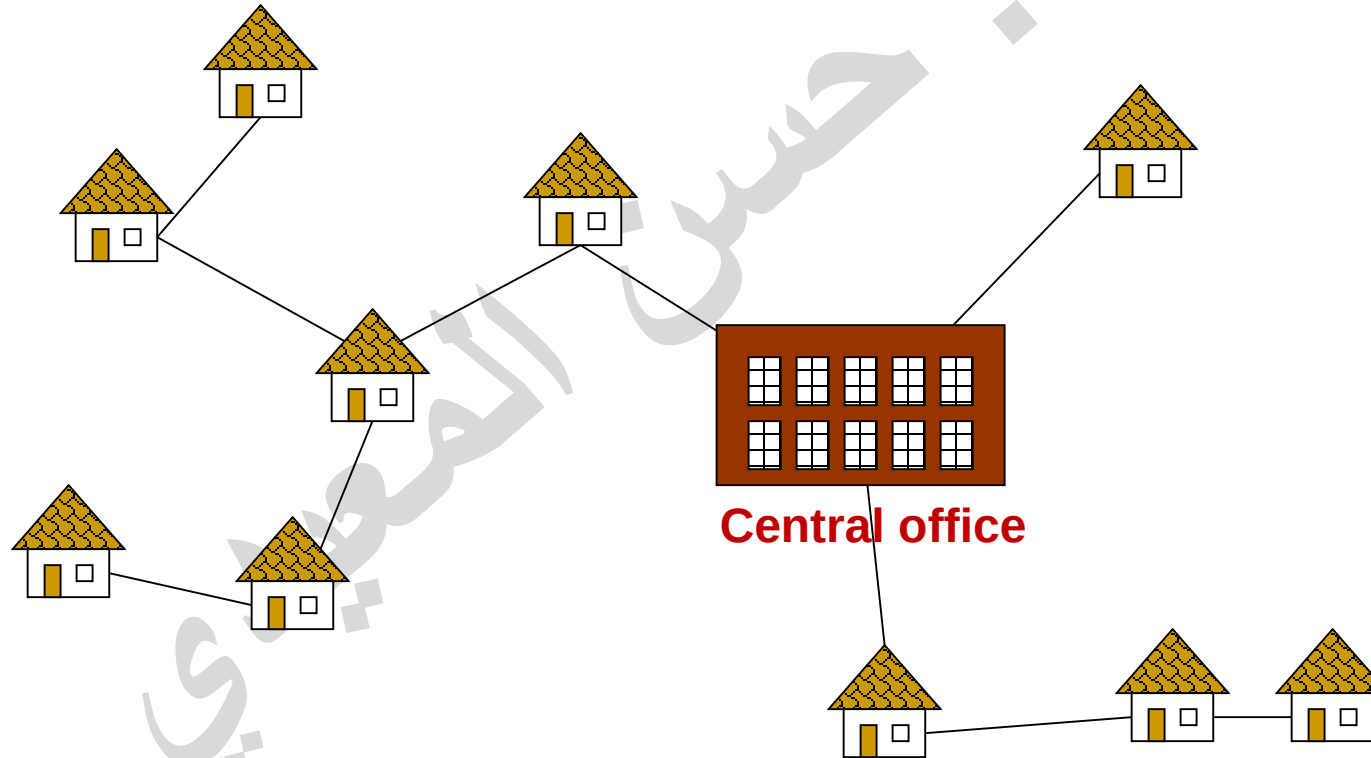
Laying Telephone Wire



Expensive!

Some Applications of MST

Laying Telephone Wire



Minimize the total length of wire connecting the customers

Minimum Spanning Tree Algorithms

MST Algorithms

Kruskal's Algorithm

Prim's Algorithm

Kruskal's Algorithm

Kruskal's Algorithm

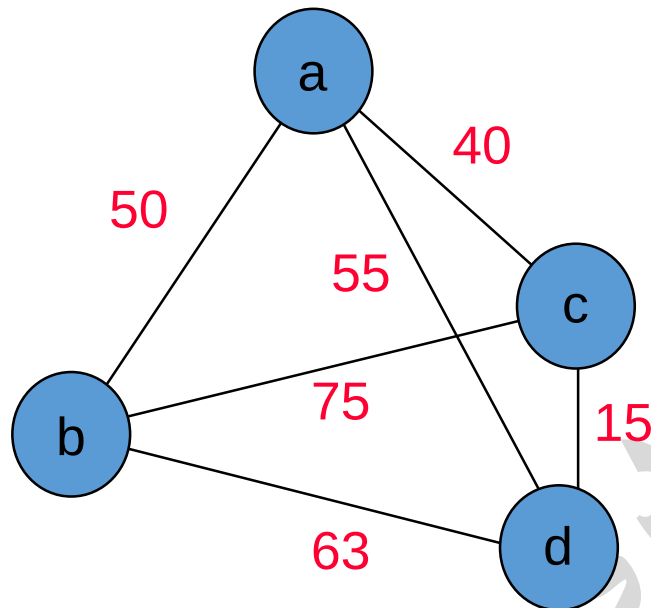
- **Idea**

1. Add an edge of minimum weight that does not create a cycle. (greedy algorithm).
2. Repeat step 1 ($|V| - 1$ times).

- ⑥ Start with an empty graph (no edges)
- ⑥ Sort the edges by cost
- ⑥ For each edge e (in increasing order of cost)
 - △ Add e to G if it would not cause a cycle

Kruskal's Algorithm

Example: Apply Kruskal's algorithm for the following graph to find the minimum spanning tree.



Graph

1- Sort edges by their weights

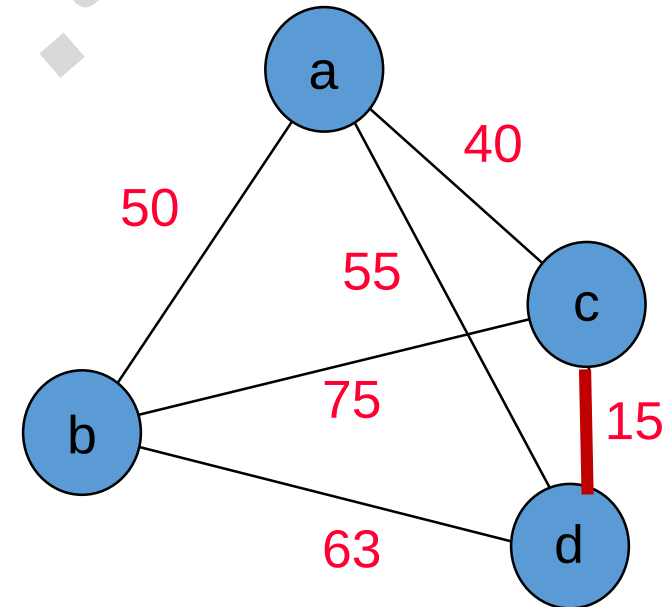
Weight	Edges

Kruskal's Algorithm

2- Mark the first edge (e) in the graph who has the minimum weight.

Weight	Edges
15	cd
40	ac
50	ab
55	ad
63	bd
75	bc

$$|E| = 1$$

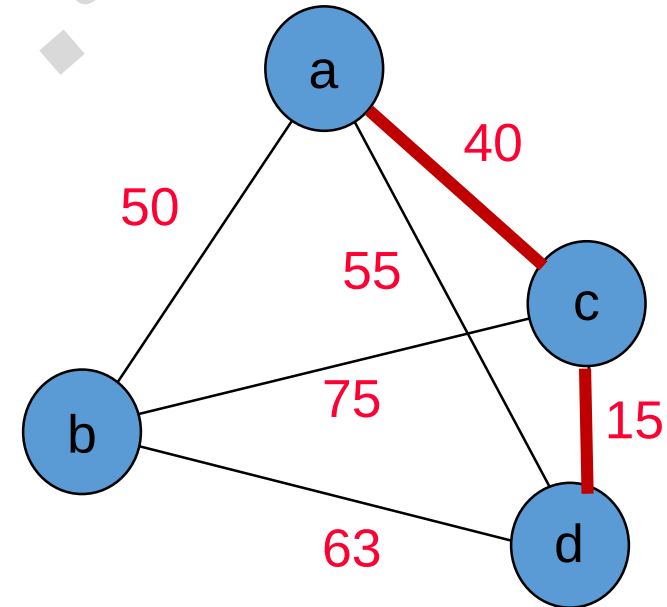


Kruskal's Algorithm

3- Mark the second edge (e) in the graph who has the next minimum weight.

Weight	Edges
15	cd
40	ac
50	ab
55	ad
63	bd
75	bc

$$|E| = 2$$

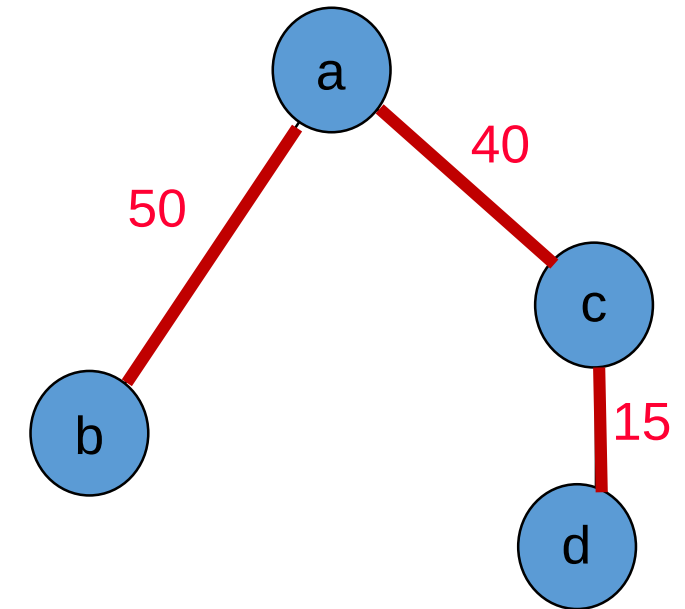
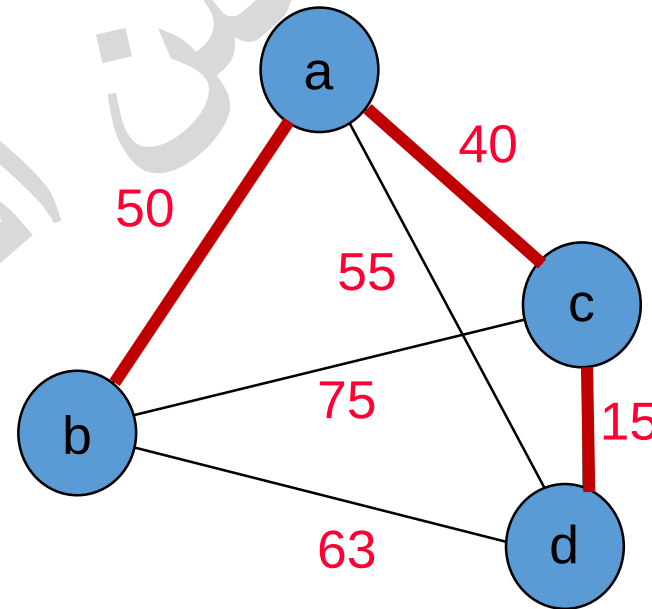


Kruskal's Algorithm

4- Mark the next edge (e) in the graph who has the next minimum weight, if it would not cause a cycle.

Weight	Edges
15	cd
40	ac
50	ab
55	ad
63	bd
75	bc

$|E| = 3$



Weight=105

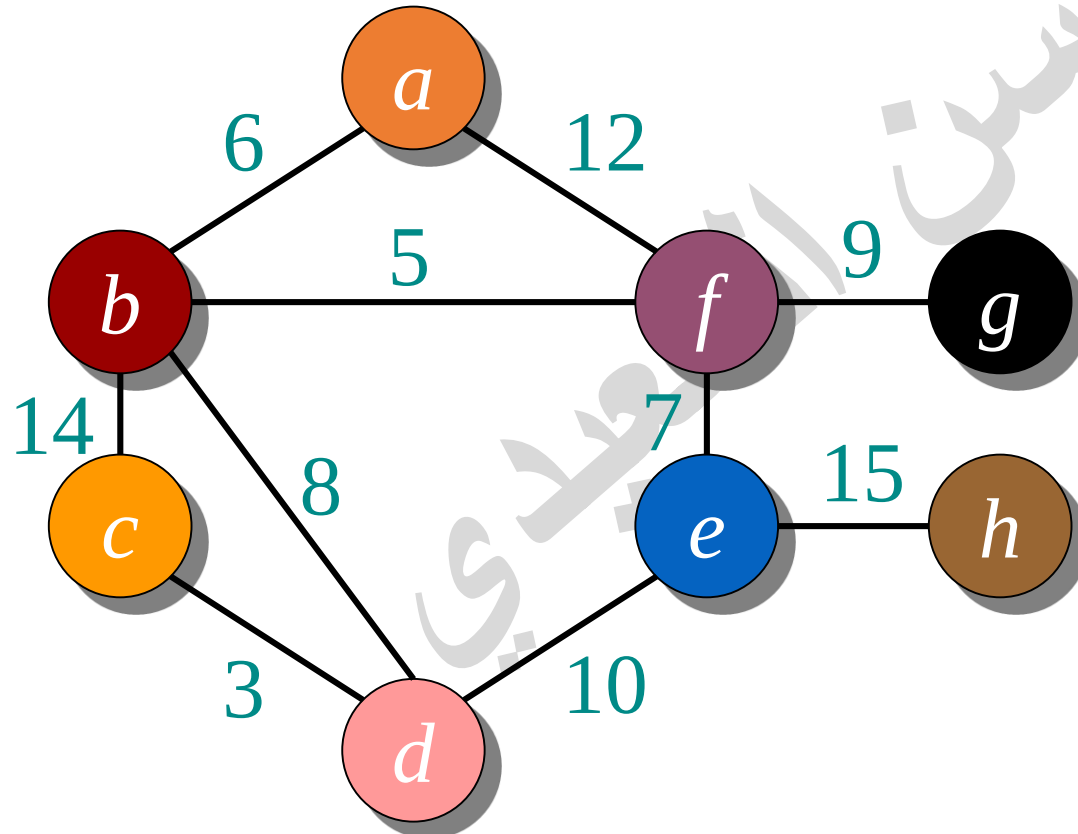
Time Complexity of Kruskal's Algorithm

Depends on implementation:

- Adjacency Matrix : $O(n^2) \rightarrow O(|V|^2)$
- Adjacency List : $O(m \text{ Log } n) \rightarrow O(|E| \text{ Log } |V|)$

Kruskal's Algorithm – Example 2

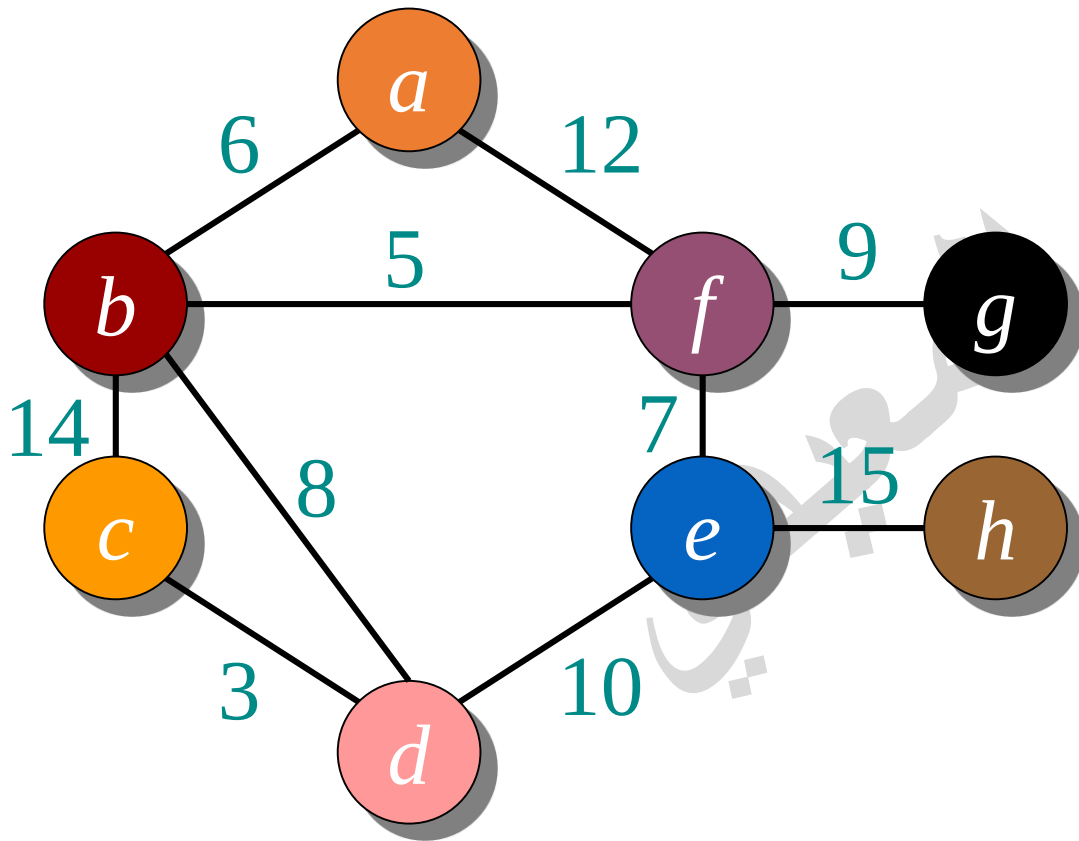
Example: Apply Kruskal's algorithm for the following graph to find the minimum spanning tree.



cd:	3
bf:	5
ba:	6
ef:	7
bd:	8
fg:	9
de:	10
af:	12
bc:	14
h2:	15

Kruskal's Algorithm – Example 2

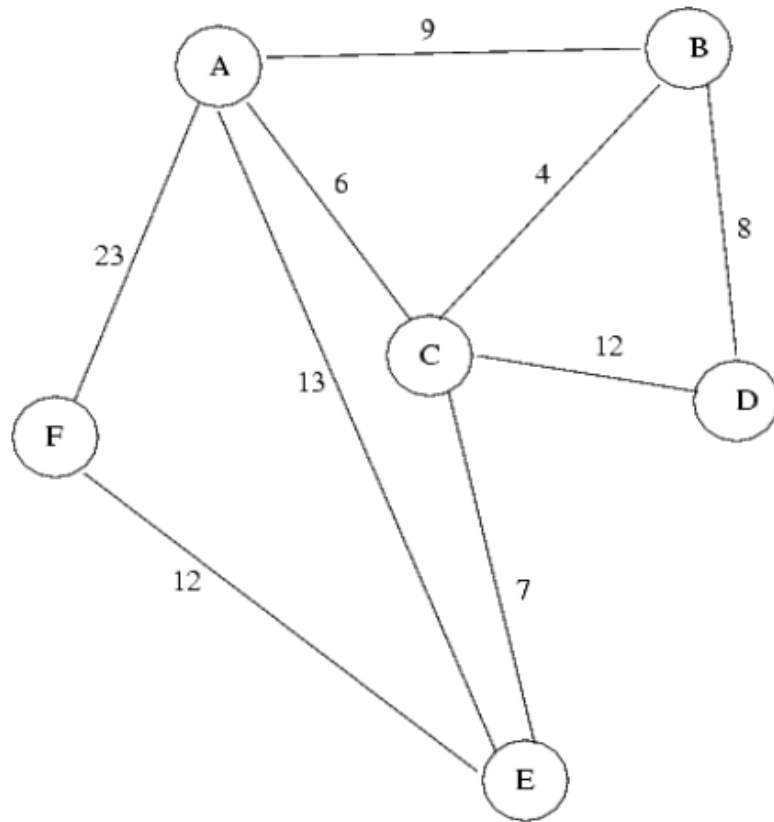
Example: Apply Kruskal's algorithm for the following graph to find the minimum spanning tree.



Weight=

Kruskal's Algorithm – Example 3

Example: Apply Kruskal's algorithm for the following graph to find the minimum spanning tree.



Kruskal's Algorithm – Example 3

Example: From the previous example answer the following questions:

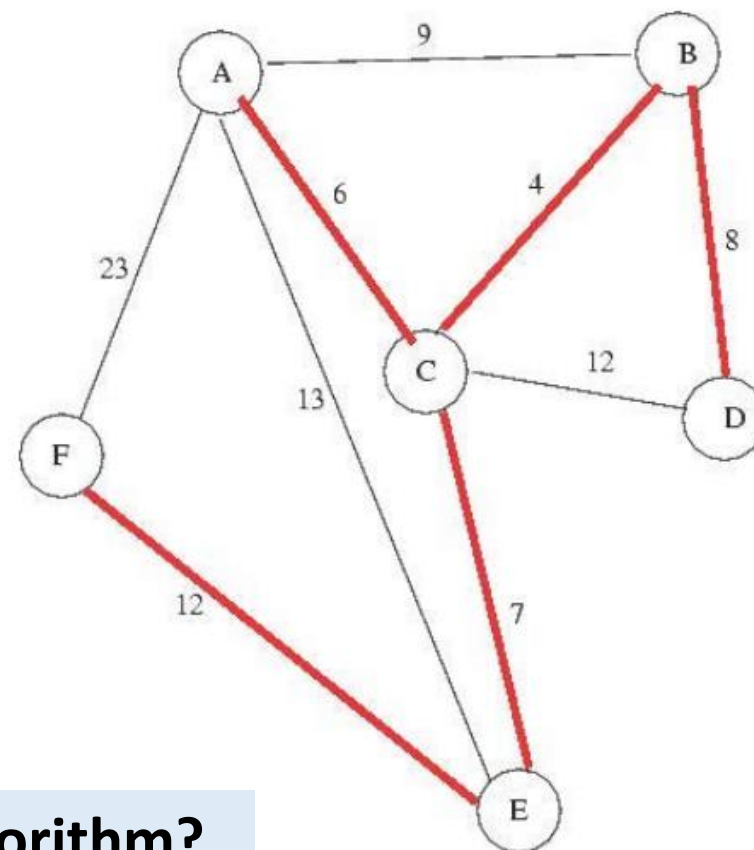
1- Which edge will be selected first?

2- Which edge will be selected second?

3- How many edges are there in the MST?

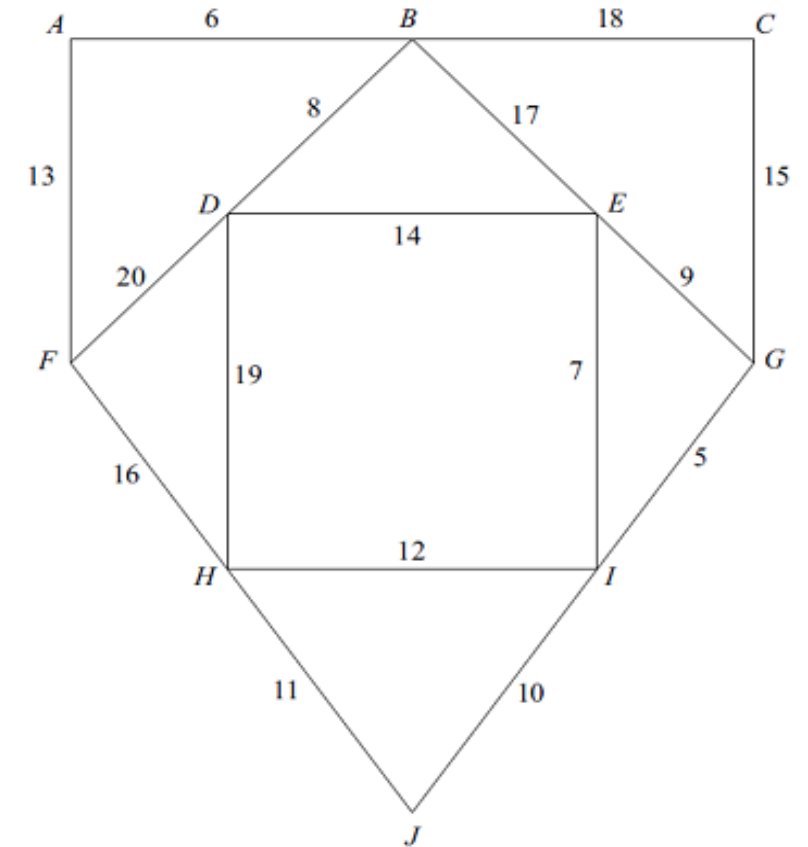
4- State an edge which is not selected in MST.

5- What is the weight of the tree resulting from Kruskal's algorithm?



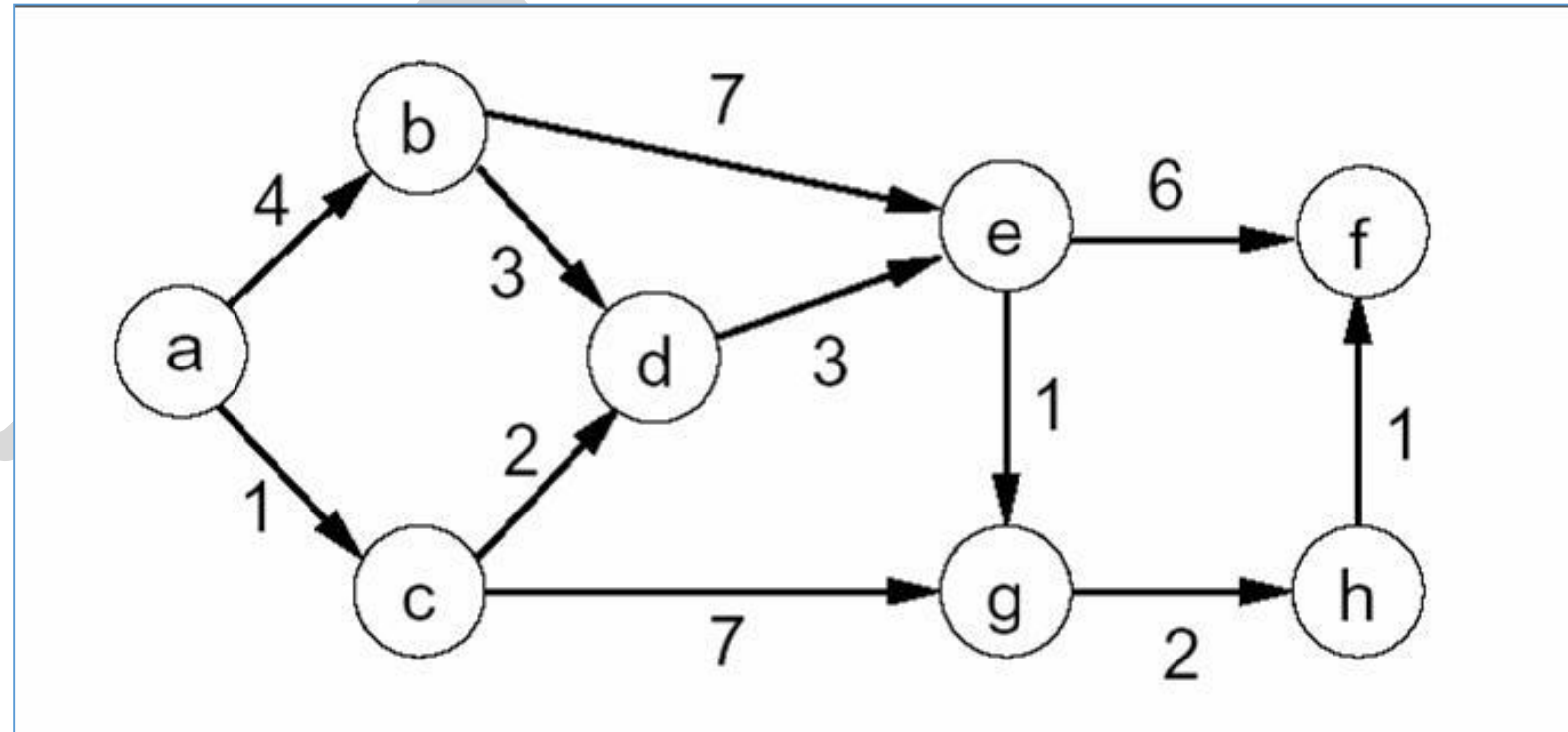
Kruskal's Algorithm Example 4

Example: Apply Kruskal's algorithm for the following graph to find the minimum spanning tree.



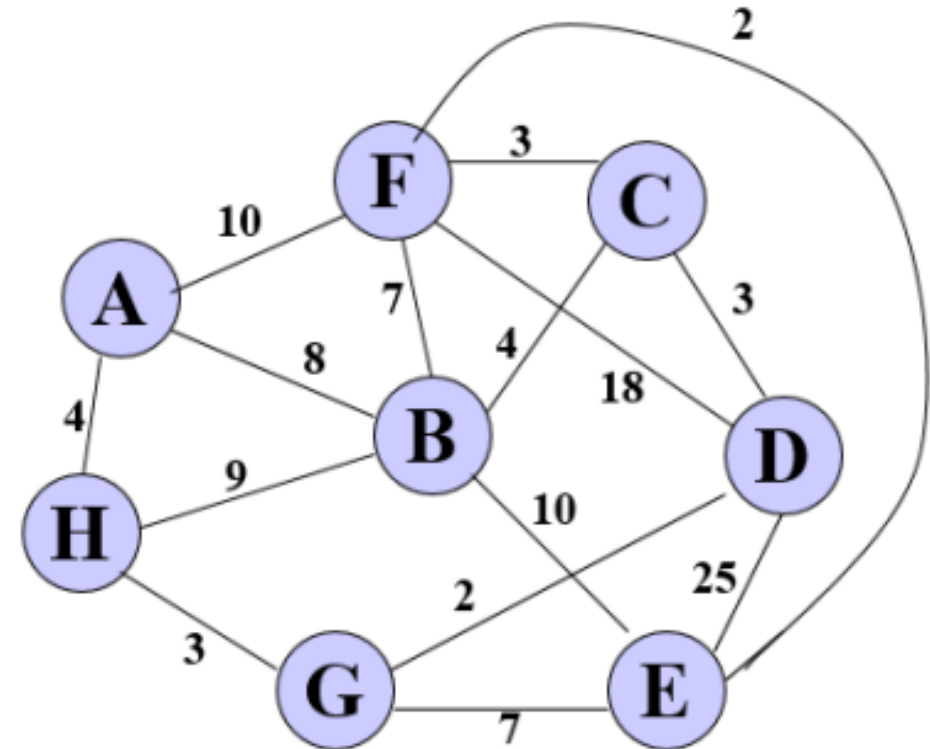
Kruskal's Algorithm Example 5

Example: Apply Kruskal's algorithm for the following graph to find the minimum spanning tree.



Kruskal's Algorithm Example 6

Example: Apply Kruskal's algorithm for the following graph to find the minimum spanning tree.



Prim's Algorithm

Minimum Spanning Tree Algorithms

MST Algorithms

Kruskal's Algorithm

Prim's Algorithm

Prim's Algorithm

- **Idea**
 - Start from an arbitrary single node.
 - Gradually build up a single larger and larger MST.

$T = v_0$

Until T spans all nodes **do**

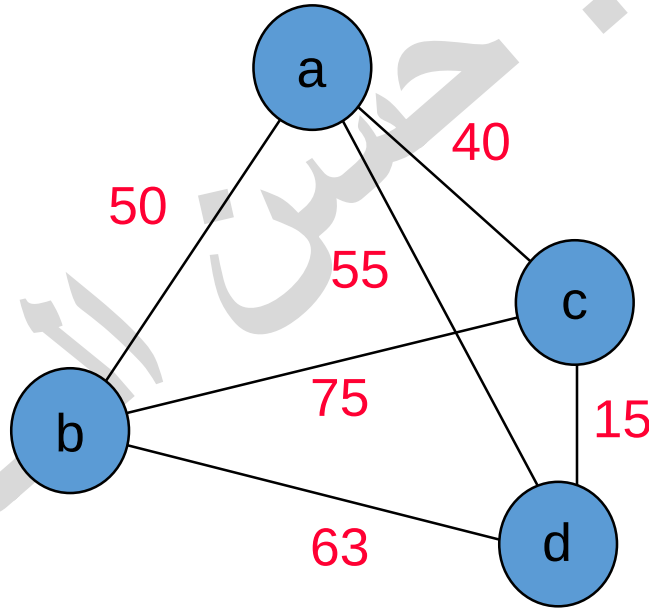
Select nodes $x \in T, y \notin T$
w/min cost(x, y)

Add edge (x, y) to T

Return T

Prim's Algorithm – Example 1

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.



Graph

Prim's Algorithm – Example 1

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.

Iteration [1]			
a	b	c	d
w			
p			

Iteration [2]			
c	b	c	d
w			
p			

Iteration [3]			
d	b	c	d
w			
p			

Complexity of Prim's Algorithm

Depends on implementation:

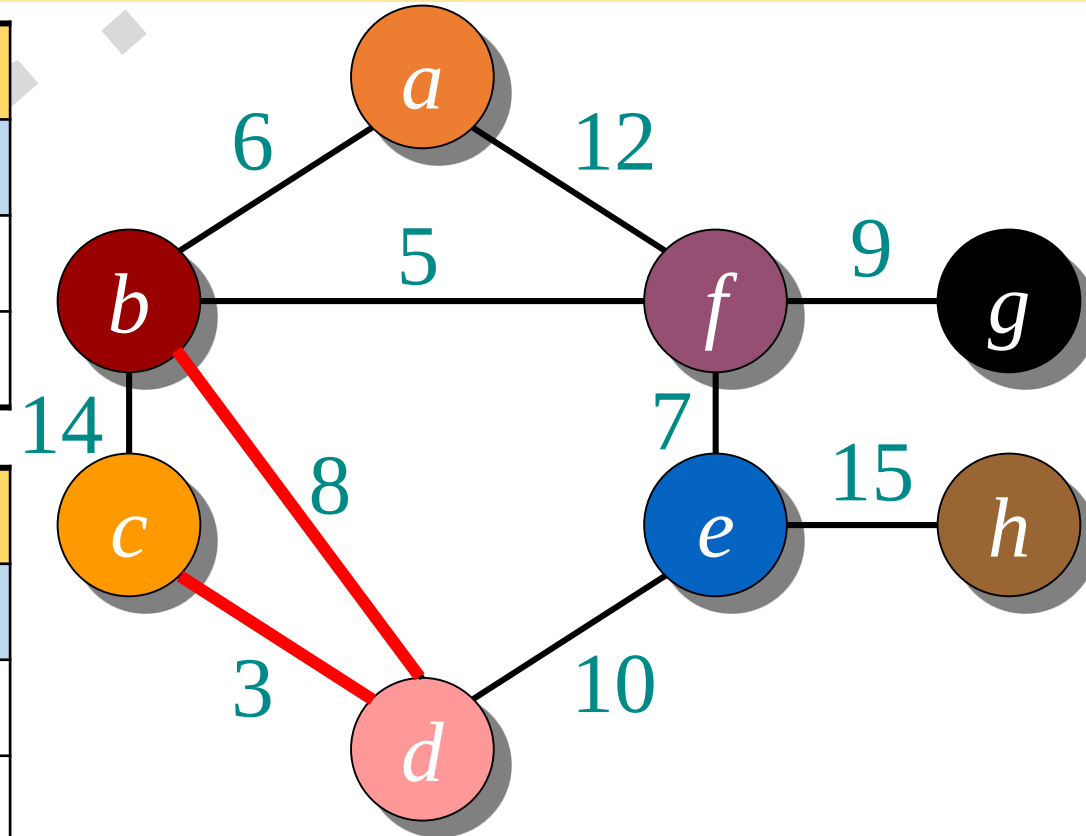
- Adjacency Matrix : $O(n^2) \rightarrow O(|V|^2)$
- Adjacency List : $O(m \text{ Log } n) \rightarrow O(|E| \text{ Log } |V|)$

Prim's Algorithm – Example 2

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree; assume **d** is first parent.

Iteration [1]							
d	a	b	c	e	f	g	h
w							
p							

Iteration [2]							
	a	b		e	f	g	h
w							
p							

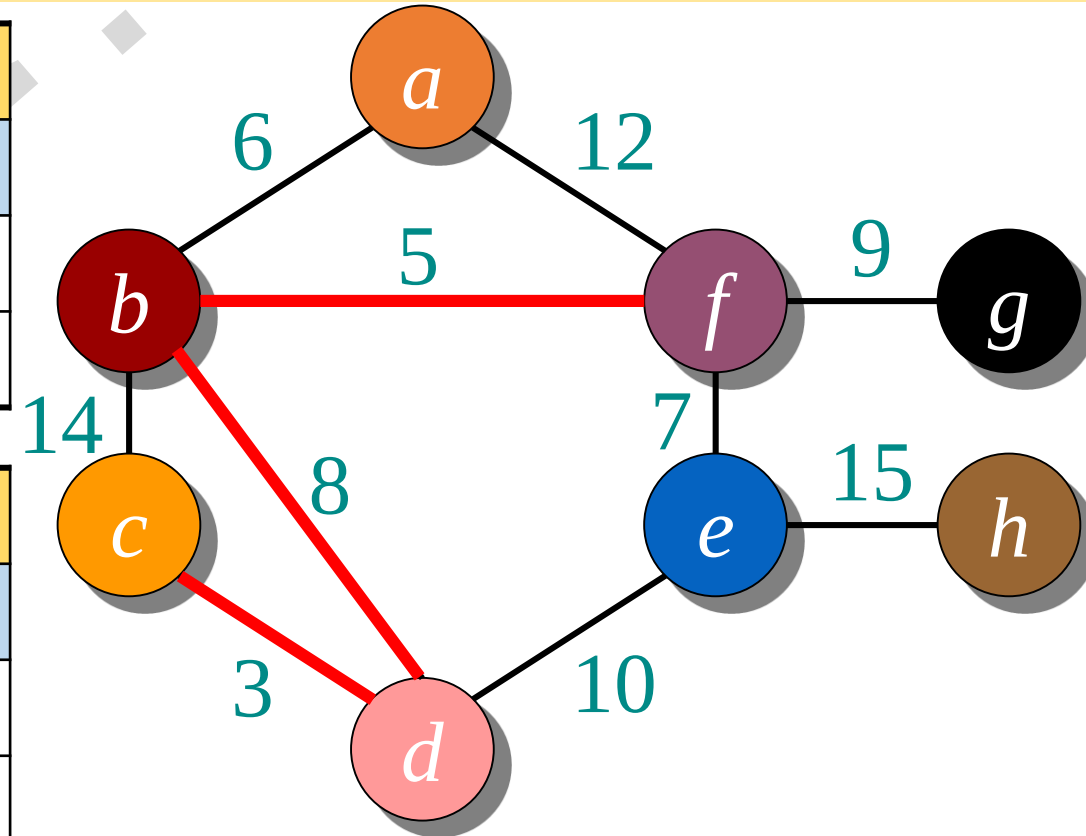


Prim's Algorithm – Example 2

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.

Iteration [2]							
c	a	b		e	f	g	h
w							
p							

Iteration [3]							
	a			e	f	g	h
w							
p							

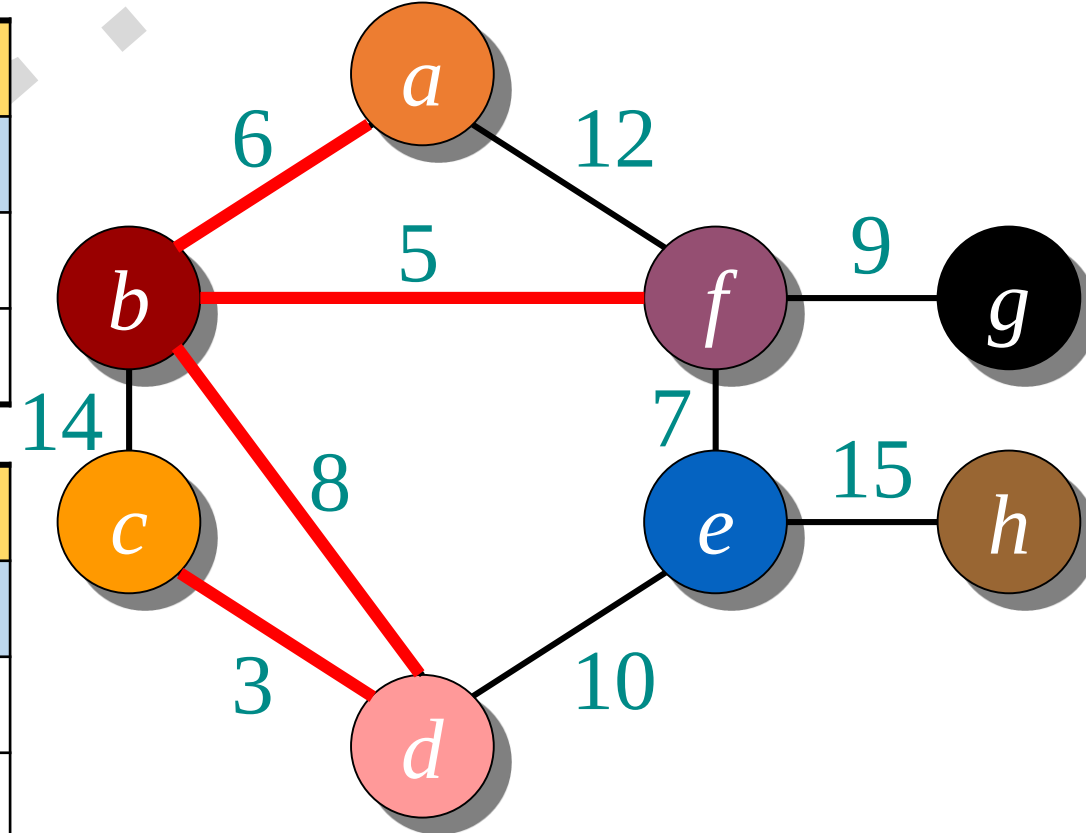


Prim's Algorithm – Example 2

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.

Iteration [3]							
b	a			e	f	g	h
w							
p							

Iteration [4]							
	a			e		g	h
w							
p							

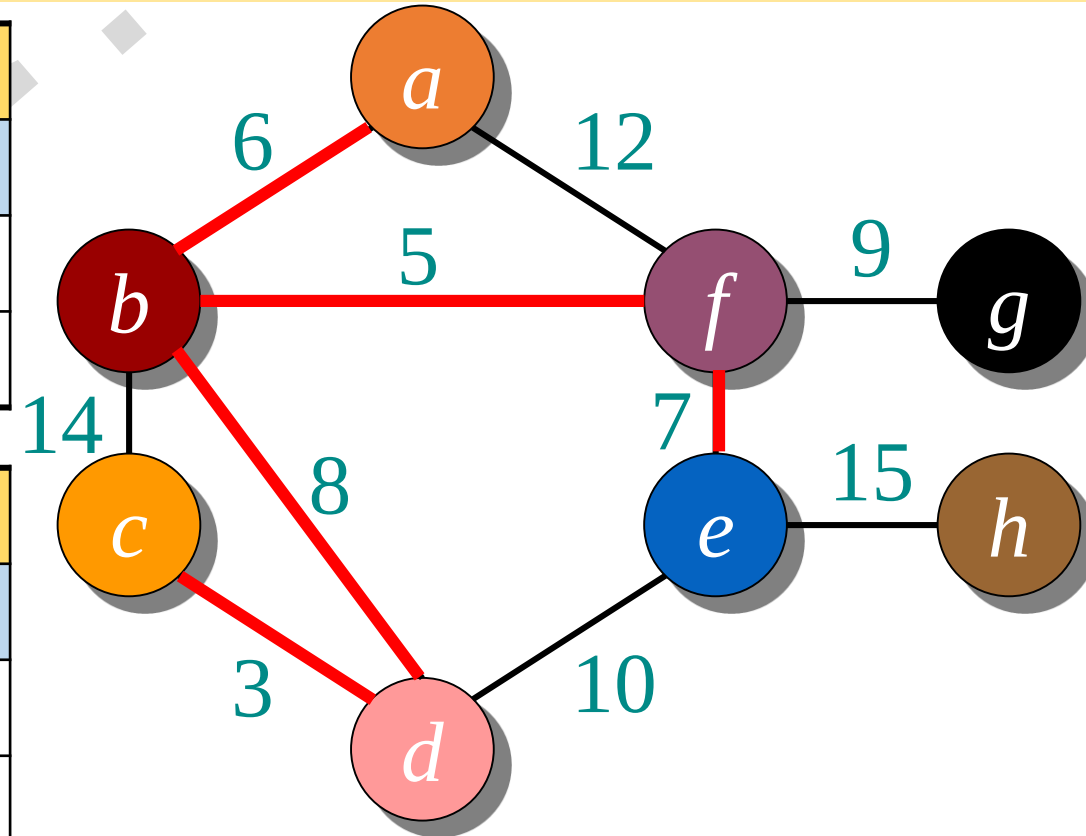


Prim's Algorithm – Example 2

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.

Iteration [4]							
f	a			e		g	h
w							
p							

Iteration [5]							
				e		g	h
w							
p							

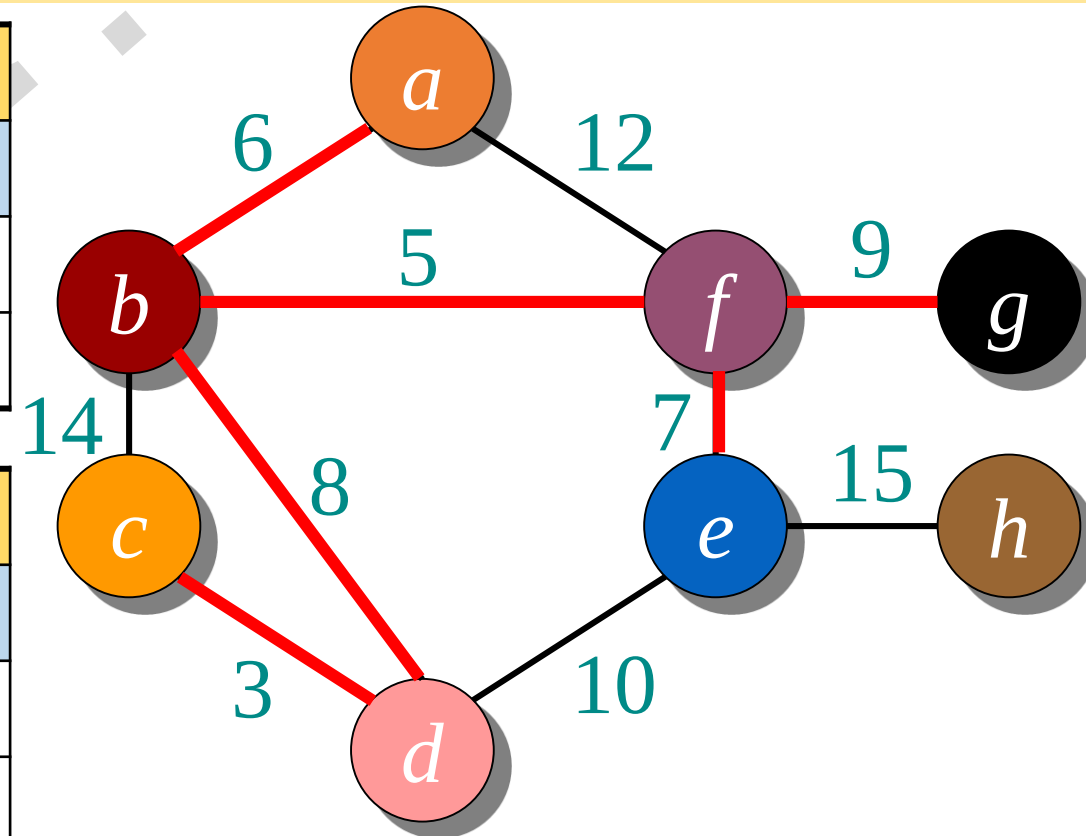


Prim's Algorithm – Example 2

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.

Iteration [5]							
a				e		g	h
w							
p							

Iteration [6]							
						g	h
w							
p							

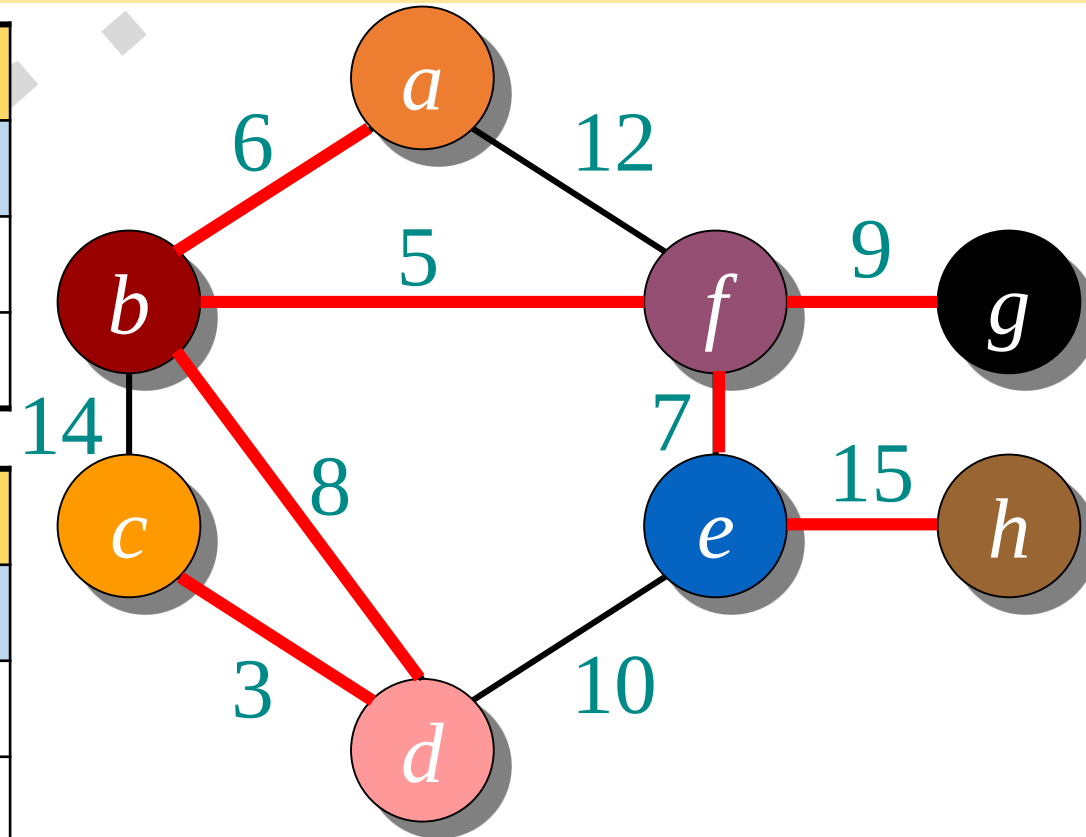


Prim's Algorithm – Example 2

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.

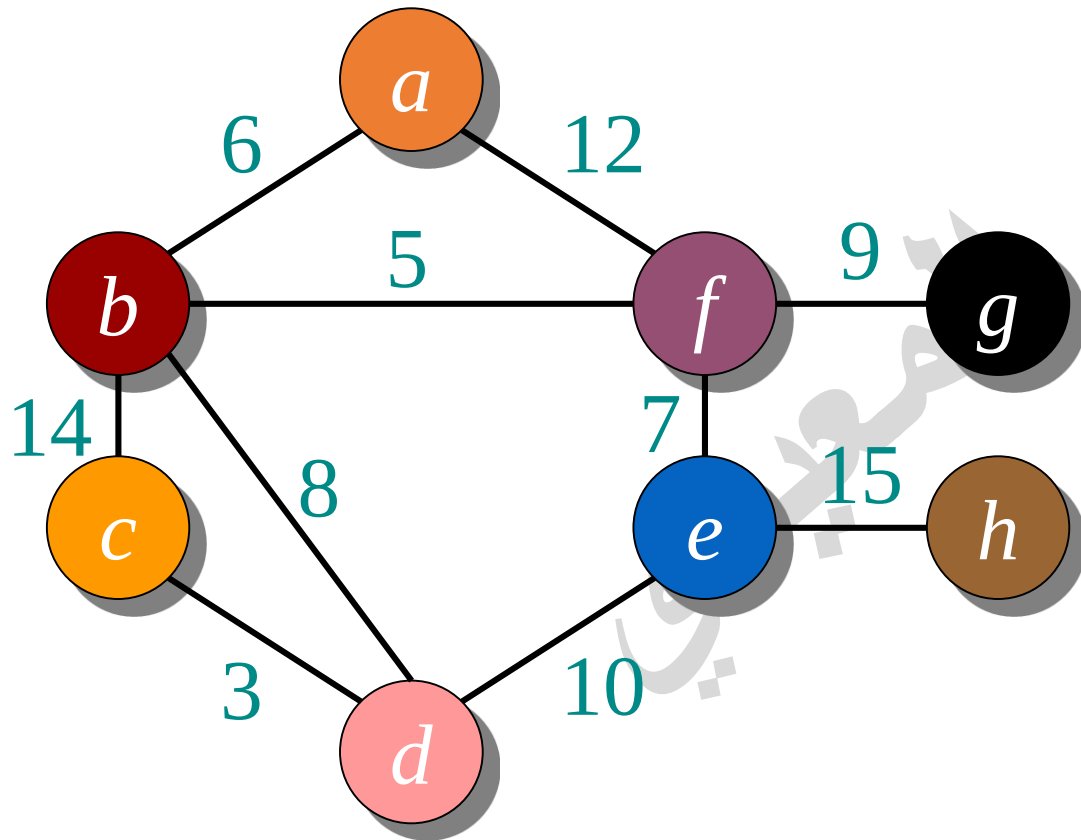
Iteration [6]							
e						g	h
w							
p							

Iteration [7]							
							h
w							
p							



Prim's Algorithm – Example 2

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.



Weight=

Prim's Algorithm – Example 3

Example: From the previous example answer the following questions:

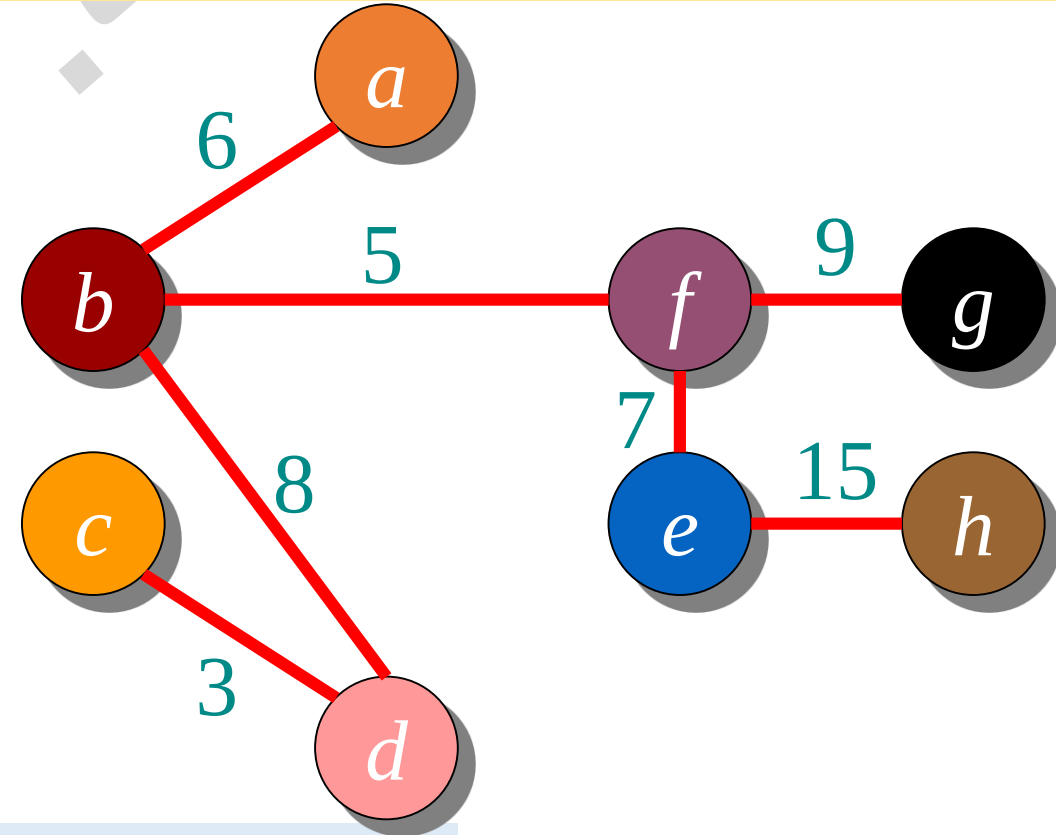
1- Which vertex will be selected at iteration 2?

2- Which vertex will be selected at iteration 4?

3- Which vertex will be selected at iteration 7?

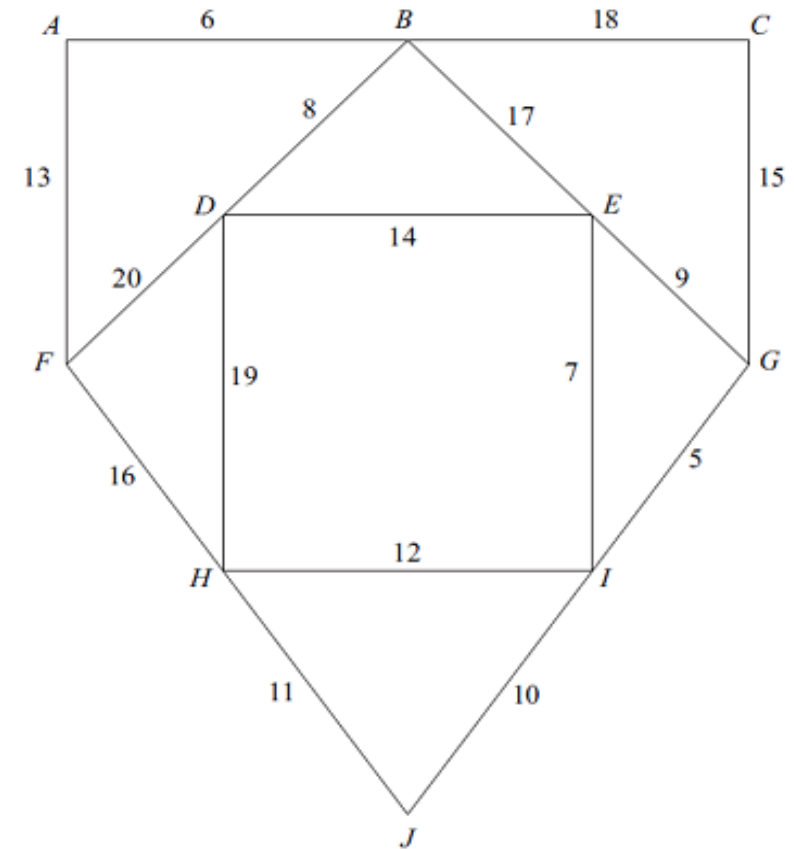
4- State an edge which is not selected in MST.

5- What is the weight of the tree resulting from Prim's algorithm?



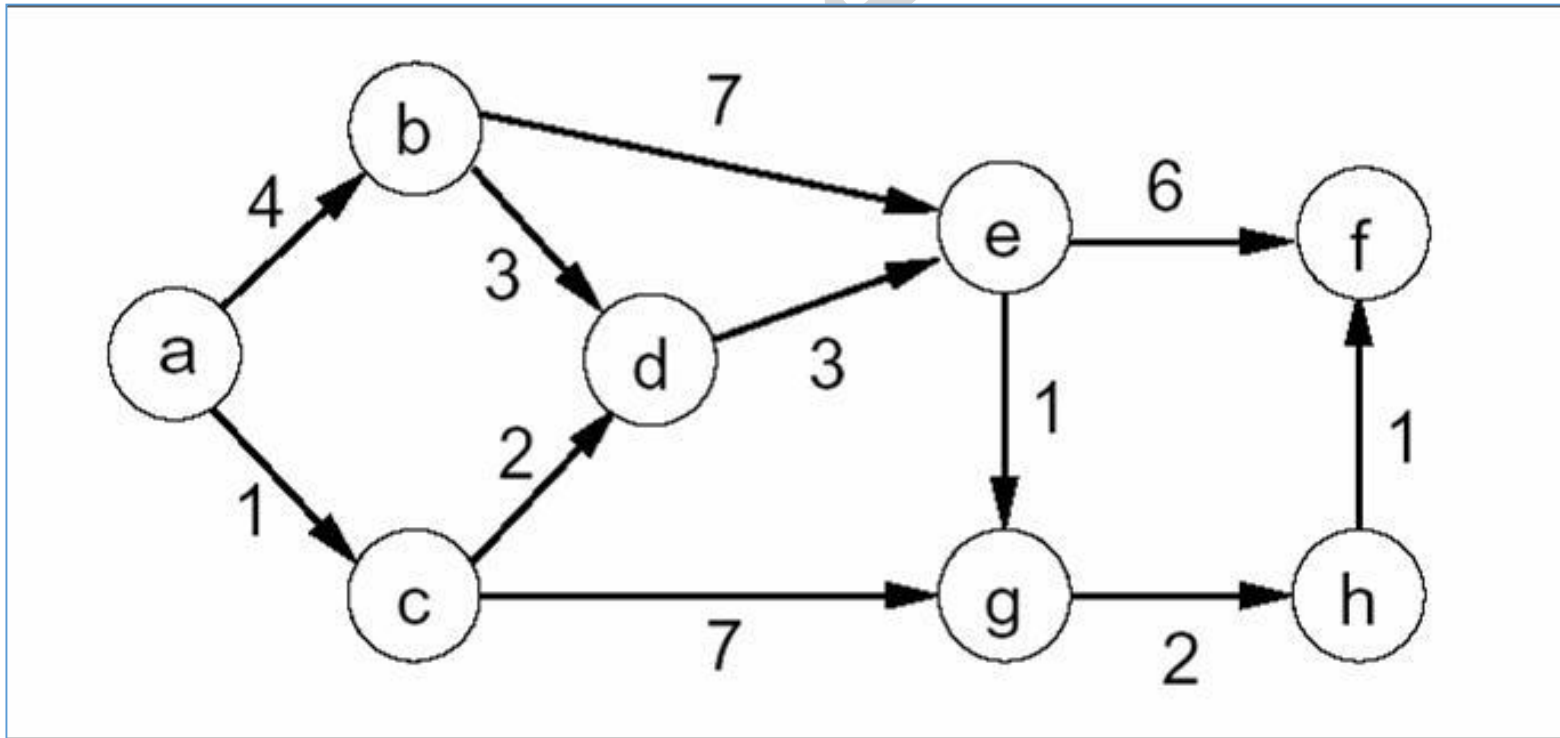
Prim's Algorithm Example 4

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.



Prim's Algorithm Example 5

Example: Apply Prim's algorithm for the following graph to find the minimum spanning tree.



Single Source Shortest-Paths (SSSP)

Single Source Shortest-Paths

How can we find the shortest route between two points on a road map?

Model the problem as a graph problem:

- Road map is a weighted graph:

vertices = cities

edges = road segments between cities

edge weights = road distances

Goal: find a shortest path between two vertices (cities)

Single Source Shortest-Paths Algorithms

SSSP Algorithms

Dijkstra's Algorithm

Bellman-Ford Algorithm

Dijkstra's Algorithm

Given a source vertex (s) from a set of vertices V in a weighted digraph where all its edge weights $w(u, v)$ are **non-negative**, find the shortest path weights $d(s, v)$ from source s for all vertices v present in the graph.

Dijkstra's algorithm solves the single source shortest-paths problem on a weighted, directed graph $G = (V, E)$ for the case in which all edge weights are nonnegative.

Dijkstra's Algorithm

$T = v_0$

Until T spans all nodes **do**

Select nodes $x \in T, y \notin T$

w/min $\text{cost}(x,y) + \text{dist}(v_0,x)$

Add edge (x,y) to T

Return T

Dijkstra's Algorithm

Dijkstra's Algorithm

$T = v_0$

Until T spans all nodes **do**

Select nodes $x \in T, y \notin T$
w/min $\text{cost}(x,y) + \text{dist}(v_0, x)$

Add edge (x,y) to T

Return T

Prim's Algorithm

$T = v_0$

Until T spans all nodes **do**

Select nodes $x \in T, y \notin T$
w/min $\text{cost}(x,y)$

Add edge (x,y) to T

Return T

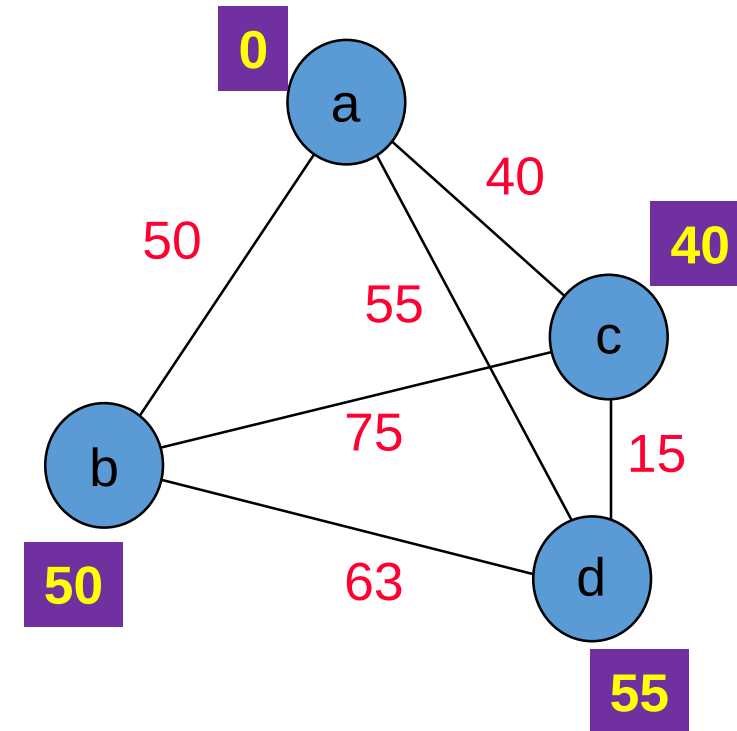
Dijkstra's Algorithm – Example 1

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **a** to all other vertices. Write down all intermediate steps.

Iteration [1]			
a	b	c	d
w			
p			

Iteration [2]			
c	b		d

Iteration [3]			
b			d



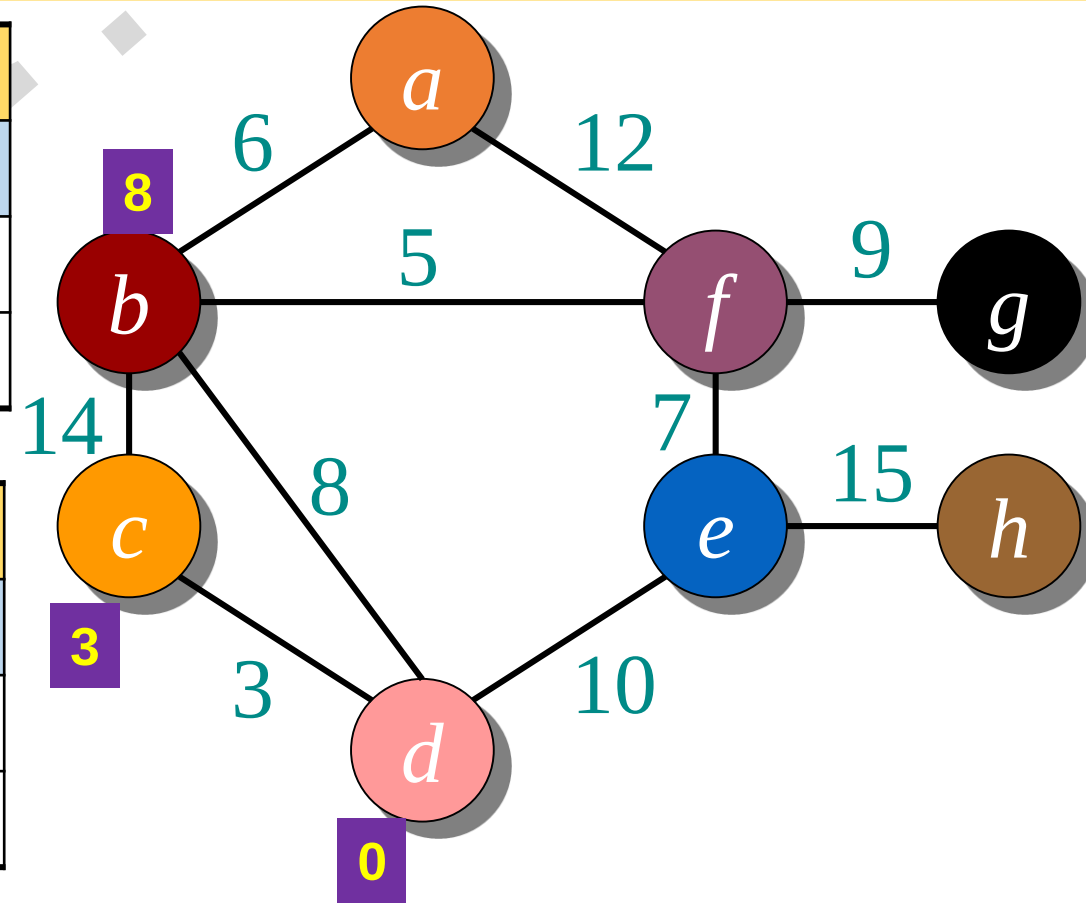
Graph

Dijkstra's Algorithm – Example 2

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **d** to all other vertices. Write down all intermediate steps.

Iteration [1]							
d	a	b	c	e	f	g	h
w							
p							

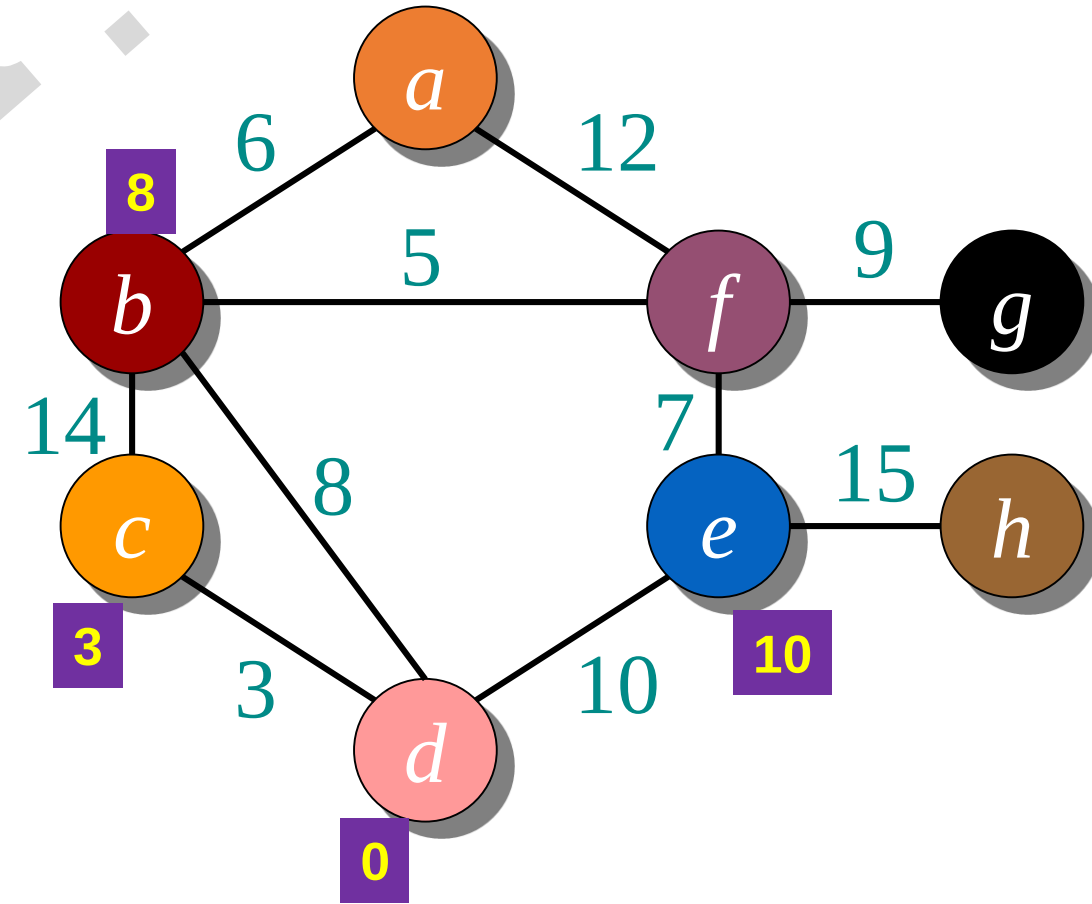
Iteration [2]							
c	a	b		e	f	g	h
p							



Dijkstra's Algorithm – Example 1

Iteration [2]							
c	a	b		e	f	g	h

Iteration [3]							
b	a			e	f	g	h



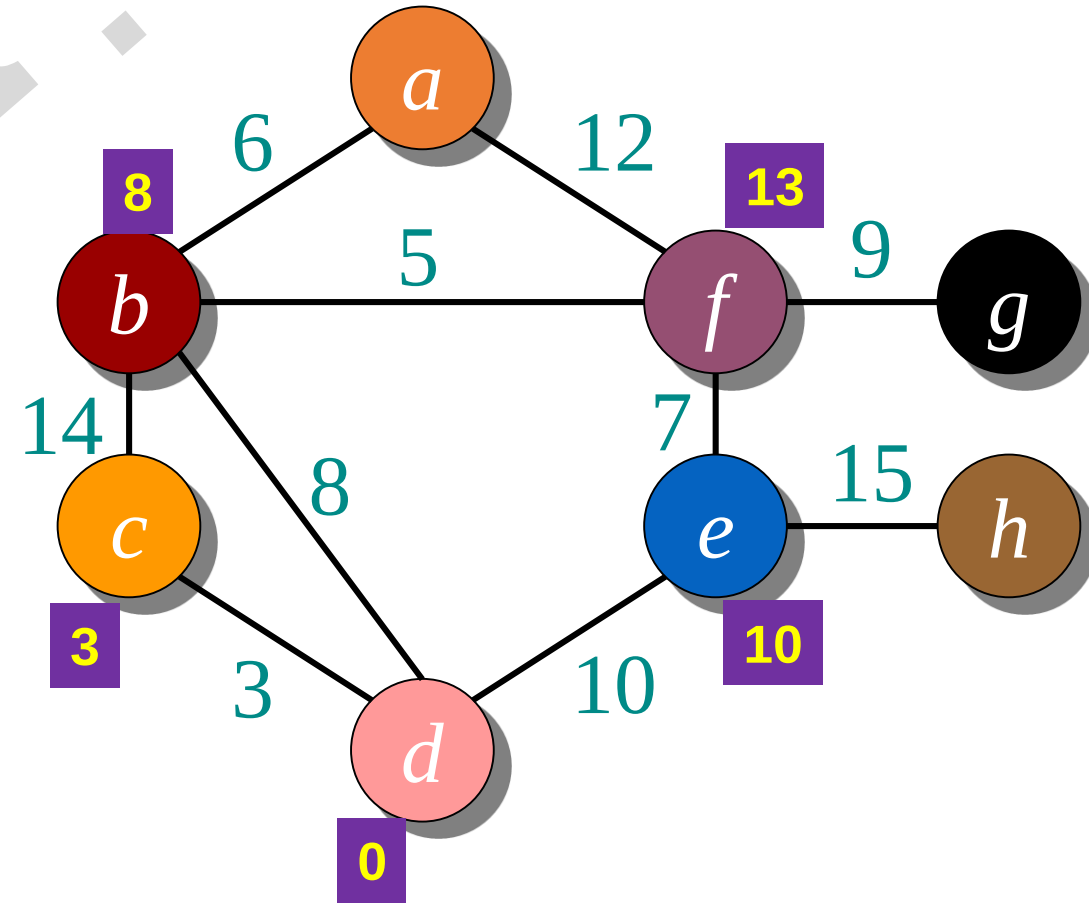
Dijkstra's Algorithm – Example 1

Iteration [3]

b	a			e	f	g	h

Iteration [4]

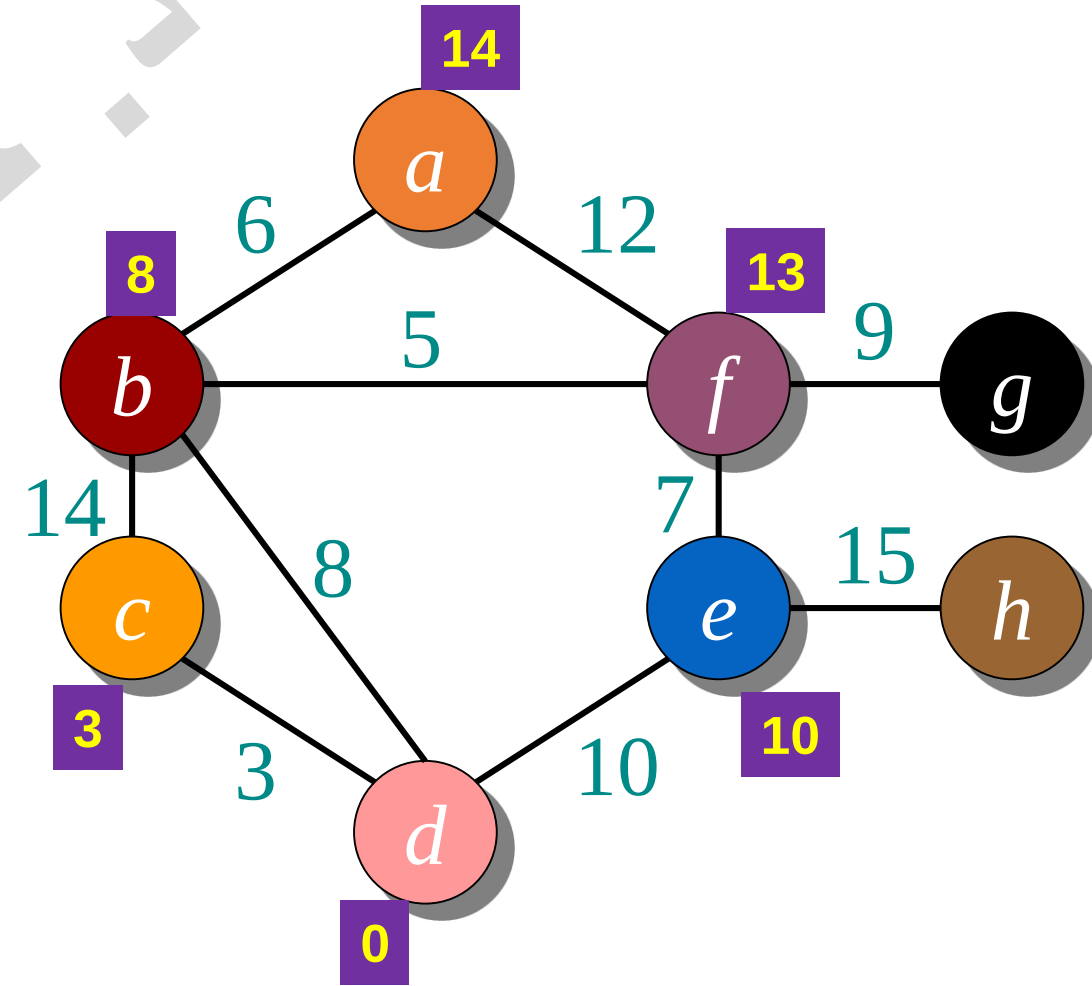
e	a				f	g	h



Dijkstra's Algorithm – Example 1

Iteration [4]							
e	a				f	g	h

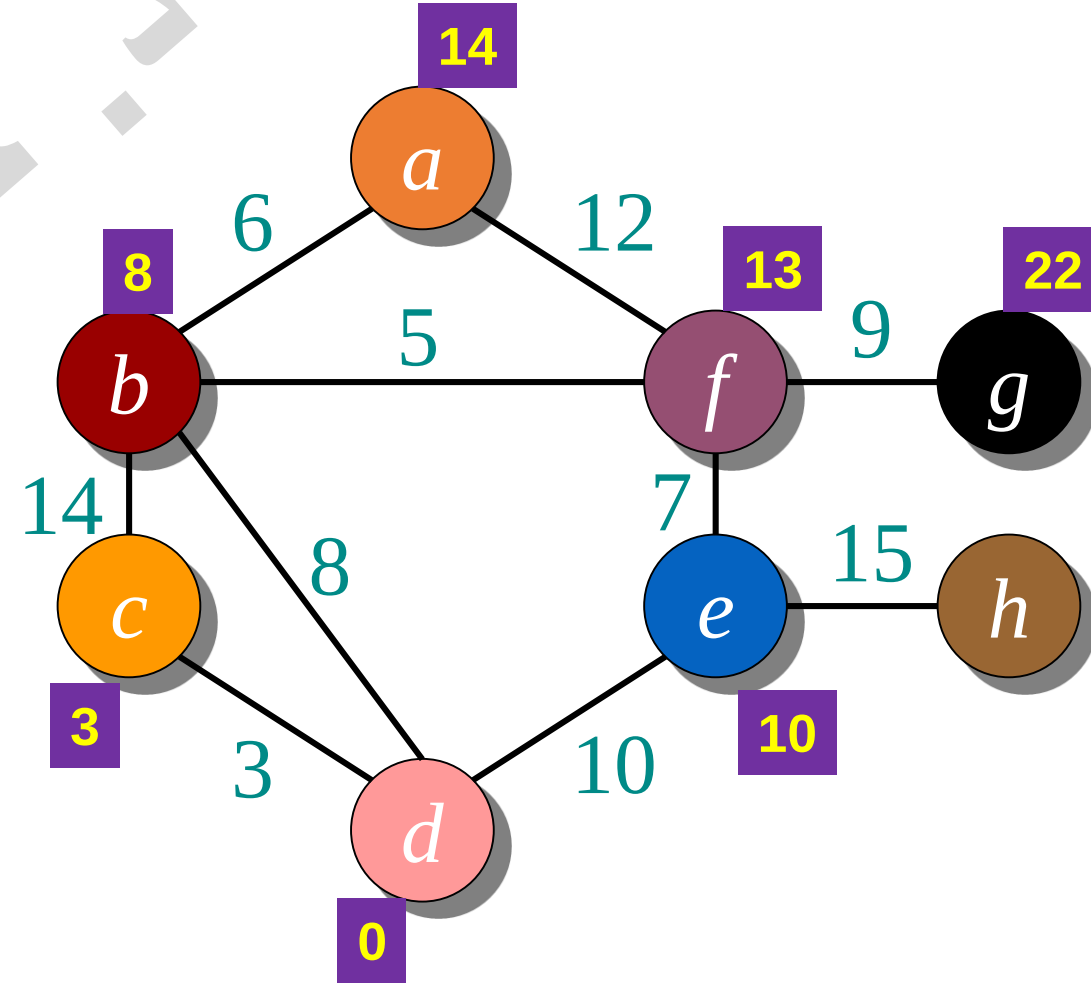
Iteration [5]							
f	a					g	h



Dijkstra's Algorithm – Example 1

Iteration [5]							
f	a					g	h

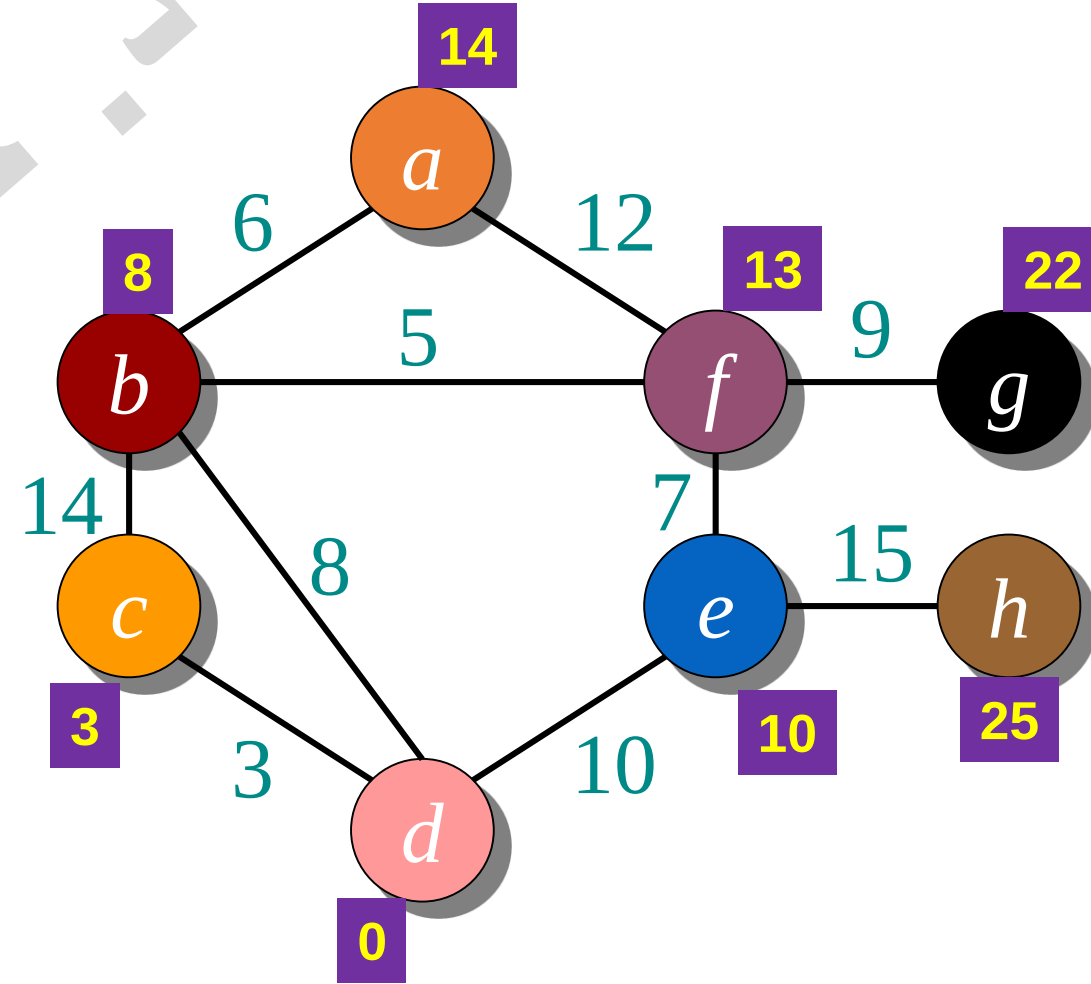
Iteration [6]							
a						g	h



Dijkstra's Algorithm – Example 1

Iteration [6]							
a						g	h

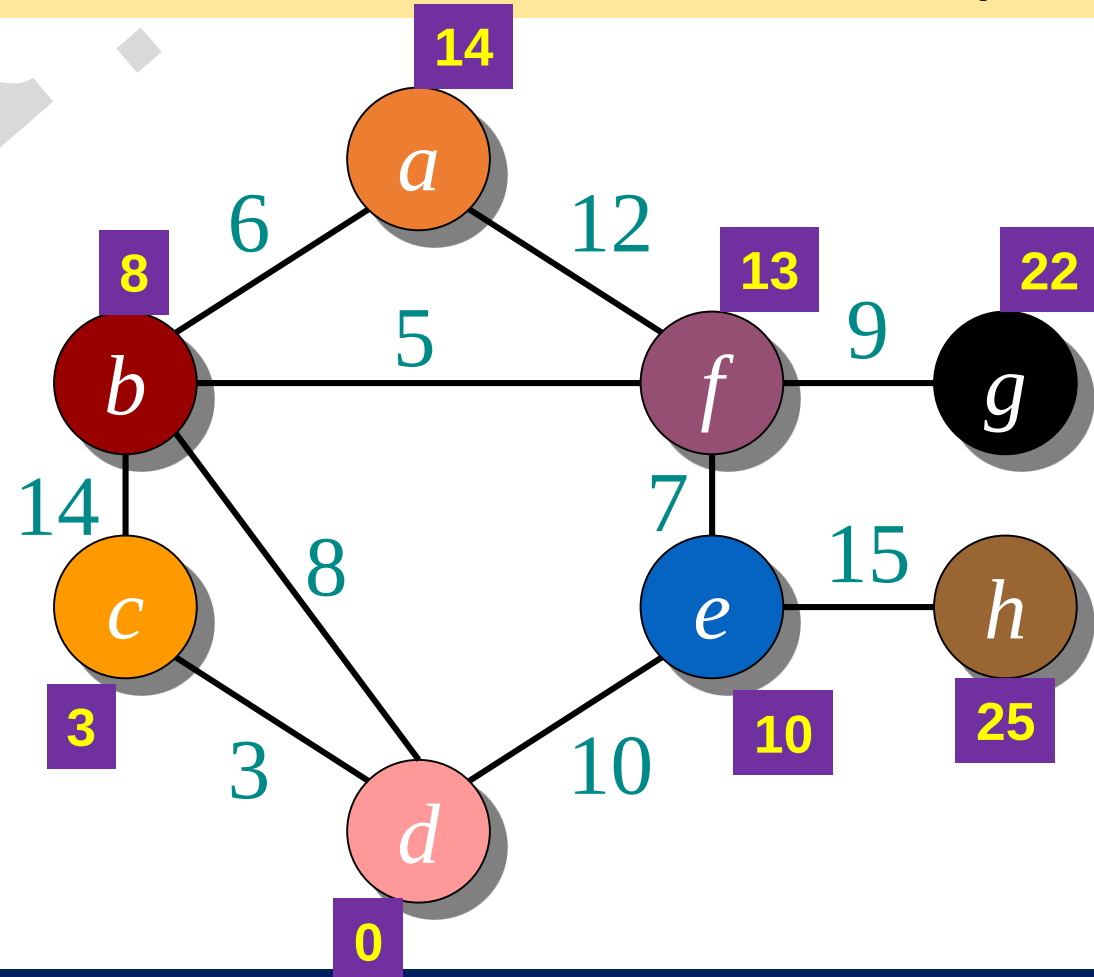
Iteration [7]							
g							h



Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

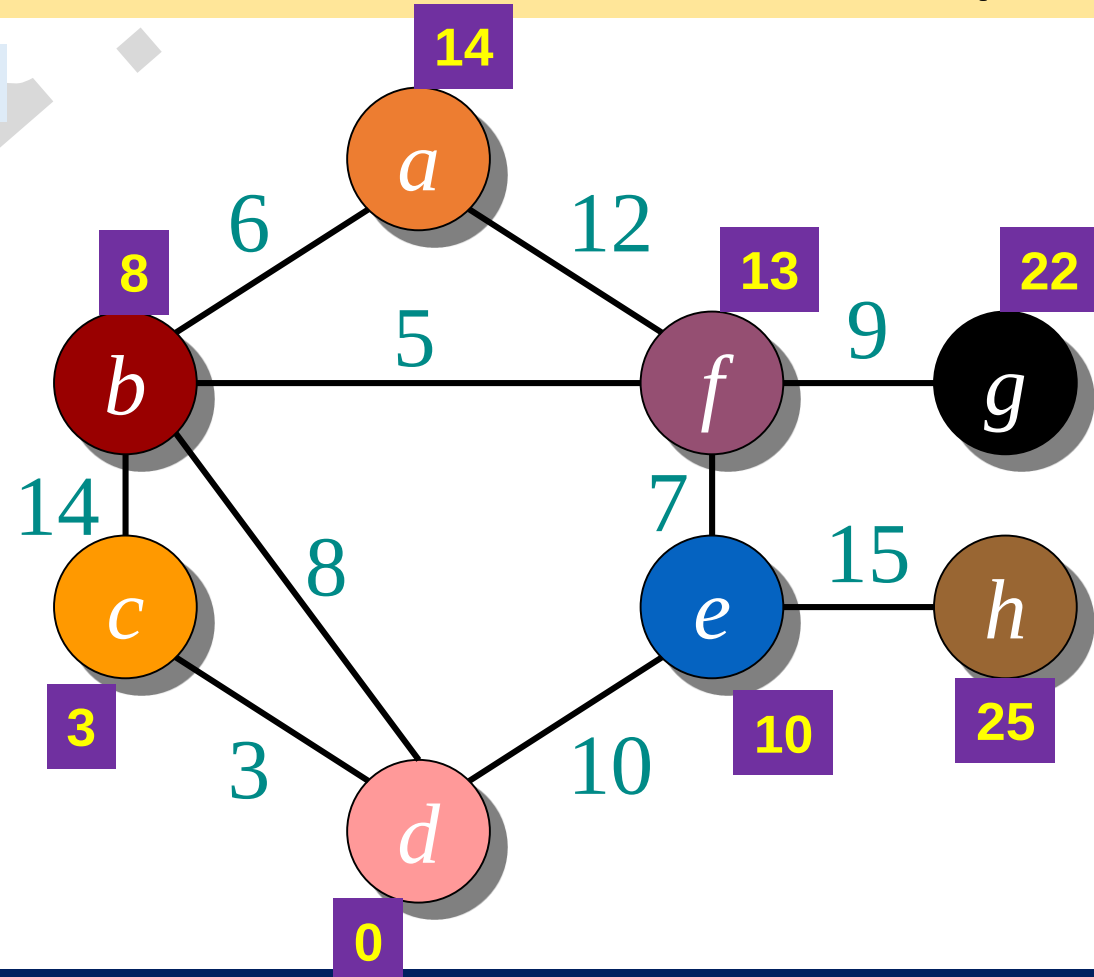
1- List the vertices in the order in which they are deleted from the priority queue and for each the shortest distance from A to the vertex.



Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

2- Draw the shortest paths tree that results

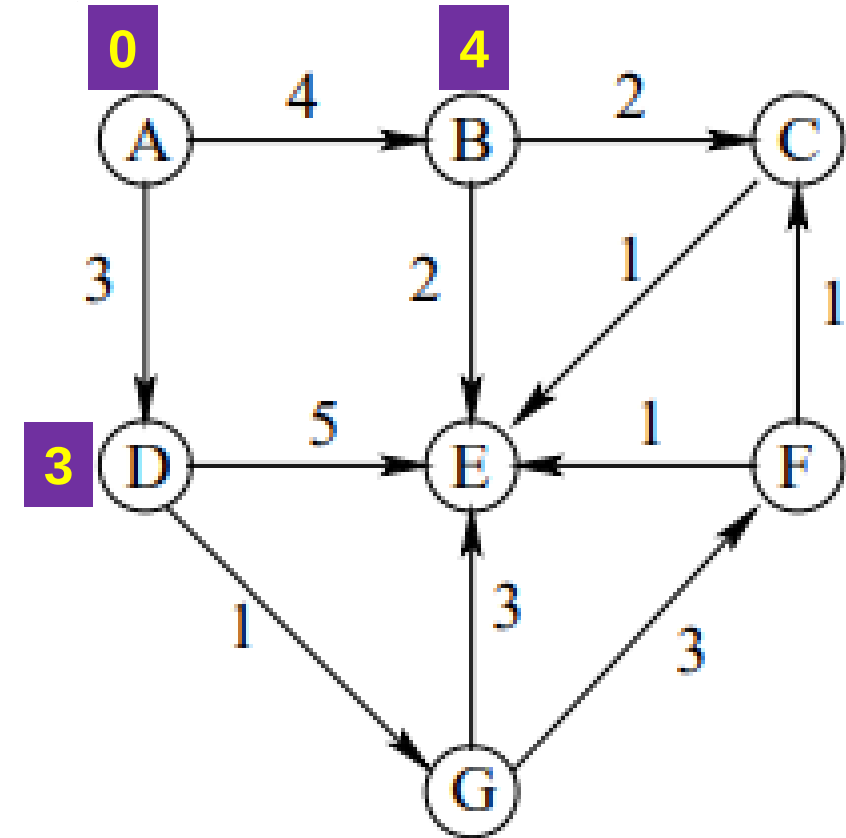


Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

Iteration [1]						
A	B	C	D	E	F	G

Iteration [2]						
D	B	C		E	F	G

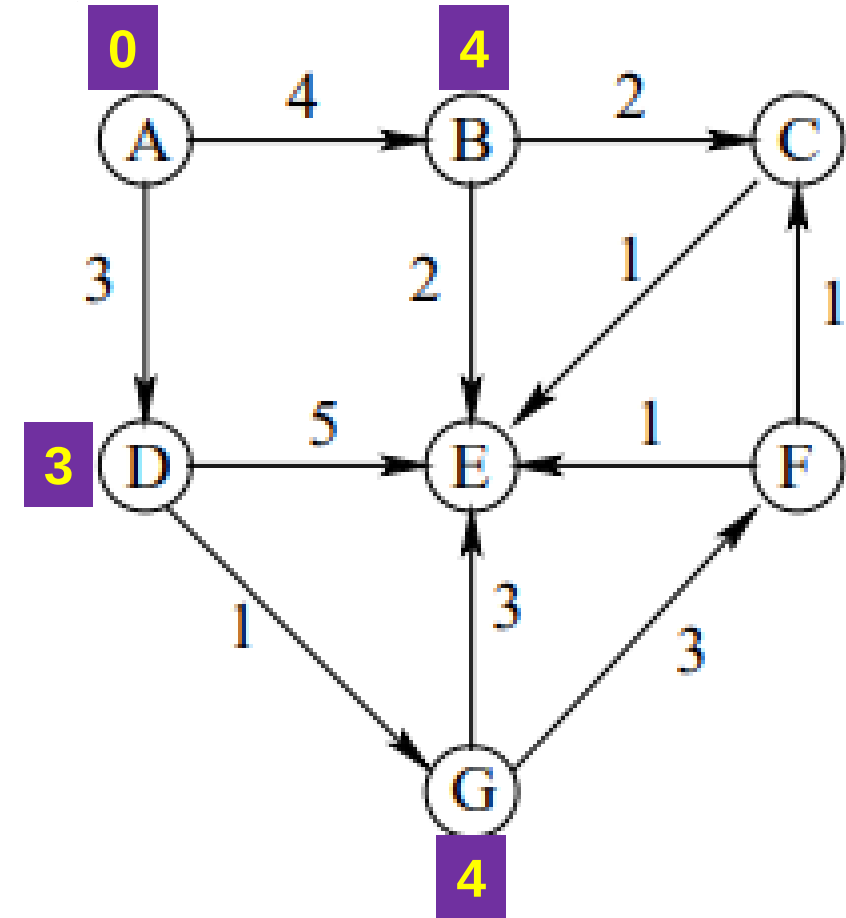


Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

Iteration [2]						
D	B	C		E	F	G

Iteration [3]						
B		C		E	F	G

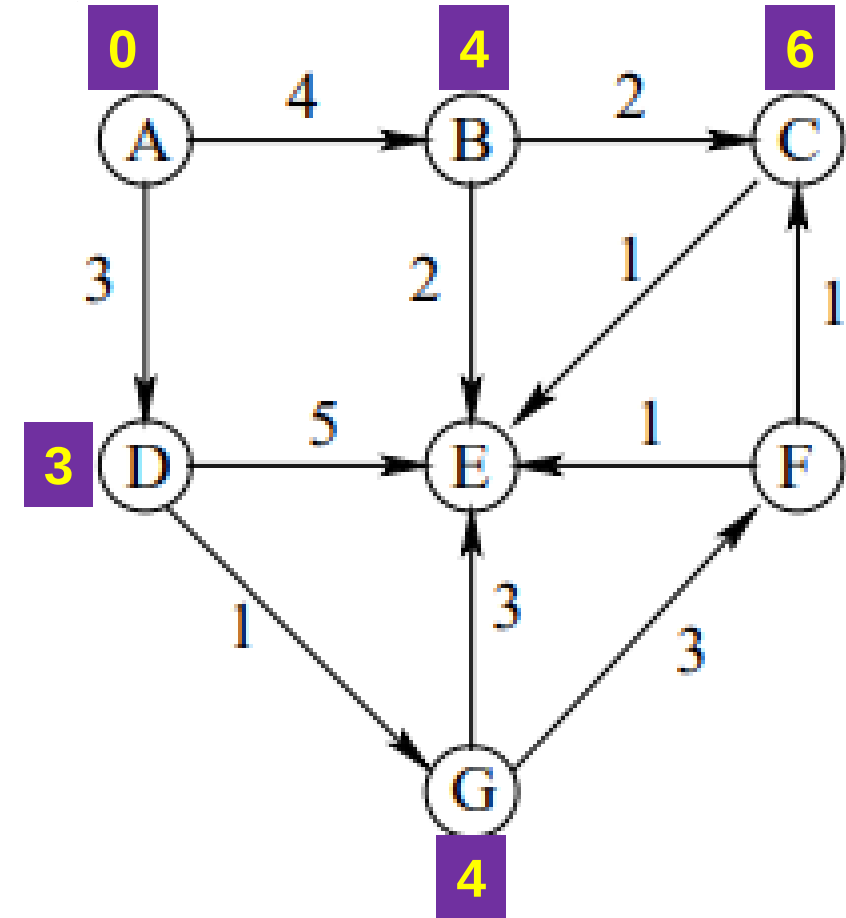


Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

Iteration [3]						
B		C		E	F	G

Iteration [4]						
G		C		E	F	

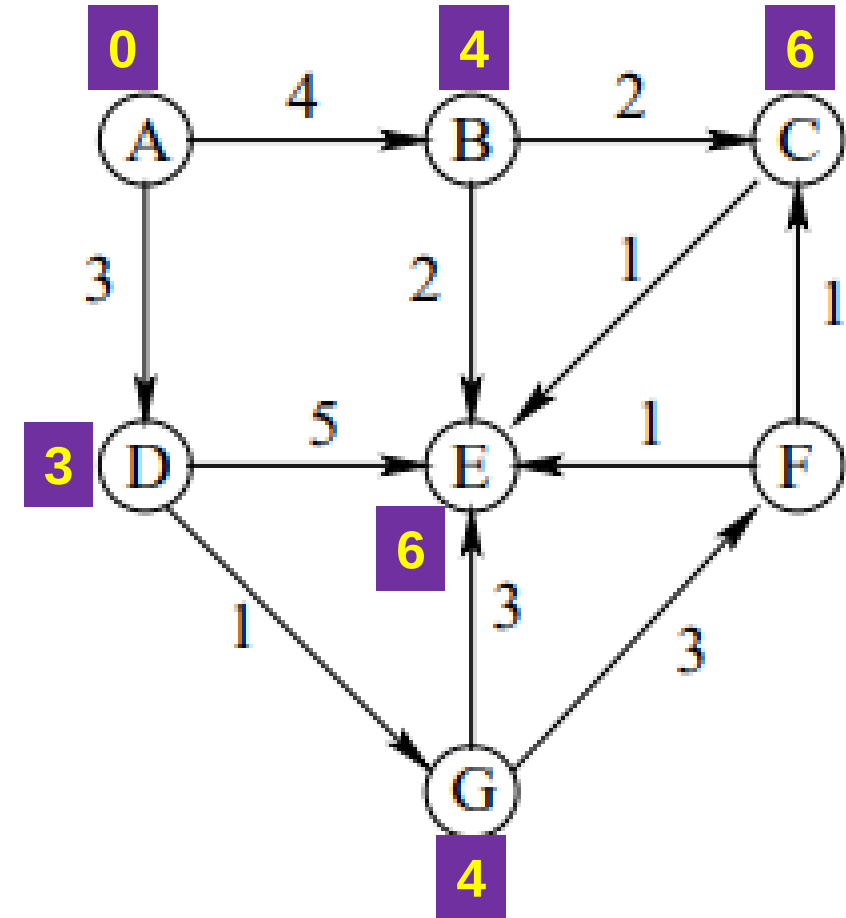


Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

Iteration [4]						
G		C		E	F	

Iteration [5]						
C				E	F	

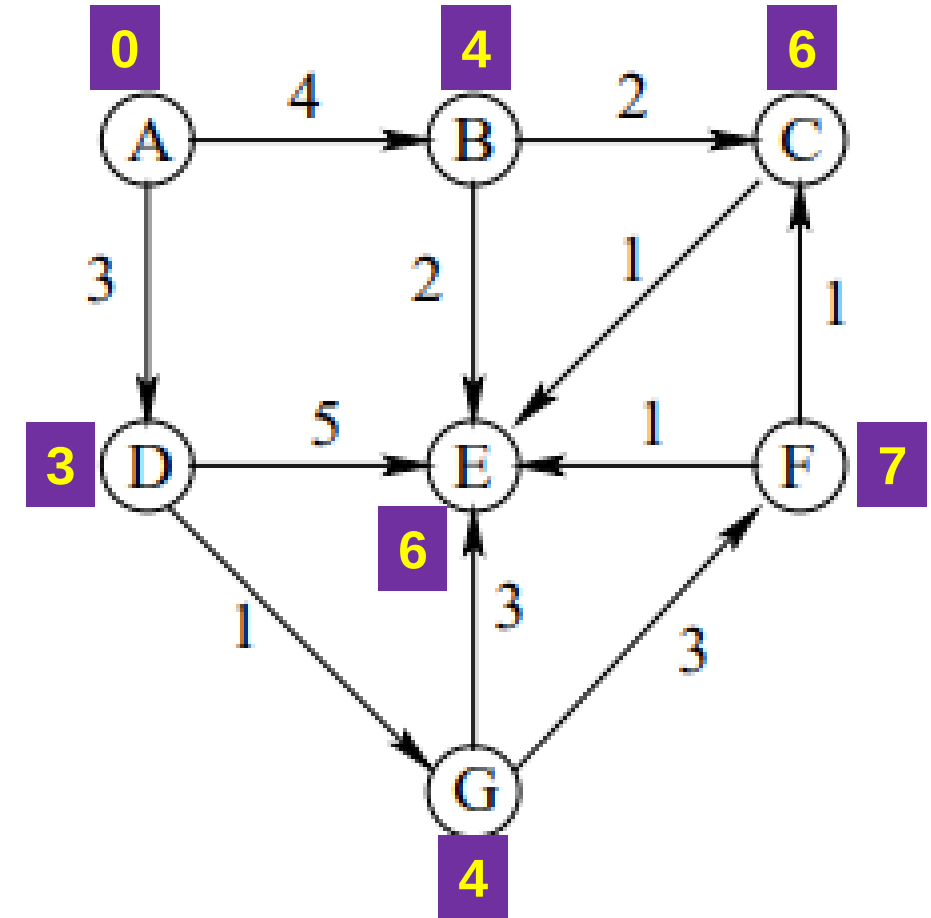


Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

Iteration [5]						
C				E	F	

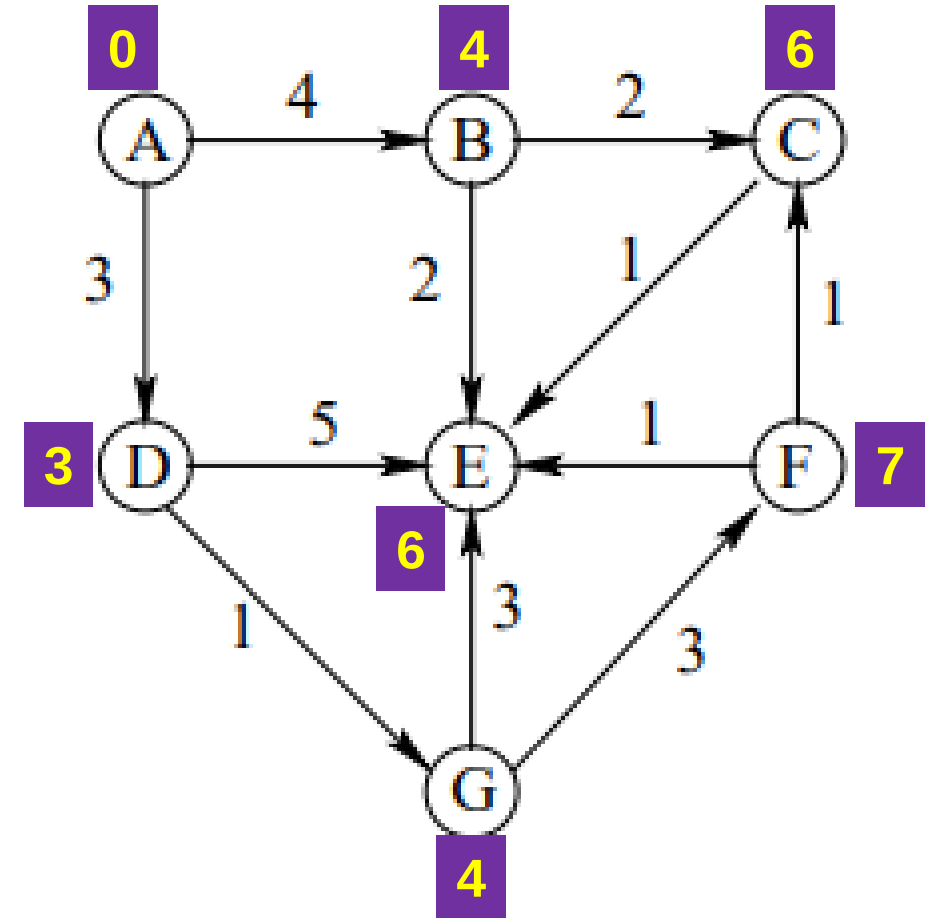
Iteration [6]						
E					F	



Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

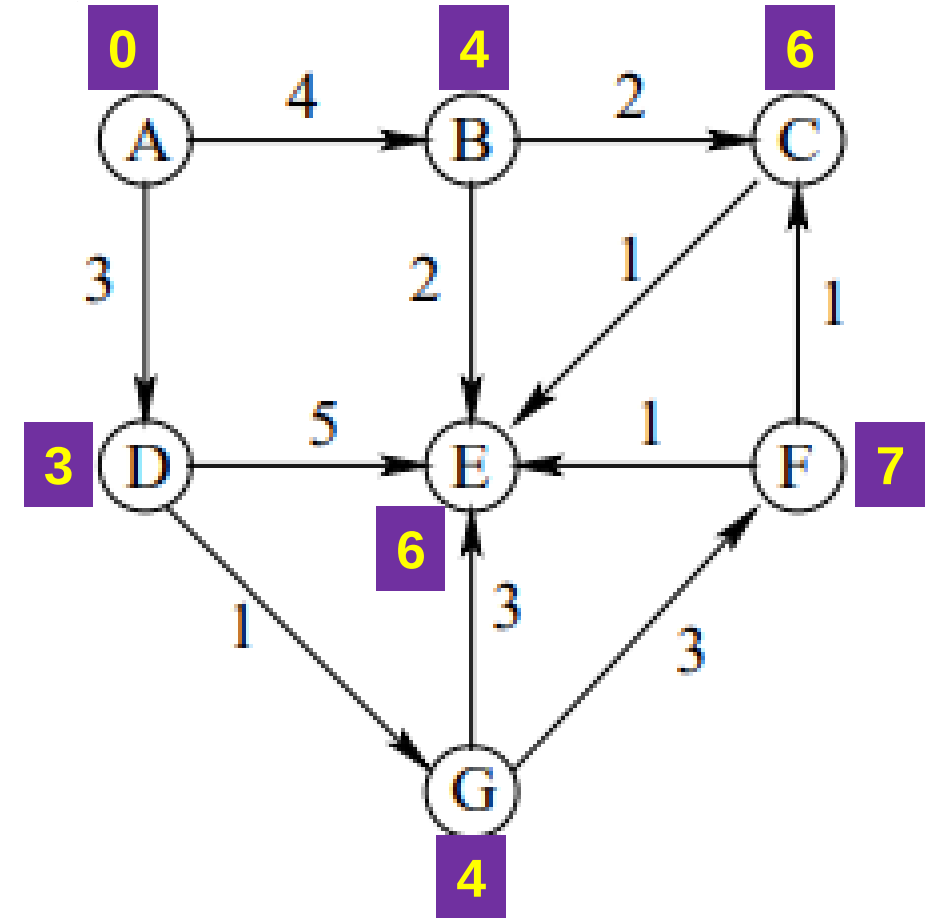
1- List the vertices in the order in which they are deleted from the priority queue and for each the shortest distance from A to the vertex.



Dijkstra's Algorithm – Example 3

Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **A** to all other vertices. Write down all intermediate steps.

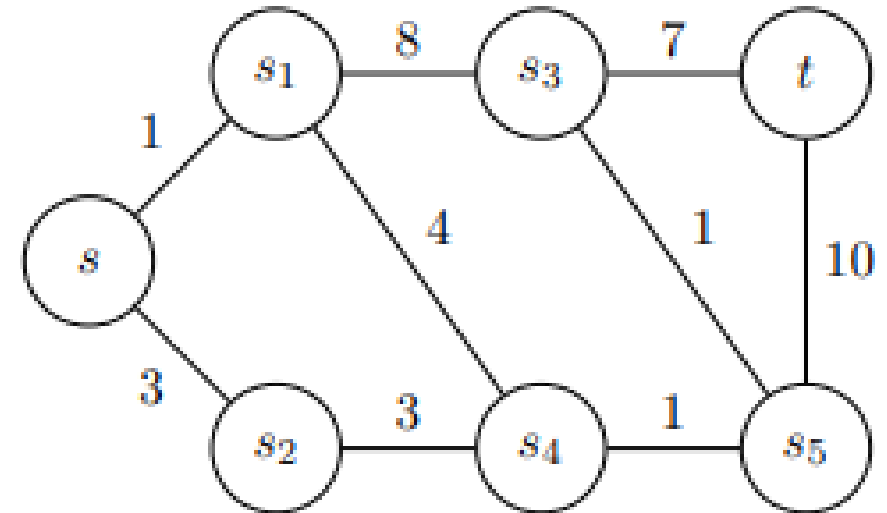
2- Draw the shortest paths tree that results



Dijkstra's Algorithm – Example 3

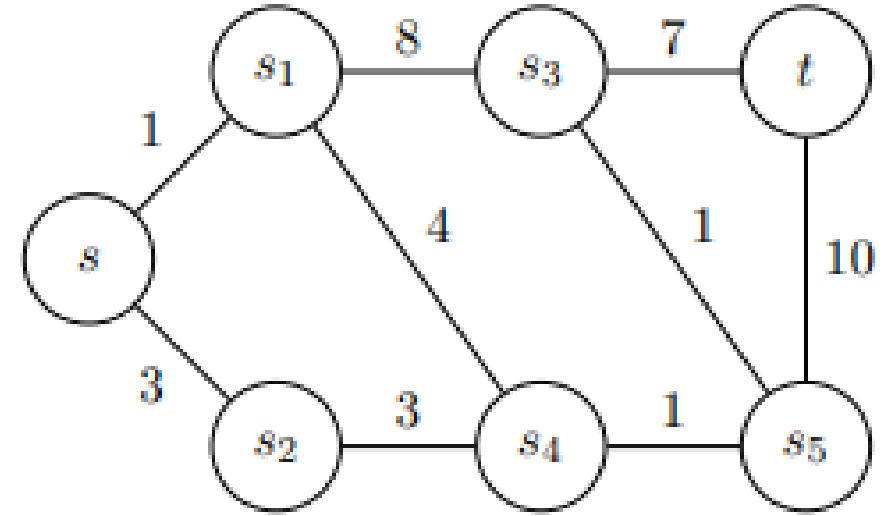
Example: Apply Dijkstra's algorithm for the following graph to find the shortest path (and its cost) from **s** to **t**. Write down all intermediate steps.

Iteration [1]						
s	s₁	s₂	s₃	s₄	s₅	t
w						
p						



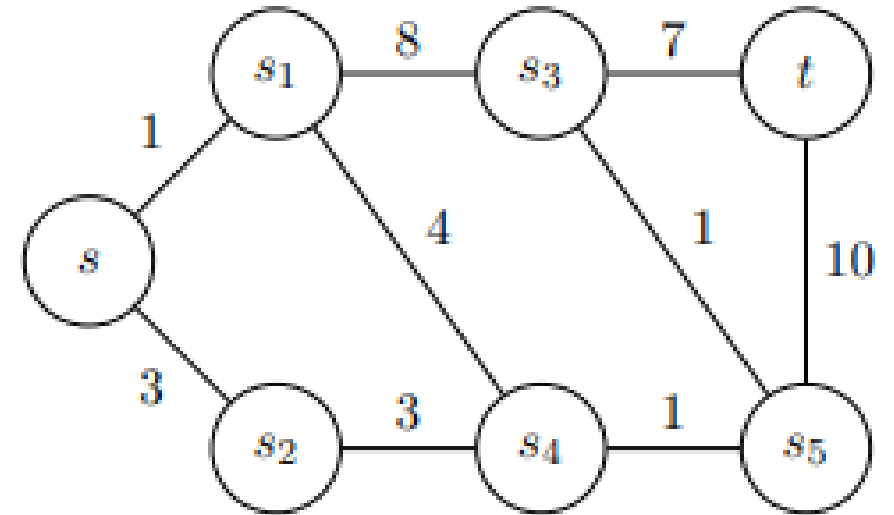
Dijkstra's Algorithm – Example 3

Iteration [2]						
s	s ₁	s ₂	s ₃	s ₄	s ₅	t
w						
p						



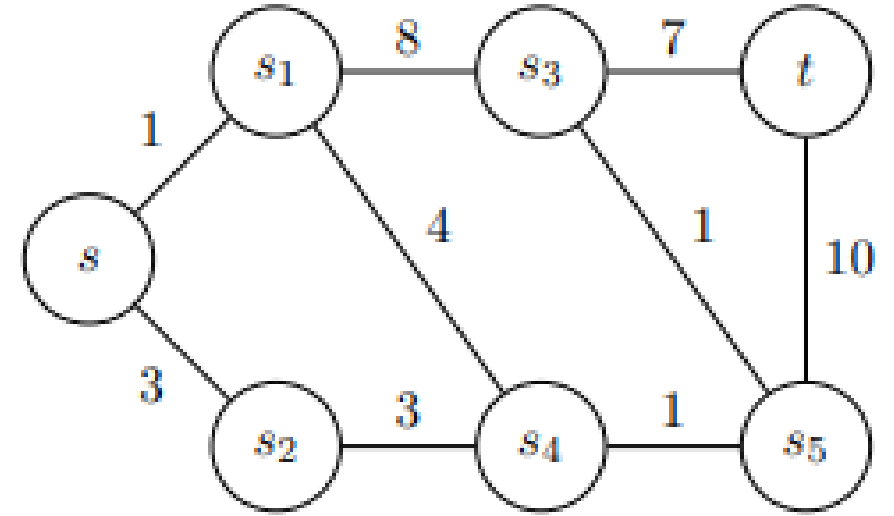
Dijkstra's Algorithm – Example 3

Iteration [3]						
s	s ₁	s ₂	s ₃	s ₄	s ₅	t
w						
p						



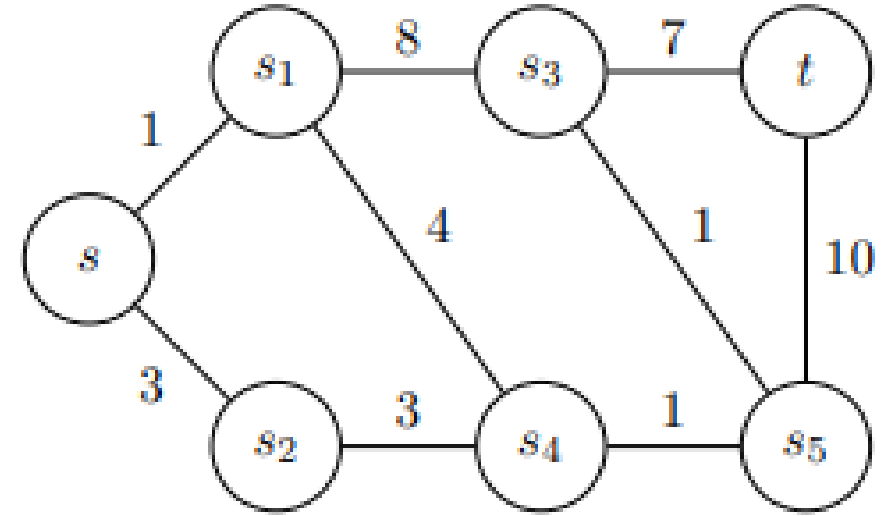
Dijkstra's Algorithm – Example 3

Iteration [4]						
s	s ₁	s ₂	s ₃	s ₄	s ₅	t
w						
p						



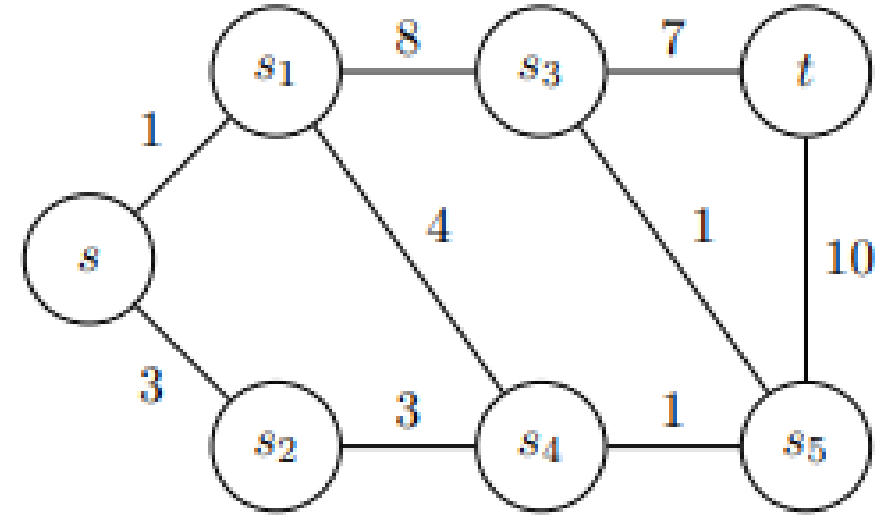
Dijkstra's Algorithm – Example 3

Iteration [5]						
s	s ₁	s ₂	s ₃	s ₄	s ₅	t
w						
p						



Dijkstra's Algorithm – Example 3

Iteration [6]						
s	s ₁	s ₂	s ₃	s ₄	s ₅	t
w						
p						



Check - Chapter Learning Outcomes

- Define a graph as a set of vertices and a set of edges.
- Determine the size of a graph.
- Distinguish between directed and undirected graphs.
- Use graphs to model situations in which the vertices represent objects, and edges represent a particular relationship between objects.
- Understand the following graph terminologies: (weighted graph, loop graph, multigraph, pseudograph, regular graph, complete graph, null graph).
- Find the degree of each vertex in directed and undirected graphs.
- Understand the handshaking theorem.
- Distinguish between connected and unconnected graphs.
- Apply graph traversal algorithms.
- Apply topological sort algorithm.
- Apply minimum spanning algorithms (Kruskal and Prim).
- Apply Dijkstra's algorithm to find the Single Source Shortest-Paths.

End of Chapter

5



The End

د. حسن المعيدي